Decisions or Tokens? What Moves an Offline ARC-AGI-3 Agent, and How to Tell

Anonymous Authors

Abstract

An ARC-AGI-3 agent plays unseen interactive games and is scored on the levels it clears and the actions it spends. We forked Daniel Franzen's public notebook, an open-weight model on one offline GPU for nine hours, and asked which changes move its score, and how one can tell. Identical code cleared 123 and then 101 levels, so we measured the noise first: six identical pairs show that two runs per arm resolve only about 23 levels. Serving gains were too small to see; a token-budget curve puts the agent near its ceiling, though the clock prices tokens about three times higher on score. Re-sending the turn's opening message less often coincided with 12.5 more levels across days; an interleaved test, with rules fixed in a dated file and amended once before any run began, did not reproduce it. Five harness changes passed their gates and four shipped; their level leads are inside the noise, and a scroll detector showed no gain. Fine-tuning on the agent's own cleared levels was followed by fewer levels, also against two same-card controls, though runs were few, on different services and days, and not interleaved. Our best public score is 40.98, the maximum of many daily draws; the private score is not yet known.

Reading guide. Pages 1 to 20 are the paper. The references follow, and the appendices (from page 26) are reference material: the full claims table, run records and how to regenerate everything. **Terms.** An *arm* is one version of the agent under test, run several times. A *draw* is one scored run on the leaderboard, one a day. A *variant* is a build meant to play like the shipped one: a draft model with a retuned vocabulary map, or a keepalive that acts only while the server boots.

1 Introduction

ARC-AGI-3 is a benchmark of interactive games [ARC Prize Foundation, 2026a]. Each game is a grid of coloured cells with several levels (six to ten in the public games) and no instructions: the agent sees the board as an image, sends actions such as a move or a click, and has to work out the goal from what changes. A level scores higher the fewer actions the agent needs compared with a human who played it, and later levels weigh more (Sec. 2). The competition plays 110 hidden games offline, on one 96 GB GPU, in nine hours; each team gets one scored run a day, which we call a *draw*, and a draw returns a score and no log [ARC Prize Foundation, 2026b].

We ran the same program twice on the 25 public games, which the organisers call the *demonstration set*, and it cleared 123 and then 101 levels. Nothing in the code differed. Almost every change we tried was smaller than that gap, so this paper is first about telling a change from noise, and then about what the instruments showed.

We asked whether, on one offline GPU with a fixed clock, the agent was limited by its decisions or by its tokens. The answer so far is partial: serving gains were too small to see, the one level gain we found across days did not reproduce when its arms were interleaved, four changes to what the model is shown, when it must act and what it can check fired as designed on runs too few to resolve a level gain, and teaching the model from its own wins lowered levels.

The problem is hard for a plain reason: a run's score sums nearly two hundred all-or-nothing level outcomes, and one run per arm cannot tell a real gain from their spread. Published ARC-AGI-3 agents use hosted models on the demonstration set [Lehmann et al., 2026, Skoutnev et al., 2026], and the open-weight write-ups we read report no measurement protocol [Bessis et al., 2026, Franzen, 2026b]. One public repository, "Before Tuning the Harness", repeats identical submissions before tuning and accounts cost per request [guoyi12311125-netizen, 2026]; we read its page and README only and did not check its model, and we price tokens in levels. Our entry forks Daniel Franzen's public notebook [Franzen, 2026a]; Sec. 2 separates what we inherited, borrowed and built. We make five contributions.

1. **A measurement kit for a noisy fixed-budget agent (Sec. 3).** Six identical pairs give a run-to-run SD of 8.25 levels, so two runs per arm resolve about 23 levels; a wave of twelve runs per arm made side by side would resolve about 9. We also used concurrent replication on rented copies of the scoring GPU, which match Kaggle within 1% on serving counters, and once a paired decision test, which replays single decisions with and without a change against the model's own sampling noise and is not yet validated against a lever with a known level effect.
2. **A token-budget curve that prices throughput (Sec. 4).** Built from recorded level attempts, it puts the agent at about 90% of its ceiling, the score that unlimited tokens would give at the depths we recorded. Our serving speed-ups are then worth about 4 levels (2 to 10 across our estimates), below what two runs per arm can see; the price is a simulator's and is itself uncertain. Read from the clock, the last tenth of the window earns 7% of the points, about three times the curve's price on score.
3. **A context lever that failed an interleaved test (Sec. 5).** Yielding a turn after 8,192 generated tokens, four times Franzen's threshold, re-sends the turn's opening message and board image about 70% less often. Across days it coincided with 12.5 more levels. Interleaved on builds that differ in one line, six runs per arm, it scored 2.5 fewer (p 0.42), which excludes the earlier gain but not small gains: drift between days and extra tokens are the candidate causes, and only the interleaved design could show that the gain did not reproduce (Sec. 5).
4. **A diagnosis-first harness program with kill rules (Sec. 6).** Stuck plays stop acting, write off object types untested and misread what the harness shows; 27% of never-cleared levels hold three or more idle turns in a row, against 8% of cleared ones. Five changes, each gated by a rule in a dated file written before its runs, fired as designed: a stall check, a corner-joined view of objects, a scroll detector, tool fixes that keep a timed-out snippet's output, and a replay check that tests the model's simulator and goal guesses against the level's history at no cost in actions. The first two led a pooled control by about five levels and the replay check by about three (over twice as much against the two Kaggle controls alone), which three runs cannot resolve; the scroll detector gave no gain and was not shipped; the replay check caught a wrong prediction in 42 of 85 replays. None adds tokens, so this is evidence about decisions, not budget. A research pass reads why such changes are the place to look: on long levels most of the clock goes to finding the rule or the goal, not to carrying out a plan.
5. **Two fine-tuning results and what came with them (Sec. 7).** (a) Rejection-sampling fine-tuning on the agent's own cleared levels lowered validation loss by 13.9% and lowered levels. Its three runs sit at or below the two lowest of 18 runs without it (rank p 0.004). Two later same-card controls without it cleared more, and the adapter's three runs are the lowest of those five (rank p 0.10). The adapter and the controls ran on different services and days, not interleaved, and the controls are two, one restarted after a preemption. The agent already clears 91% of the training games' levels, so the filter kept nearly everything, slow wins included; the adapter's reasoning per action fell toward the training games' level, and its resets and deaths rose. This chain is our best reading of why; two alternatives are untested.

   (b) In reinforcement learning on this stack the training forward pass is not deterministic: two identical passes agree exactly on about 12% of tokens and differ by 0.37 nats per token (RMS). A drift guard must therefore be read net of the stack's own floor; read raw, our predeclared guard would have discarded the run on noise. Round A shipped no adapter: a first-wave lead did not replicate (Sec. 7.2).

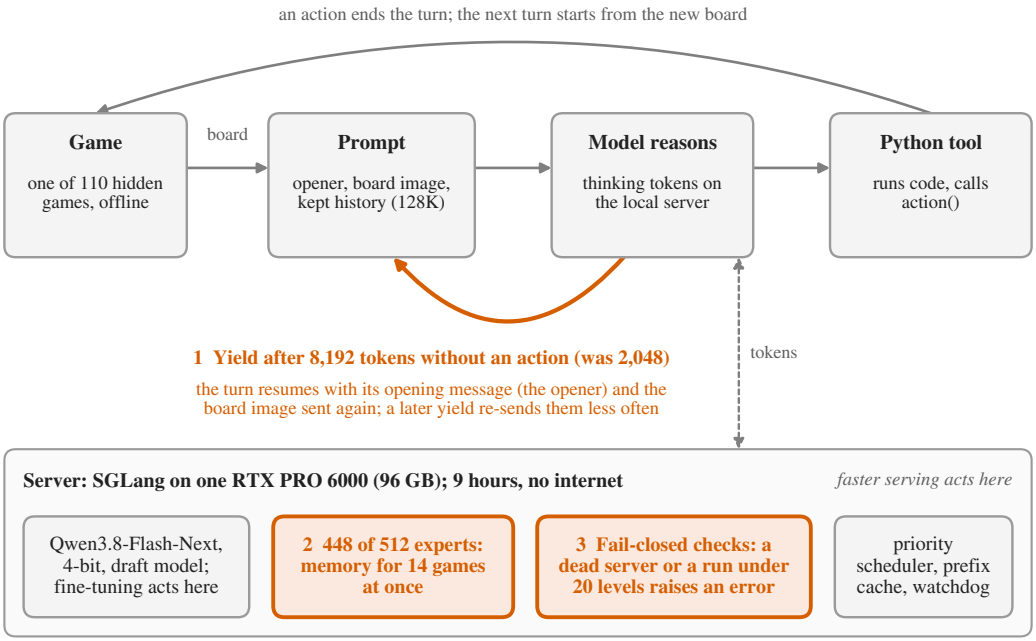


Figure 1: One turn of the agent, and where each lever acts. A turn ends when the model acts. A yield ends a request after a set number of tokens generated without an action and resumes the turn with the opener (the turn's opening message) and the board image sent again. Accent: the three key decisions of Sec. 2 (the 8,192-token yield; expert pruning that frees memory for 14 concurrent games; the fail-closed checks, which have not yet fired on a scored run); gray: inherited parts. Schematic.

What we do not claim. We attribute no board draw to any lever, and no level gain to the yield or to the harness changes. Numbers from the demonstration set are development numbers, not held out. Our 40.98 is the best of 28 live draws; the agent's seventeen valid draws average 34.59. We make no claim about whether the model's licence is eligible for a prize.

Result. Public score **40.98** on half of the 110 hidden games: Kaggle submission (withheld), drawn 2026-10-20. It is the best of 28 live draws since the fork; the agent's seventeen valid draws (errors, zeros, adapter draws and field-wide failure days excluded) span 28.97 to 40.98 (mean 34.59). The private score is pending.

2 Setting and system

The agent. Our entry is Daniel Franzen's Milestone 2 notebook [Franzen, 2026a,b] with changes of ours (Appendix A). The model is Qwen3.8-Flash-Next [Qwen Team, 2026], a mixture-of-experts model with about 6B active parameters, in Intel's 4-bit weights [Intel, 2026], served offline by the Pennyroyal build of SGLang [Pezzulli, 2026, Zheng et al., 2023] with a multi-token-prediction drafter [albucino, 2026]. The harness is Tufa Labs' Duck [Bessis et al., 2026] as patched by Franzen: the model reasons, runs Python in a sandbox through one tool, and acts by calling `action()`; each new board arrives as an enlarged image that marks the cells that changed, and the reasoning stays in a history of up to 128K tokens (Figure 1).

A turn and its yield. A *turn* starts when a new board arrives and ends when the model acts; inside it the model may call its tool many times, each call a server request that continues the same context. The harness also ends a request once the model has generated a set number of tokens without acting, the *yield*, and resumes the same turn with a new request that sends the turn's opening message (the *opener*) and the current board image again. Franzen's notebook yields after 2,048 tokens and ours after 8,192 (Sec. 5).

Score. A game has n levels and level l has weight l. An agent that clears k levels scores

$$G = 100\,\frac{W(k)}{W(n)}\,\min(1,\bar{e}), \qquad \bar{e} = \frac{1}{W(k)}\sum_{l\le k} l\,s_l, \tag{1}$$

where $W(m) = m(m+1)/2$ and $s_l = \min\big(1.15, (h_l/a_l)^2\big)$ compares the agent's actions a_l on level l with the human baseline h_l. The first factor is the completion share, with unreached levels counted as zero; the second is efficiency. Equation (1) restates the published scorer and matches its code on every recorded play we checked. Every action sent to the game counts and thinking is free, so completion carries the score: nearly all the points a run loses are levels never reached, not levels cleared slowly.

Development runs. We develop on the demonstration set: its 25 games, played by 14 concurrent streams in a shared window of 7,200 s, which gives each game about the token budget it gets in the scored run. We call one such run a *development run*; the 25 games hold 183 levels. The organisers warn that demonstration-set scores do not measure progress [ARC Prize Foundation, 2026a], so we use these runs for noise and price, and the leaderboard is our only held-out number.

Inherited, borrowed, ours. Most of the system is inherited: the model, its weights and drafter, the serving build, the harness, Franzen's patch and his schedule for the scored run. Several of our changes borrow other people's ideas, credited in Sec. 10 and Appendix A. Three decisions matter more than the rest, and Figure 1 marks where they act: the yield after 8,192 tokens; expert pruning, which drops an eighth of the experts in each layer so that 14 games run at once; and fail-closed checks. The checks make two kinds of dead scored run, a server that exits before it is ready or a run under 20 levels at the end, raise an error, which Kaggle does not count against the daily slot. Neither has fired on a scored run, they cannot catch a run killed from outside, and a 0.00 of unknown cause still banked on 2026-10-18 (Sec. 8).

3 Measuring a noisy agent

Identical code, different scores. Six pairs of byte-identical development runs differ by up to 22 levels, and the second run was lower in all six (sign test p 0.031), for a reason we have not found. Their pooled SD is 8.25 levels, with a wide interval (Table 1); around the shared first-minus-second shift it is 5.6. Any comparison run on different days can carry such a shift, which is the strongest argument for running arms side by side.

Eight byte-identical runs on one day, on rented copies of the scoring GPU (six started together, two 2.5 hours later), cleared 115 to 132 levels with no order effect. The leaderboard shows the same thing at a coarser grain: one agent's seventeen valid draws span 12 points (Figure 2).

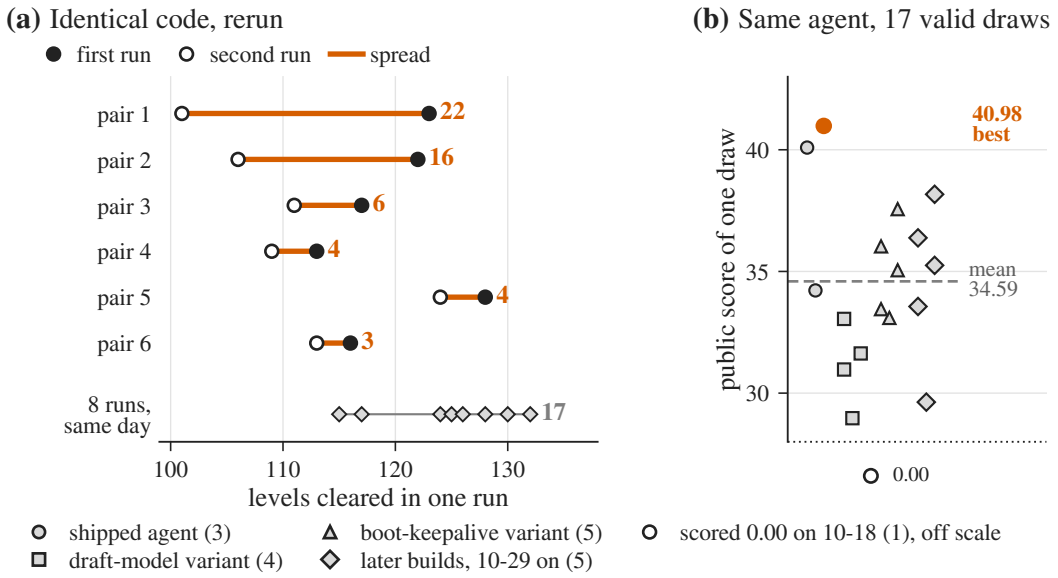
What two runs can see. With run SD σ and n runs per arm, a two-sided test at the 5% level with 80% power detects shifts above

$$\mathrm{MDE}(n) = 2.80\,\sigma\sqrt{2/n}. \tag{2}$$

At the pairs' SD, two runs per arm resolve 23 levels, and a wave of twelve runs per arm made side by side would resolve about 9 (Figure 3). Nearly every lever we built is smaller than 23 levels (Appendix D), so a level count from two runs can veto gross harm and cannot promote.

One rule for intervals. We report every comparison at the identical-run SDs, 5.95 for eight runs on one day and 8.25 for six pairs run one after another, hours to a day apart, two-sided. Bootstrap intervals over runs treat runs on different days as exchangeable, which the shift above argues against, so they come second. Intervals are 95% unless marked.

The gate. Our gate reads mechanism first. (1) Before any run, declare the counter a change acts on, such as tokens per second, draft acceptance length or prefix-cache hit rate, and its bounds; these counters vary 0.4% to 3.2% between identical runs, against 7.3% for levels. (2) Compare with the pooled control, never one control run. (3) Use levels only as a veto: both runs must clear a floor nine levels below the pooled control. (4) Ship a tie when the counter passed and no harm counter, a metric that signals damage such as tracebacks or deaths, moved. The floor is weak by design: it



Mean of the first 12 draws (10-13 to 10-28) and of all 17: both 34.59.

Figure 2: Identical code, different scores. (a) Levels cleared on the 25 demonstration games by six pairs of byte-identical development runs (filled: first run; open: second, lower in all six pairs; number: the spread) and by eight byte-identical runs on one day (diamonds). Higher is better. (b) The same agent's seventeen valid board draws, one point per draw, grouped by build. A *draw* is one scored run. Shipped agent: the configuration pinned on 2026-10-12. Draft-model variant: the same agent with a retuned vocabulary map for its draft model, which should leave the output distribution unchanged. Boot-keepalive variant: the shipped agent with a keepalive that acts only while the server boots. Later builds: the 15-stream and harness builds drawn from 2026-10-29. The one draw that scored 0.00 (2026-10-18, a 3,411-byte output, cause unknown) is the open dot, off the scale. Dashed line: the mean of the seventeen. The mean of the first twelve draws, to 2026-10-28, is also 34.59, by coincidence. 40.98 is the best of these draws and of all 28 live draws since the fork. Raw values; no error bars.

rejects a harmless change 26% of the time and passes a true loss of nine levels 25% of the time, so it screens only gross harm. The draft model's reduced vocabulary passed such a gate, at a gain below its prediction (Table 1).

A paired decision test. A change to what the model sees can be judged one decision at a time: take recorded states, ask the model for two replies with the change and two without, and compare how often replies agree on their first action with how often two unchanged replies agree; the second rate is the model's own sampling noise. We used it on a leaner history rule, which replaces older reasoning and images with short markers and would save about 3.5% of tokens, a lever worth about one level on the curve and invisible to any affordable number of runs. On 117 recorded states the leaner history changed the first decision more than the model's sampling noise on the point estimate, mostly by acting sooner, and so failed its predeclared tolerance (Table 1). The 90% interval of that change contains the tolerance, and as a significance test the result is marginal (two-sided paired p 0.09); we have not checked that first-action agreement predicts levels. The test cost under one GPU-hour, and it cannot see effects that build up over a game.

Concurrent replication. Rented copies of the scoring GPU (Vertex AI jobs on the same RTX PRO 6000) run the unchanged notebook and matched a Kaggle run within 1% on every serving counter, one run each. They can run side by side, so a wave of twelve runs per arm would finish in one round of about two and a quarter hours and would remove the day and order confounds of sequential tests, though not run noise; so far we have run eight identical runs on one day (six started together), not a two-arm wave.

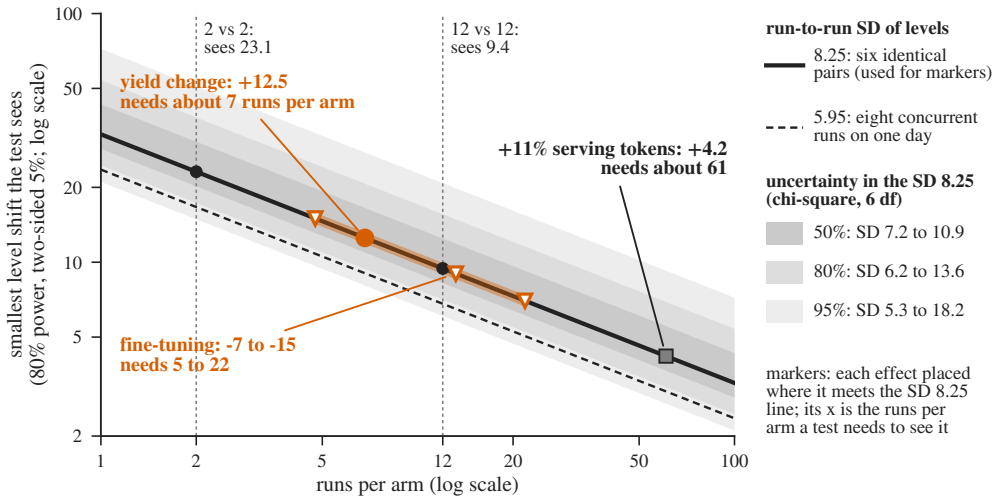


Figure 3: Two runs per arm resolve about 23 levels. Smallest shift a two-sided test sees at 80% power (Eq. 2) against runs per arm, log scales; lower is better. Solid: the six pairs' SD of 8.25, with graded bands for its 50%, 80% and 95% intervals; dashed: SD 5.95 of eight runs on one day. Markers sit where each lever's effect meets the solid line, so their x is the runs per arm a test needs; the serving marker is the curve's prediction, not a measurement.

Table 1: What each instrument resolves, on development runs (25 demonstration games, not held out). MDE: minimum detectable shift in levels (Eq. 2). Costs at list prices; a development run is about 2.2 free Kaggle GPU-hours or $3.88 to $5.45 on a rented RTX PRO 6000. Intervals are 95% unless marked.

Instrument	Noise	Resolves	Cost
Two runs per arm	SD 8.25 levels from six identical pairs (6 df; 5.3 to 18.2)	MDE 23.1 levels	4 runs
Eight identical runs, one day	SD 5.95 (7 df; 3.9 to 12.1); no order shift	MDE 16.7 at 2 per arm, 6.8 at 12 per arm	8 runs
Concurrent wave, 12 per arm (not yet run)	SD 8.25 assumed	MDE 9.4 levels	24 jobs, one 2 h 17 min round, $93 to $131
Serving counters	between-run CV 0.36% (cache hit) to 3.23% (tokens per second); levels 7.31%	draft-vocabulary gate: decode $+1.2\%$ against $+1.4\%$ to $+1.7\%$ predicted	1 run per arm
Paired decision test	two unchanged replies agree on the first action in 0.778 of 117 states	leaner history: agreement -0.071 (90% interval -0.132 to -0.004) against a tolerance of -0.05; paired p 0.09	468 replies, 0.72 GPU-h

4 The token-budget curve: a simulator puts the agent near its ceiling

The curve. We simulate the score the scheduler earns on a budget of B tokens by resampling 1,114 level attempts recorded in eight development runs, seven at the 8,192 yield and one without a token yield. Score rises in proportion to budget, bends, and flattens at a *ceiling*: the expected score with unlimited tokens when each level is redrawn from its recorded attempts (Figure 4), the shape of the plateau in Pourcel et al. [2025]. Levels that no recorded attempt cleared never clear, so the ceiling is an upper bound at the depths we recorded, and no run had more than this budget.

The operating point. At its own budget a development run earns about 90% of its ceiling. The share is partly built in, because the attempts were recorded at that budget: against the eight runs that built the curve it reads 94%, and it falls as more runs are pooled and the ceiling rises (Appendix F).

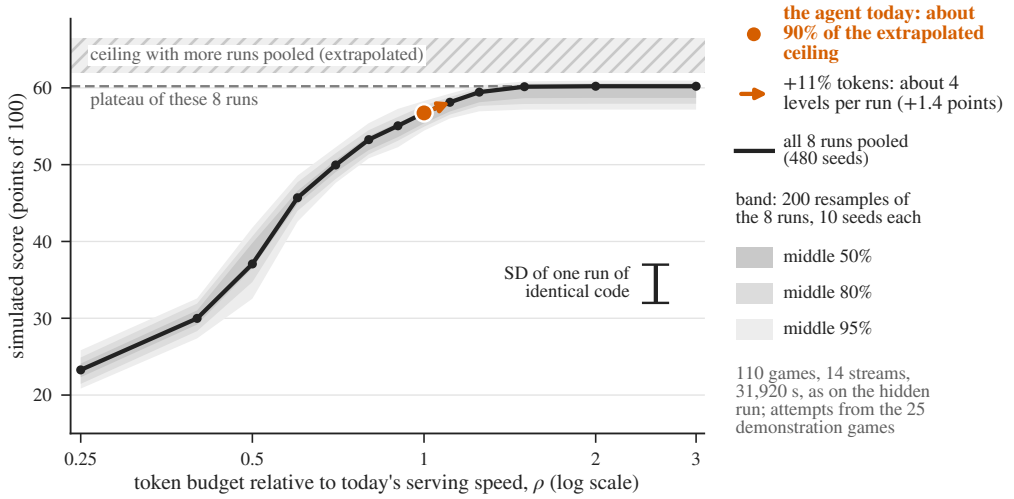


Figure 4: A simulator puts the agent near its token ceiling. Simulated score against token budget relative to today's serving speed, for 110 games in the hidden run's window, from attempts recorded in eight development runs; bands: middle 50%, 80% and 95% of 200 resamples of the runs. Dot: the operating point, about 90% of the ceiling extrapolated to more runs (94% of these runs' plateau); arrow: 11% more tokens; dashed: these runs' plateau; hatched: the ceiling with more runs pooled; bar: the SD of one run. Higher is better. The simulator assumes that levels no attempt cleared stay stuck; no run had more than this budget.

The price of throughput. The elasticity, the percent change in levels per percent change in tokens, is 0.31 at the operating point, and 0.18 to 0.49 across subsets of four runs. Our serving changes together added 11% tokens per second, which the curve prices at about 4 levels.

What it takes to see that price. Seeing it takes about 61 runs per arm at the pairs' SD, or 32 runs made on one day, both two-sided (Table 14); our gates ran two. So the serving gains are unresolved, not zero.

A second reading, from the clock. The clock gives the slope without a simulator. In the last tenth of the window, 34 development runs still cleared 4.4% of their levels and earned 7.4% of their points. If that late rate held on a stretched clock, the elasticity would be 0.44 on levels and 0.73 on score.

Where the two readings part. On levels they roughly agree: the clock's slope sits inside the curve's range across subsets and next to the fit to past serving steps (Appendix F). On score they do not: the curve's marginal levels weigh less than its average level, while the levels real runs clear late are deep ones that weigh more. Read on score, the clock prices our 11% speed-up at about 5 of 59 points, about three times the curve's price and still below what two runs can see. Why the curve's marginal levels are shallower we have not tested; until we know, it ranks levers by levels and underprices throughput in score.

Why throughput may not be the lever. At the deadline the simulator spends 62% of tokens on levels it clears, 31% on levels that no attempt cleared and 7% on clearable levels left unfinished. For the nine yield runs, 57% of the points lost to unreached levels sit on levels that no run in the yield comparison cleared; for its controls the share is 48%. Whether more tokens or different decisions would clear those levels is open: the curve assumes they stay stuck, and no run had more than this budget.

How far to trust it. We built the curve after the changes it prices, so we use it to order levers and not to forecast a score: leaving one run out, it is close at matched tokens but 4.9 levels low end to end, about as large as the price it gives. A token-only cut to 70% of the budget would test it; the curve predicts a loss of about 20 levels. Read at the same point of their window, the runs of the clock reading had cleared 20 fewer, though they lost about twice the curve's share of score, and a truncated run is not a run given fewer tokens, which we have not made.

Past serving steps. Twelve earlier serving steps fitted together give an elasticity of 0.45, and one of them, expert pruning, bought 19.5 levels against 3.3 predicted from its extra tokens, so a throughput change can be worth more than the curve says (Table 15).

5 A context lever that did not survive interleaving: what the model sees when a turn resumes

The change. A yield ends a request after a set number of tokens generated without an action and resumes the turn with the opener and the current board image sent again (Sec. 2). Duck yields on a time threshold, Franzen made it a token threshold of 2,048, and Lord Han Solo's public notebook yields rarely [Bessis et al., 2026, Franzen, 2026b]. We raised the threshold to 8,192; the idea of yielding less often is not ours, the value and the measurement are.

The comparison. Nine development runs with the rarer yield average 124.0 levels, and 21 with the old one average 111.5. The runs were not interleaved: the controls ran first and the treated runs over the following days (Appendix E). A day-level shift like the one in the identical pairs (Sec. 3) can carry part of any such gap.

Its uncertainty. At the pairs' SD the 12.5-level gap runs from $+4.5$ to $+20.6$. A run bootstrap, which treats runs on different days as exchangeable, gives $+9.2$ to $+15.8$; the treated runs lead on all but three games (Figure 5; Table 8).

The counter that moved. The mechanism counter is the resume opener, a request that re-sends the opener and image, several thousand new prompt tokens each. Resumes fell about 70%, from about 600 per run to about 190, while turns, requests per turn and tokens per turn held. Replaying the recorded turns at each threshold reproduces the counts. The freed prefill is at most 2.3% of GPU time by arithmetic and under 1% by the server's own timers, about half a level on the curve, so the gain is not the saved compute.

The token confound. Seven of the nine treated runs also carry later serving changes and generated 17% more tokens than the controls. At the curve's elasticity those tokens are worth about 5 of the 12.5 levels, and up to 9 at the highest per-token rate we measured.

What the model-free checks say. At equal tokens per game the later seven match the controls through 150K and lead only beyond 200K, and their clearance per token equals the controls' (Appendix E). Their gain fits their extra tokens as well as it fits the yield; if it is tokens, the curve underprices throughput (Sec. 4).

The clean comparison is weak. The first two treated runs differ from their controls only in the yield line and in two checks that act only in a scored run, and they generated the same tokens. They led by 15 levels (p 0.33), which two runs per arm cannot resolve. They were the best of 17 first-run comparisons on this stack, and if every arm had done nothing the best of 17 would reach that lead about once in ten at the pairs' SD.

So across days the rarer yield coincides with 12.5 more levels, and the counter it acts on moved as designed; we cannot say how much of the gap is the yield itself. The interleaved test below decides that.

Interleaved test. We then ran six runs per arm on one day, on two builds that differ in one line, alternating regions; four more were launched and never ran (Appendix E). The longer threshold scored 2.5 levels below the shorter (permutation p 0.42), while re-sending the opener a third as often.
At matched tokens the gap kept its sign. The across-days gain did not reproduce when interleaved; drift between days and about 5 levels of extra tokens are candidate causes, and this test cannot apportion them. At the pairs' SD the interval on the difference is -14.2 to $+9.2$ (Figure 5c), so the test rules out gains above about 9 levels and not smaller ones. The verdict under the rules fixed in a dated file, amended once before any run started, is null.

The rarer yield moved the counter it targets, and interleaved it showed no level gain we could see. If a resume does change decisions, we do not know why, and this test cannot show that it does.

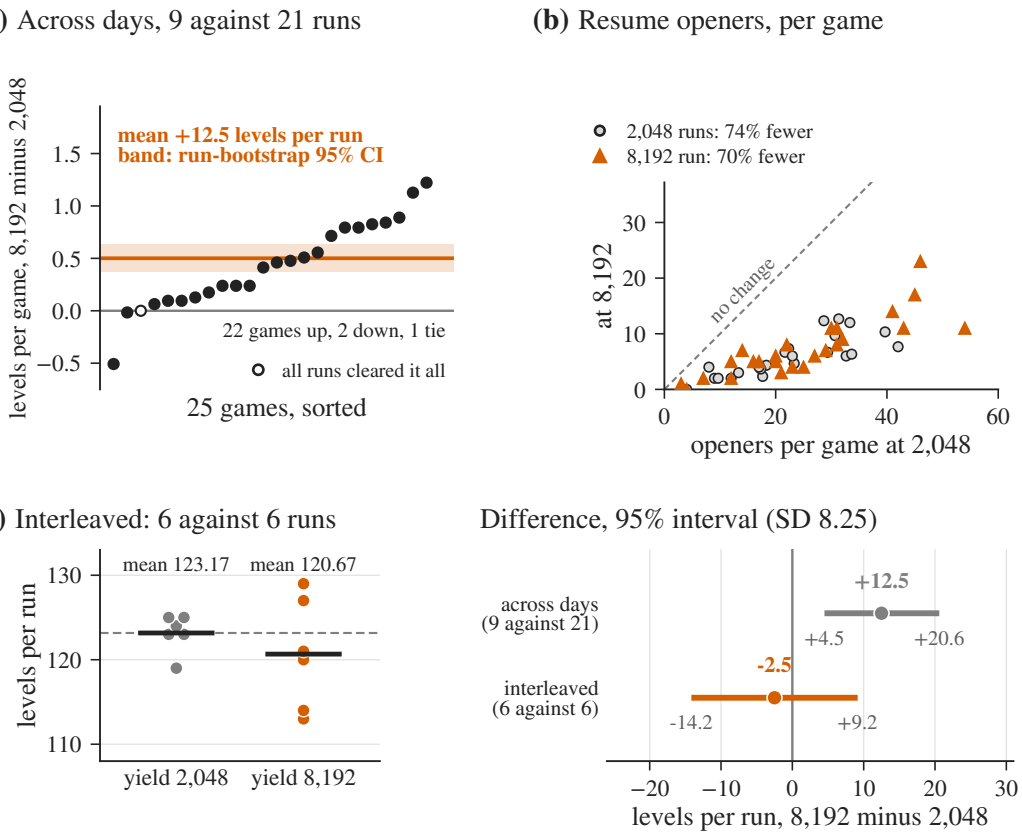
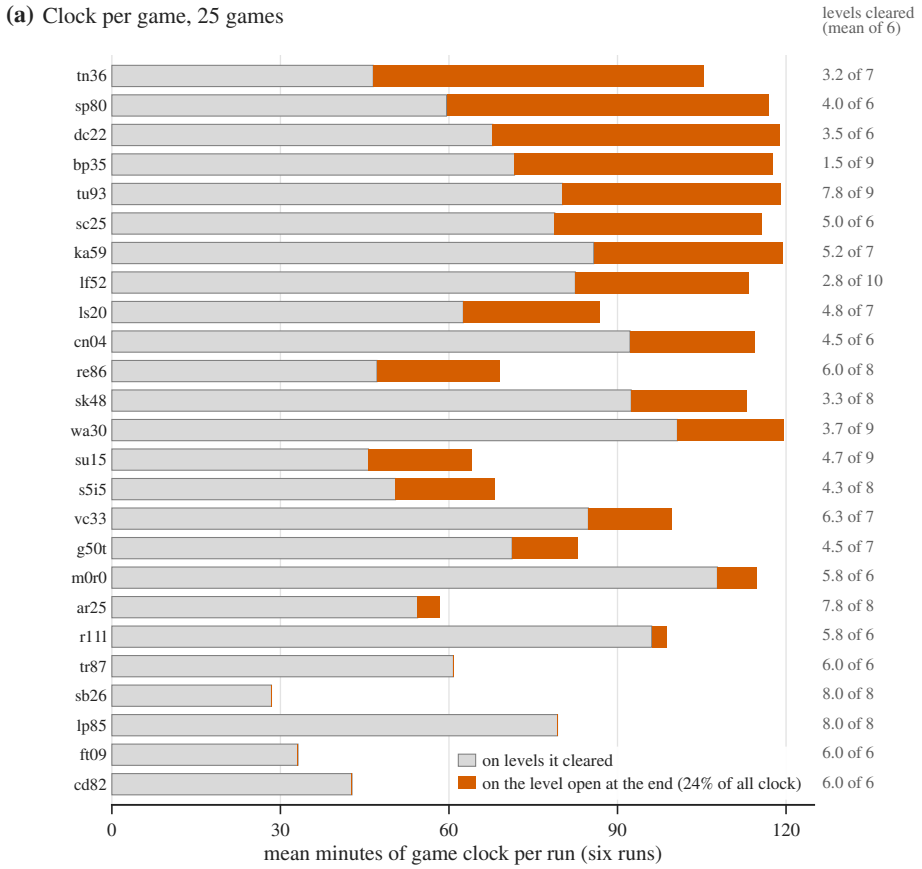


Figure 5: Re-sending context less often coincided with more levels across days, and not when interleaved. (a) Across days: per demonstration game, mean levels of nine runs that yield after 8,192 tokens minus 21 runs that yield after 2,048, sorted; line and band: the mean and its 95% bootstrap interval over runs (+9.2 to +15.8). (b) Resume openers per game from the four runs with usage logs. Circles: three runs at 2,048, observed (x) against the same turns replayed at 8,192 (y), 74% fewer in total; triangles: one run at 8,192, replayed at 2,048 (x) against observed (y), 70% fewer. Below the diagonal means fewer interruptions at 8,192. (c) Interleaved, two builds that differ in one line, alternating regions: levels per run of six runs at 2,048 and six at 8,192 (means 123.17 and 120.67), and the two differences, 8,192 minus 2,048, each with a 95% interval at the six pairs' SD of 8.25 (6 df). The across-days comparison was not interleaved, and seven of its nine 8,192 runs also generated more tokens. The interleaved interval, −14.2 to +9.2, excludes the across-days +12.5 and any gain above about 9 levels, and does not exclude smaller gains; four launched runs never ran. Higher is better.

6 Fixing what the model is shown and can check: stalls, objects, tools and replays

Why look here. The yield changed when a request resumes and moved nothing once the arms were interleaved (Sec. 5). Two other places shape the model's decisions: what the board looks like to it, and what happens when it stops acting. From 2026-10-29 to 2026-11-01 we ran a short program on both, and on what the model can check, diagnosis first and a written kill rule per change. Its result is four changes whose mechanisms fire and whose level means we report and do not claim, and a fifth that passed its gate and showed no gain on its target games.

Diagnosis. We read paired plays, the same level cleared in one development run and stuck in another, from four runs made before the program, and counted or quoted every claim from the transcripts. Stuck runs stop acting. An *idle turn* is a turn of the model that ends, at a yield or the length cap, with no action executed. Of the levels never cleared, 27% hold three or more idle turns in a row, against 8% of the cleared levels. Their clock tells the same story: 40% of it goes to such turns, against 29%. Stuck runs also write off an object type or an action they never tested, and misread what the harness shows them.



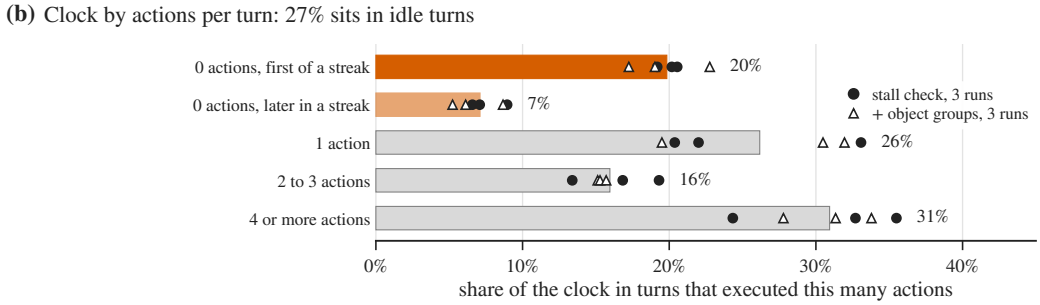


Figure 6: Where the clock goes. (a) Per game, mean minutes of clock on levels it cleared (gray) and on the level open when the window closed (orange), sorted by the orange tail; right: mean levels cleared of the game's levels. (b) Share of the clock by the number of actions executed in a turn; bars pool the runs, dots are single runs (filled: stall check; open: plus corner-joined regions). Six development runs of the first two changes below, 25 games each; time stamps include queueing on shared serving. The figure shows where time goes, not waste and not what a faster turn would buy, and a game that cleared every level has no tail by construction.

Where the clock goes. In the six runs of the first two changes below, 76% of the clock goes to the levels a game clears, about 13 minutes each, and 24% to the level it was on when the window closed (Figure 6). Idle turns, those that execute no action, hold 27% of the clock, but a read of them found mostly real work, simulators and reads of the history, so we built no blunt cut.

Where the clock goes, by level. Late in the program, a research pass over our transcripts, our harness code, public harnesses and the agent literature, with a skeptic for each top candidate, asked what sets the clock per level (Appendix K). Execution is cheap: multi-action plans rarely fail, and on the levels it clears the agent uses well under a human's actions. Discovery is not: levels that take

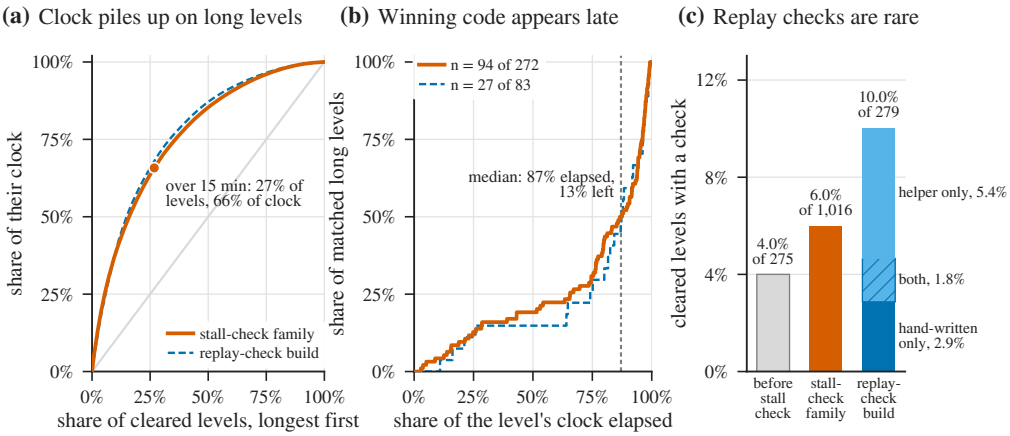


Figure 7: Where the clock goes, by level. Transcripts of development runs, cleared levels from level 2 on; a level counts as cleared when a later turn shows a higher level, which misses a level cleared on a game's last turn. Solid: 11 runs of the 15-stream harness family before the replay check (the stall check build and three builds that add object groups, the scroll report or the tool fixes), 1,016 cleared levels. Dashed: the replay-check build, 3 runs, 279 cleared levels; its numbers are readings, not claims. (a) Levels sorted longest first: cumulative share of levels against cumulative share of their clock; the dot marks 15 minutes (27% of levels, 66% of the clock); gray: equal shares. (b) On levels over 15 minutes, the share of the level's clock elapsed when code like the clearing turn's (identifier overlap 0.5 or more) first appears; the text proxy matches 94 of 272 levels (27 of 83); line: median, 87% elapsed. (c) Cleared levels whose code checks a model against the recorded history: by a detector of hand-written replay loops before the stall check (3 earlier runs) and in the stall-check family; the detector cannot see a helper call, so the replay-check bar stacks hand-written only, both, and helper only. The figure shows where the clock goes, not that any change clears more levels.

over 15 minutes are 27% of later-level clears and hold 66% of their clock (Figure 7). On those levels code like the clearing turn's (a text proxy, matched on about a third of them) first appears with a median 13% of the level's clock left, which fits most of a long level going to finding the rule or the goal, not to carrying out a plan.

Five changes, five kill rules. Each change has a gate, written before its first run: the counter it exists to move must move, the model must respond to it, actions must not rise, and mean levels and game score must stay within 2.5 levels and 1.5 points of the control. Any miss kills the change, and no run is added after reading. Each change ran three times on Kaggle against a pooled control of seven runs of the 15-stream build that the first change extends (mean 123.4 levels, SD 7.8; five of the seven on Vertex AI).

A stall check. When a level has gone three, five or seven idle turns in a row, the harness tells the model to execute an action in this turn, a short probe with its effect predicted or the checked first steps of its plan. It also lists the object types (colour and size) and the valid actions not yet tried on the level, and asks for step-by-step checking: mechanics first, replay recorded steps before a long plan, retry a no-op from another state. Of the idle turns that were the third, fifth or seventh in a row, 87% were followed by an action, against 54% in the runs we diagnosed. Streaks of five or more idle turns fell from 2.75 per run to 0.33. Every predeclared item passed on all three runs (Figure 8; Appendix J).

A first version failed its own rule. Its first build never printed the untested list, because the harness reports actions under one set of names and the history under another. The rule killed it on that counter, and we fixed the names. Its two runs cleared 131 and 127 levels, as many as the fixed build, so levels from so few runs cannot tell a half-built mechanism from a built one.

Objects the model could not see whole. The model's main view of a board is a segmentation into regions of one colour joined along edges. A glyph drawn with corner contacts falls apart in it: in bp35's third level the X marks that are the level's key read as 15 single pixels, and a stuck run called them decoration (Figure 9). On 184 level starts in human replays, 76 hold such an object. We

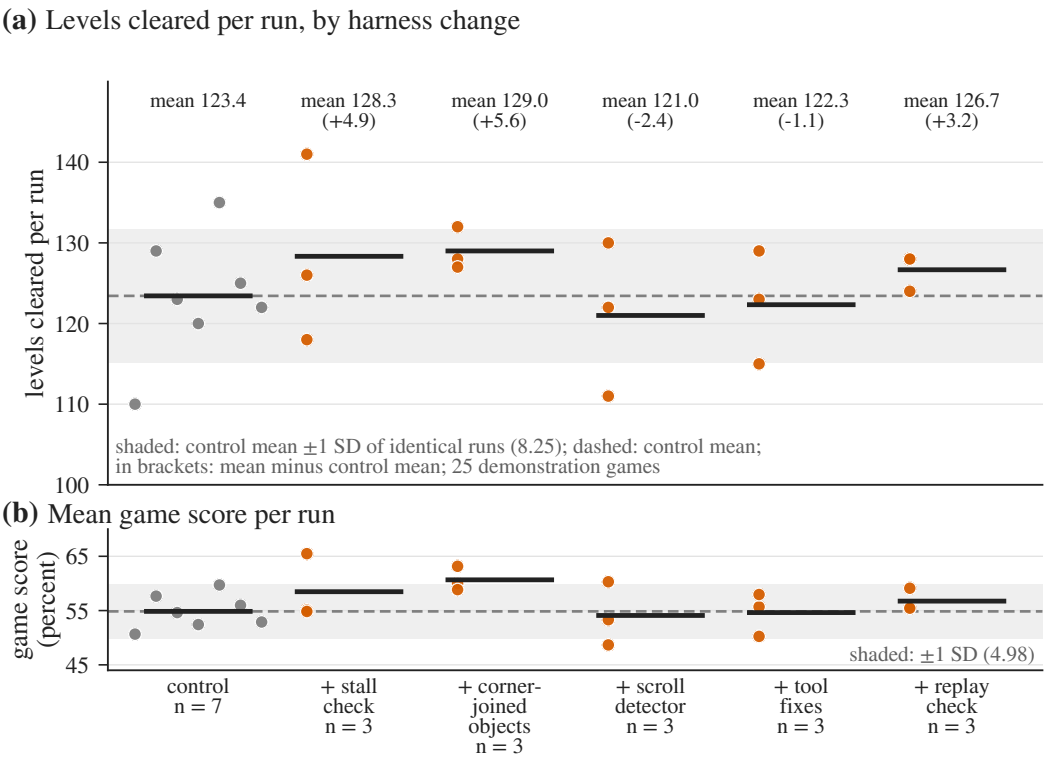


Figure 8: Levels (a) and game score (b) per development run. Gray: the control, seven runs (five on Vertex AI, two on Kaggle); orange: each change, three runs on Kaggle; bar: mean; dashed line: the control's mean; shading: one SD of identical runs (8.25 levels, 4.98 points). Higher is better. Twenty-two runs in all. The tool fixes and the replay check extend the corner-joined build, not the scroll build. The control is mostly from another platform (Vertex AI) than the changes (Kaggle). The figure draws no test: no change's level mean differs from the control's by more than one such SD (only the corner-joined build's score mean sits just above its band), so it cannot rank the changes, and the control and the changes ran on different platforms and days.

gave each region a `group`, the lowest id among regions of its colour joined to it by corner contacts, and changed nothing else. In every run the model read `group` in 33 to 64 code blocks, against none before, and every predeclared item passed.

A camera that moves. In some games the whole board scrolls with the player, and the diff tool then reports hundreds of unrelated changes: one step of bp35 gave the model 1,343 changed cells and 89 moved dots, and one run read the scroll as draining water. The diff tool now detects a whole-interior shift, reports it as `scroll`, compares objects after undoing it, and the turn opener says when the view scrolled (Figure 15 in Appendix J). On the human replays the detector fires in bp35 and lf52 only, and never in the other 20 games.

A mechanism that fires without a level gain. The detector fires in play: about 24 scroll messages per run in those two games, and none in the others. Its three runs cleared 111, 122 and 130 levels. The mean is below the control's and below the build it extends, and bp35 stayed at one level in all three runs while lf52 reached two, three and four. The gate printed PASS, by a tenth of a level on its no-harm item, but a passed gate is necessary and not sufficient: the change showed no benefit on the games it targets, so it was not shipped. Three runs cannot say that it hurts.

Tool failures that threw work away. Two failures of the python tool discarded work the model had done. A snippet that ran past its time limit came back as "Tool timed out" with everything it had printed lost, and `inspect.getsource` on a function kept from an earlier snippet failed; in the six runs before, these happened 6.0 and 7.8 times per run. The tool now stops a snippet 3 seconds before

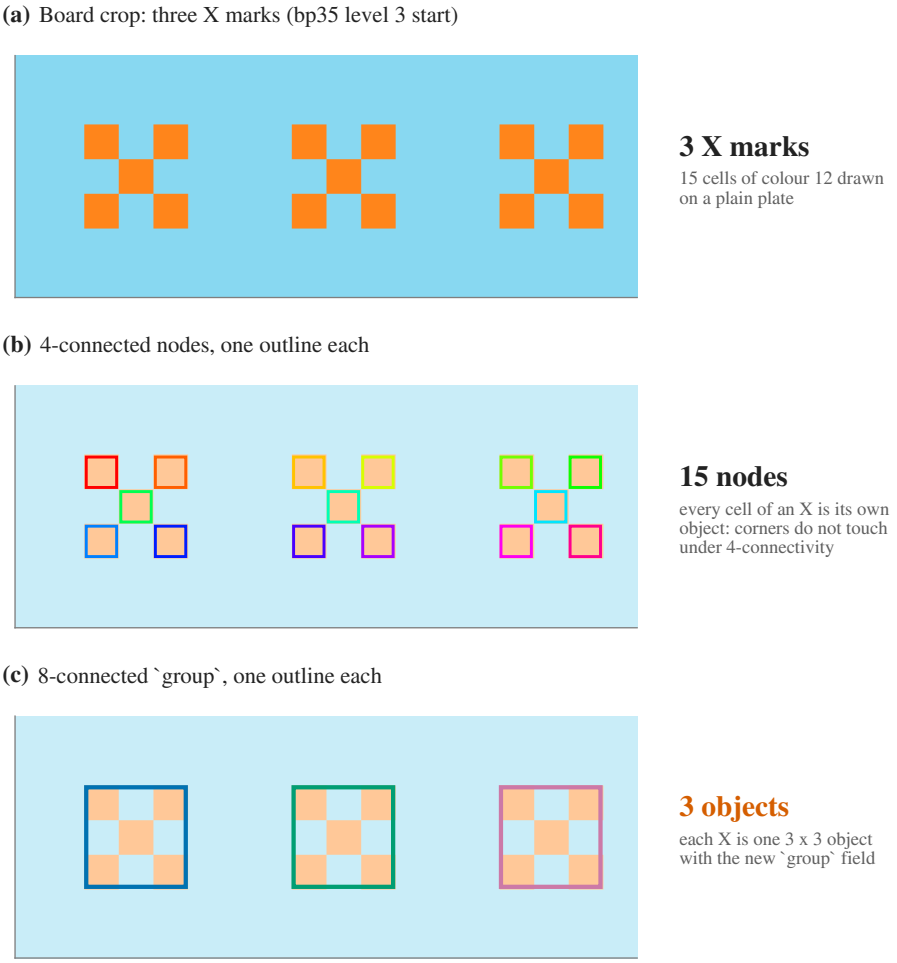


Figure 9: Edge-joined and corner-joined regions on one board. bp35 level 3, start frame of a human replay, cropped to the three X marks: (a) the board; (b) edge-joined nodes turn the 15 X cells into 15 single-pixel nodes; (c) with `group`, three objects of 3 by 3; (d) 76 of 184 level starts in 22 human replays hold a group of two or more nodes at least 3 by 3 cells (73 if sparse dot patterns are dropped). Counts come from the real patched module and describe the boards, not play: the figure does not show that the model reads a group better or that any level cleared because of it.

its limit, counting only its own compute and never time spent waiting on the game, and returns what it printed; `getsource` reads the stored source. In three runs both failures fell to zero, the early stop fired about six times per run, and every predeclared item passed (Appendix J).

Checking a theory against the history. The cheapest counterexample is one the history already holds, at no cost in actions, yet the model checked its simulator against recorded steps on about 6% of cleared later levels (the research pass counted goal guesses tested against history at about 0.3 times per run, not reproduced here). Public harnesses with the largest fixed-weight gains make that replay a rule [Skoutnev et al., 2026, Rodionov, 2026a]; Twin's harness took the same model and

agent from 61.1 to 93.3, though its ablation removed five parts at once. Skeptics judged each of the top four candidates, this one included, too small for three runs to see, so we built it to be gated on use and no harm. We gave the sandbox two helpers. `check_model(step, observe)` replays the level's recorded transitions through the model's own simulator, newest first, and reports the first mismatches with predicted and actual state; it also notes when a board and action led to two different results, which means hidden state. `check_goal(is_goal)` runs a goal test on every recorded board of the level; the level ended on none of them, so a hit means the test is wrong or incomplete, or that board needed one more action. Neither can take an action, and both stop at a time limit.

What the helpers did. The model wrote about 34 code blocks per run that call them, in five or six games, about one block in a hundred and close to the rate of an earlier validation helper on this model. Of 85 replays, 42 named a wrong prediction and ten more reported only exceptions raised by the model's own code; no goal check found a counterexample.

Checks against the history, across builds. Cleared later levels with a simulator check rose from 6% to 9%, or 10% with goal checks (Figure 7c). Hand-written replays read slightly lower; these are readings across builds, not claims, and not evidence of substitution. Every predeclared item passed, including a bar against the build it replaced (Appendix J).

What three runs can say. Three runs per arm cannot resolve a lead of five levels: identical code scored 123 and then 101 (Sec. 3), and three runs against seven resolve about 16. The stall check and corner-joined builds lead the control by about five levels, the replay build by about three and the tool-fix build trails by about one, and we report these without claiming them; the gates test that a mechanism fires and that nothing is harmed, and they did.

Which control. The size of a lead depends on the control: against the two Kaggle controls alone, the four shipped changes lead by about 6 to 13 levels, about twice their leads over the pooled control (Appendix J). The builds ran one after another on Kaggle, the control mostly on Vertex AI, and each change contains the one before (the tool fixes extend the corner-joined build, not the scroll build), so day, platform and independence are all open. None alters the token budget, so a lead would be a lead in decisions; three runs cannot say whether there is one. The build with the first two changes drew 33.56, and the build with all four shipped changes drew 36.38 in the final draw; one draw each says nothing about a mean (Sec. 8), and the two Final Submissions are earlier draws of an earlier build.

7 Fine-tuning: what the agent learns from its own play

7.1 Imitating its own wins: all three adapter runs scored low

What we trained. The shipped agent played 127 games of RE-ARC-3, Tufa Labs' offline generator of ARC-AGI-3-style games [Tufa Labs, 2026], at the development budget per game. We kept as targets every reply from a level the run cleared and trained a rank-16 LoRA adapter [Hu et al., 2021] on the attention, recurrent and shared-expert projections, one pass, for about $8 on one rented GPU.

What happened. Validation loss on held-out RE-ARC-3 windows fell 13.9%, and levels fell too. Two runs at full strength cleared 109 and 115 levels and one at half strength cleared 117. Serving was intact: the adapter generated more tokens per second and no more tracebacks.

Against what. All three sit at or below the two lowest of 18 development runs of the yield-8,192 family on Kaggle and Vertex, which average 124.4 (rank p 0.004). Those 18 ran on other days than the adapter, which ran on a rented VM, so we added a same-card control: two no-adapter runs of the adapter's own base arm on the same RTX PRO 6000 as Vertex AI jobs cleared 132 and 130 levels, above all three adapter runs. Among these five runs the three adapter runs are the three lowest (exact rank $p = 1/10 = 0.10$). The design limits stand: two controls; the adapter and the controls ran on different services (rented VM, Vertex AI) and 15 days apart, not interleaved; one control restarted after a preemption; and in the harness program's later control the Vertex AI runs averaged 9.7 levels above the Kaggle runs (two of those), so an offset is not excluded. We read this as the adapter staying

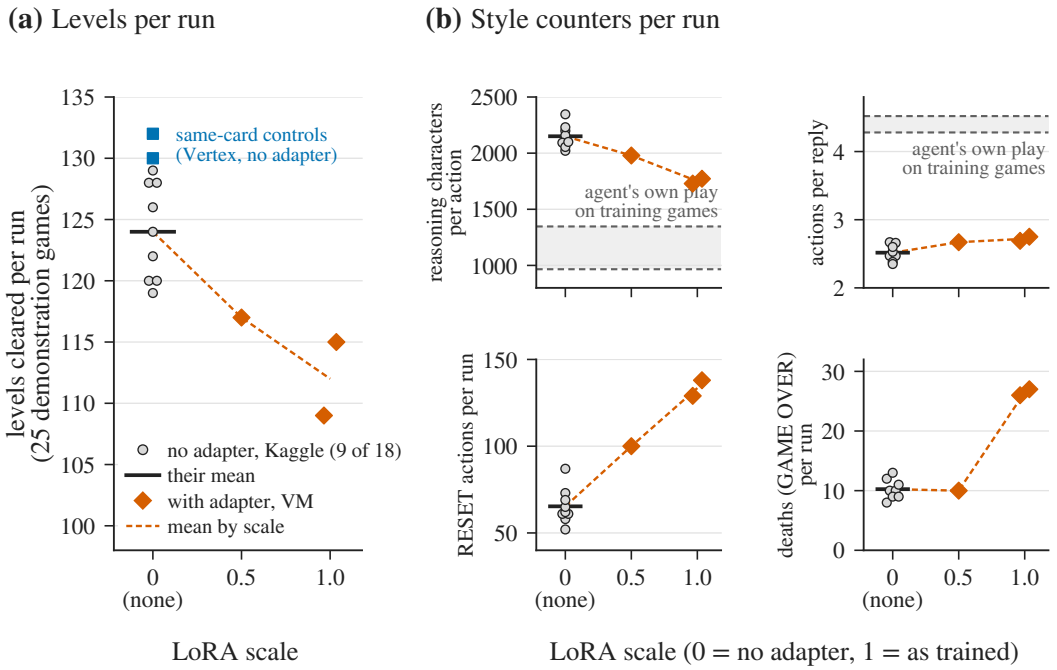


Figure 10: All three adapter runs scored low. LoRA scale 0: no adapter, nine of the 18 runs without it (the nine on Kaggle; the other nine ran on Vertex AI and are not drawn; the mean of all 18 is 124.4), one point per run; squares: two same-card no-adapter controls of the adapter's base arm, run later on Vertex AI (132, 130); 0.5: one run; 1.0: two runs; 25 demonstration games. (a) Levels per run; higher is better. (b) Style counters; gray band: the agent's own play on the training games (the RE-ARC-3 generator), drawn only for the per-action and per-reply rates. Reasoning per action moves toward the band, actions per reply barely moves, and resets and deaths rise, as weaker play would. Not interleaved: adapter runs on a rented VM on 2026-10-19, the squares on Vertex AI on 2026-11-03, the gray circles on Kaggle.

below a same-card control, not as proof that it caused the drop. One run at half strength cannot show a dose response in levels.

A reading: from the data to the style to the levels. The counters fit one reading, which we have not tested (Figure 10; Table 25 in Appendix I.1). The filter kept almost everything: the agent clears 91% of RE-ARC-3 levels against 68% of demonstration levels, so "a cleared level" admitted 63% of training windows. Counted by turn, half the targets came from levels that took more than twice the human baseline's actions, which slow levels supply by construction; we have not measured that share on the agent's own demonstration-set play. Play on these easier games has its own style, less reasoning per action and more actions per reply; the adapter's reasoning per action fell toward it, its actions per reply moved about a tenth of the way and its NO-OP share moved away. It also took more actions, reset about twice as often and died two to three times as often, which weaker play produces anyway.

What the reading leaves open. Deaths do not carry the loss, since at half strength they returned to the family's range while levels stayed at its bottom; one run at half strength cannot rule out over-strength. Two other causes are untested: the adapter saw only suffixes of the contexts the agent played with, and it was fit with a loss at a different temperature from the sampling. The lesson we draw, untested: lower loss on the agent's own cleared turns was not better decisions, and a self-training filter may need to select what the agent does not already do, such as efficiency on games it has not won.

7.2 Reinforcement learning, and a noisy training forward pass

Round A. We then tried a group-relative policy gradient in the style of GRPO [Shao et al., 2024], with a trust-region mask and a small KL penalty (Appendix I). The reward counts levels cleared

within a budget like the hidden run's, minus a penalty per death, with no teacher and no efficiency term. Three runs were trained: a reference, one with a doubled learning rate, and one on data from only the training half of the demonstration set. Training stops when a drift guard on held-out windows fires: the KL estimate k_3 [Schulman, 2020], the usual low-variance estimator, above 0.02, or entropy moving more than 10%, both declared before any training window existed.

The training forward pass is not deterministic. Before the first update the adapter is zero, so the guard compares the base model with itself; it read 12.9 against a stop at 0.02. Five base-only passes over the same held-out tokens agree exactly on about 12% of tokens, with a root-mean-square gap of 0.37 nats per token, on our VM and on Vertex alike (Figure 14 in Appendix I). The raw k_3 exponentiates these gaps, so one token can dominate it, and across pairs of identical passes it spans four orders of magnitude. Read raw, the predeclared guard would have stopped the reference run at step 8 with nothing kept.

Read the guard net of the stack's own floor. We clip each token's log ratio to ± 2 nats inside the guard and read both stops net of the run's own first reading, its noise floor. The clipped estimate is stable across passes and machines, at about 0.064 (SD 0.0008) against the 0.02 stop margin; its whole range is about seven times smaller than that margin, and the readings come from five passes compared in pairs, so they are not independent. Reading net of the floor assumes that noise and drift add, an approximation inside a clipped, nonlinear estimate. Both amendments came after the first base-against-base readings and before any reading of a trained adapter. On this stack, whose clipped floor is three times the stop margin, the guard needs this correction; any stack should measure its floor before trusting a raw KL guard.

What the runs did. The reference run and the doubled-rate run both stopped on their drift guards and kept an earlier checkpoint, at equal learning rate times steps and with nearly equal drift. On most steps the update pushed down the probability of tokens from failed plays more than it pushed up tokens from successful ones. The third run did not finish in time for round A.

Round A gate outcome. Two of three adapters finished in time. On the demonstration games, against shared controls, one led by 1.9 levels in a first wave and the other was killed. The lead did not replicate: the second wave alone read -2.8, and both waves pooled read $+0.25$ against a $+1.75$ bar.
The served adapter kept only 28% of its trained change, because rounding merged weights to BF16 erases most of a small update. A stochastic-rounding merge keeps almost all of it; that full-strength adapter read -0.6 levels against pooled controls (bar $+1.5$). No round A adapter shipped, and the style gate passed every adapter it read.
Rounds B and C. A round B adapter, trained on levels the agent clears only sometimes, drew three times on the board while its gate ran (35.90, 39.79, 35.98), because we submitted it before the gate read. It then failed: -1.4 levels on the games it trained on. Served at three times its scale (round C), it rose on those games but read -0.8 levels on games it never saw, against a $+1.5$ bar, and reinforcement learning ended there.

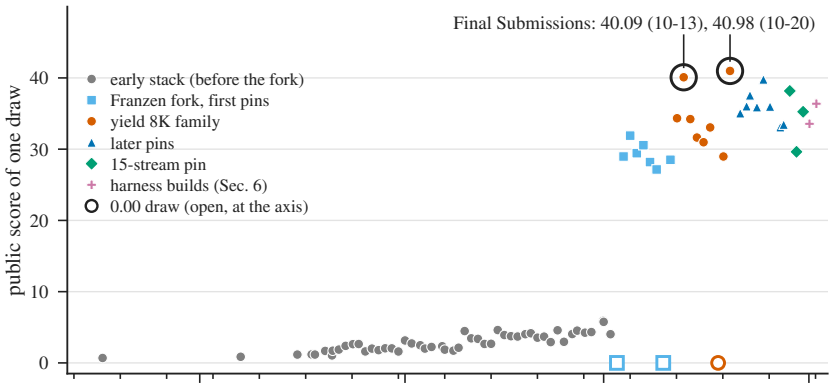
8 The leaderboard result and its selection

A maximum. Our best public score, 40.98, covers half of the hidden games. It is the best of 28 live draws since the fork, and one of three draws of its configuration (Appendix C). On the final public board, read after the deadline, our rank is withheld here and the top score was 75.56 (Figure 11).

One agent's draws spread widely. The same agent in two serving variants, one with a draft vocabulary that should leave the output distribution unchanged, averaged 38.43 over three draws and 31.15 over four (Appendix C). Five later draws with a keepalive that acts only while the server boots averaged 35.04, between the two, and three draws of a build with one more game in flight averaged 34.35. The variants were drawn in separate stretches of days, so a calendar or Kaggle-node effect plus chance is the economical reading. The same calendar term applies to every time-ordered comparison here, including the board step after the yield change and the development-run comparison of Sec. 5.

The board ranks the stack and attributes nothing. The seven live draws before the yield change average 29.25 and the 21 after it 34.96, three of them an adapter's (Sec. 7). Those are different, time-ordered configurations, each carrying other changes, so the step is a trend and not a lever's effect.

(a) Daily draws, n = 81 scored submissions



(b) Our best public score against the bars (one snapshot per day)

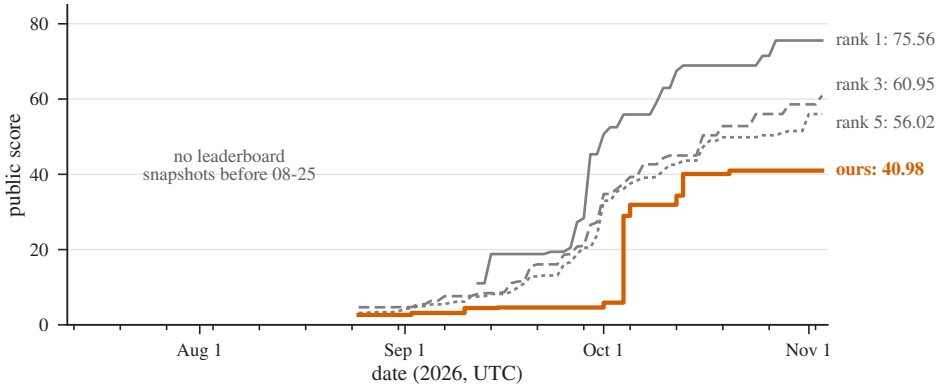


Figure 11: The live board history. (a) Every scored submission, by build family (Appendix C lists each draw's build); the Final Submissions (40.98 on 2026-10-20 and 40.09 on 2026-10-13) are ringed, and open symbols at the axis are the three 0.00 draws. (b) Our best public score against the scores at ranks 1, 3 and 5, one snapshot a day from 2026-08-25. Public scores on half of the hidden games, higher is better; the last point of (b) is the final public board, read at 2026-11-03 01:06Z after the deadline. The figure does not show private scores, which are pending.

How much of the best draw is selection. If the first eight live draws after the yield change shared one mean, the best of eight would sit about 1.4 SD above it, near 40.3 (Appendix C). Resampling the twelve valid draws of the agent's family, whose record closed on 2026-10-28, gives an expected best of twelve of 40.33 (Figure 12). So 40.98 is what a maximum of this family looks like, not evidence of a higher mean. Picking two Final Submissions by public score is a mild form of the holdout reuse studied by Blum and Hardt [2015] and Roelofs et al. [2019].

What to expect on the private half. Within a configuration the public and private halves of a draw correlate 0.00 to 0.53 on our model, so a draw picked by its public score should be expected to score near the family mean on the private half, not near its public score. The agent's seventeen valid draws and its zero of unknown cause average 32.67 together.

Scale check. The valid draws average 0.61 of the development-run mean game score, inside the earlier range of 0.53 to 0.63; the ratio checks scale and forecasts nothing.

Dead draws. Three of 31 completed draws since the fork scored 0.00: two in field-wide failures of the midnight submission wave, and one of unknown cause, since a scored run returns no log (Appendix H).

Final Submissions. Our written rule, adopted after our best draw was submitted (the finals rule, Appendix C), marks the agent's two best valid draws, 40.98 and 40.09, unless a fine-tuned adapter passes all its gates; none did (Sec. 7). On 2026-11-02, after the last draw, we marked these two by hand on the submissions page, where the selection held after a reload; they are also the two highest public draws, so Kaggle's automatic pick would have chosen them, and the final standing is the better of their private scores. Both are draws of builds that predate the harness changes of Sec. 6. The first harness build drew 33.56 and the build with all four shipped changes 36.38 in the final draw; one draw each says nothing about either build's mean.

9 What to copy, and what transfers

What to copy. (1) Run identical code at least eight times on one day to size the noise before trusting a level gain; here two runs per arm saw only effects above 23 of 183 levels. (2) Judge a small change by the metric it targets (tokens per second, prefix-cache hits, draft acceptance), and use levels only to veto gross harm. (3) Price a speed-up two ways before paying for it, by resampling logged level attempts at other budgets and by what the window's last tenth still earns; here the two differ about threefold on score, so check with a token-only cut. (4) Interleave arms on one build before believing a harness gain across days: here a rarer re-send coincided with 12.5 more levels across days and with none when interleaved. (5) Diagnose before building: set a cleared play of a level beside a stuck play of the same level, count what separates them, and write each change's kill rule before its first run. (6) Untested here: where the agent already clears most levels, filter self-training data by efficiency, not by "cleared". Also untested: a paired test on recorded states (do replies with a context change disagree on the first action more than two unchanged replies do?), which we used once and have not validated against a lever with a known level effect.

Five objects that transfer. *A noise ruler:* any evaluator whose score varies between identical runs can be sized by repeating them on one day, and a gate built on mechanism counters needs nothing more. *A budget curve:* any agent with a token or time budget and logged attempts can price a speed-up before paying for it, and can see whether its losses sit in problems that more search would reach. *A re-send policy:* any harness that pauses a model and resumes it by re-sending its prompt is choosing how often to interrupt; here that choice coincided with a level gain across days that vanished when the arms were interleaved, so interleave before believing a small harness gain. *A stall rule:* any agent that can think without acting needs a rule for when thinking must end in an action; ours fired as designed, and levels from three runs cannot say whether it helped. *A self-training warning:* a success filter on tasks the agent already solves keeps its habits, slow ones included; in our one experiment the adapter's reasoning style shifted while its levels fell.

What we did not test. Each object was tested on one model, one harness and one benchmark; only the fine-tuning result crosses a boundary, from RE-ARC-3 training games to demonstration games whose families the adapter never saw.

10 Related work and credit

Noise in evaluation. Single runs are weak evidence in reinforcement learning [Henderson et al., 2017] and for agents [Bjarnason et al., 2026, Kapoor et al., 2024, Anthropic, 2026]. Remedies include intervals [Agarwal et al., 2021], power analysis [Colas et al., 2018, Miller, 2024], protocols judged by their error rates [Bouthillier et al., 2021] and the expected maximum of several runs [Dodge et al., 2019]. On this benchmark, forum posts attribute most of the variance of repeated runs to whether single levels clear [borro1980, 2026] and find that identical builds differ widely [Makarov, 2026]. We add the size of that noise on this score, and concurrent replication on rented copies of the scoring GPU; we also tried a paired test of single decisions, used once and not validated.

Budgets and context. Score that flattens with compute is the picture of Pourcel et al. [2025]; budget forcing ends reasoning at a budget [Muennighoff et al., 2025], while a yield resumes the turn. Ending a request at a tool call forces recomputation [Abhyankar et al., 2024], and keeping reasoning across tool turns is a known harness rule and lever [DeepSeek-AI, 2025, Kamradt, 2026a]; we found no earlier measurement of a client-side yield threshold.

Self-training and reinforcement learning. Training on a model's own successes is the idea of STaR, ReST and rejection-sampling fine-tuning [Zelikman et al., 2022, Gulcehre et al., 2023, Yuan et al., 2023], and RL recipes already drop prompts that a policy always solves, as in DAPO's dynamic sampling [Yu et al., 2025]. Ours is the supervised case of the same problem: a success filter removes too little when the agent already succeeds, and what it keeps carries the agent's slow habits. Round A follows GRPO [Shao et al., 2024], with its ratio band from the public RL code agent-factory-3 (near DAPO's asymmetric clip [Yu et al., 2025]), its KL weight from GiGPO [Feng et al., 2025] and the k_3 estimator [Schulman, 2020]. Serving outputs are known to depend on batch composition [He and Thinking Machines Lab, 2025, Yuan et al., 2025]; we measure what that does to a training-side drift guard.

ARC-AGI-3. Tycho and Twin use hosted models on the demonstration set [Lehmann et al., 2026, Skoutnev et al., 2026], and the benchmark report warns that such scores do not measure progress [ARC Prize Foundation, 2026a]. The open-weight work we found is mostly write-ups [Bessis et al., 2026, Franzen, 2026b]. Public harnesses converge on an executable world model checked against the log of play [Skoutnev et al., 2026, Lehmann et al., 2026, Wu, 2026, Gowtham, 2026, Rodionov, 2026a], the pattern of WorldCoder and PoE-World [Tang et al., 2024, Piriyakulkij et al., 2025]; they differ in whether the harness enforces the check or the prompt asks for it, and only Twin blocks an action until its program replays every observed transition. Twin's ablation removes its whole harness at once, and Rodionov's verification variant ranks first in every setting but bundles verification with other changes, so neither isolates the replay; graph exploration reaches deep states at near-zero action efficiency [Rudakov et al., 2025], and truncating history costs little when facts can be looked up again [Lencevicius, 2026]. Our replay check is the optional, prompted form of that rule on an open-weight model: it fired, and three runs cannot say whether it helps (Sec. 6, Appendix K). The two leading teams have published no method; their public repositories and their results on ARC-AGI-2 are consistent with post-training at a scale we cannot match.

What is new here. As far as we found, three things: a token-budget price for an ARC-AGI-3 agent, a measurement of a client-side yield threshold with an interleaved test on which its apparent gain did not reproduce, and a KL drift guard read net of a nondeterministic training forward pass; a paired test of single decisions is a fourth, but it was used once and is not validated.

Credit. Inherited, with the component table of Appendix A giving each part's origin: Qwen's model [Qwen Team, 2026], Intel's weights [Intel, 2026], albucino's drafter [albucino, 2026], J. Pezzulli's Pennyroyal build [Pezzulli, 2026], Tufa Labs' Duck harness and RE-ARC-3 [Bessis et al., 2026, Tufa Labs, 2026], and Daniel Franzen's patch, notebook and schedule [Franzen, 2026a,b]. Borrowed: shiiin9's priority formula [shiiin9, 2026], ursuciprian's drafter refit [ursuciprian, 2026], lovedheart's index-key fix [lovedheart, 2026], ideas from Son Pham's team, face-of-agi, lee-chang-93 and Lord Han Solo, Jeroen Cottaar's idle-rule fix [Cottaar, 2026a], and Tufa Labs' step-verification addendum, which Franzen's settings leave off and our stall-check change turns on [Bessis et al., 2026]. Ours: the build, the measurements, the guards and the fine-tuning experiments. Our paper and data are under CC BY 4.0 and our code under CC BY 4.0 or MIT-0, at the reader's choice, which meets both Kaggle's rules and arcprize.org's request; the repository is public (Appendix M). Third-party files keep their licences; the model is under the Qwen Community License, and we make no claim about its prize eligibility. Appendix M gives the licences, the release status and how AI coding agents were used.

11 Limitations and conclusion

Limitations. The evidence covers one model, one GPU and one harness. Every development number comes from the demonstration set, where we also chose what to ship, so it may overstate levels on fresh games; the only held-out number is a board maximum. The noise estimates rest on six identical pairs and one set of eight runs on one day (six started together), and the pairs carry a first-minus-second shift we cannot explain: only one of their twelve runs, a first run, began play more than three minutes before its server was ready, and without that pair the second run is still lower in the other five. The rejection-sampling comparison was not interleaved. A same-card control (two no-adapter runs on Vertex AI, 132 and 130 levels) stayed above the adapter's three runs, but it has two runs, it ran 15 days after the adapter and on a different service (the adapter ran on a rented VM),

and one run restarted after a preemption; an interleaved set on one service would separate service and day from the adapter. The curve is a simulator over recorded attempts, built after the changes it prices, and its ceiling is partly set by construction. The clock's reading assumes that the late rate of clears would hold on a stretched clock, and that rate was still falling. Each harness change has three runs or fewer; the arms ran one after another on Kaggle against a control that is mostly from Vertex AI, each arm contains the one before (the tool fixes extend the corner-joined build, not the scroll build), the arms are not interleaved, and a development run's clocks can start before its server is ready, unevenly across arms, so a lead of five levels is within the noise. The scroll detector passed its gate narrowly, showed no gain on its target games and was not shipped. The two Final Submissions predate every harness change, and the four-change build's final draw, 36.38, displaced neither, so the score we are judged on tests no harness change. A scored run returns no log, so nothing here explains a single board draw.

Conclusion. We asked whether decisions or tokens limit an offline ARC-AGI-3 agent, and how one can tell. Measured: run-to-run noise, with two runs per arm resolving about 23 levels. Simulated: a price of about 4 levels for our serving speed-ups; read from the clock, about three times the curve's price on score. Open: whether either price is right (one recorded step bought 19.5 levels, about six times its predicted price), and whether more tokens or different decisions would clear the levels no run has cleared. Not reproduced when interleaved: the level gain that a rarer yield coincided with across days, which excludes the earlier gain and not small gains; drift between days and extra tokens are the candidate causes. Fired and unresolved: five changes to what the model is shown, when it must act and what it can check; all five passed their gates and four shipped, with leads of -1 to $+6$ levels over the pooled control (larger over the two Kaggle controls alone) that three runs cannot resolve, while the scroll detector showed no gain on its target games and was not shipped. Negative, with design limits: fine-tuning on the agent's own cleared levels was followed by fewer levels, also against two same-card controls (rank p 0.10 among five runs), not interleaved and on different services and days, and we shipped none.

References

R. Abhyankar, Z. He, V. Srivatsa, H. Zhang, and Y. Zhang. InferCept: Efficient Intercept Support for Augmented Large Language Model Inference, 2024. URL https://arxiv.org/abs/2402.01869. ICML.

R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare. Deep Reinforcement Learning at the Edge of the Statistical Precipice, 2021. URL https://arxiv.org/abs/2108.13264. NeurIPS 2021, outstanding paper award.

albucino. Qwen3.8-Flash-Next-W4A16-FP8PLE (model card), 2026. URL https://huggingface.co/albucino/Qwen3.8-Flash-Next-W4A16-FP8PLE. Hugging Face model card, accessed 2026-10-13; source of the quantized MTP draft checkpoint named in Franzen's write-up.

A. Amankulov and D. Mamatin. The Limits of Speculation: Bounding Speculative Decoding in Mixture-of-Experts, 2026. URL https://arxiv.org/abs/2609.22156.

Anthropic. Quantifying infrastructure noise in agentic coding evals, 2026. URL https://www.anthropic.com/engineering/infrastructure-noise. Engineering blog, published February 5, 2026.

ARC Prize Foundation. ARC-AGI-3: A New Challenge for Frontier Agentic Intelligence, 2026a. URL https://arxiv.org/abs/2603.24621.

ARC Prize Foundation. ARC Prize 2026, 2026b. URL https://arcprize.org/competitions/2026. Competition overview page, accessed 2026-10-13.

M. Barney. System prompt: restored full-content draft after draftB lost 12 points, 2026. URL https://github.com/sonpham-org/arc-3/commit/bcbd6eb7b14962924397f239aa2cd9880c41c94a. Commit bcbd6eb7 in sonpham-org/arc-3, 2026-10-08; the repository later returned 404; copy in the repo evidence folder docs/research/2026-10-10-field/evidence/github.

O. Ben Shoham. Balancing Coverage and Draft Latency in Vocabulary Trimming for Faster Speculative Decoding, 2026. URL https://arxiv.org/abs/2603.05210.

H. Bessis, J. Cottaar, I. Pressman, A. Smit, M. Tešnar, and S. Viel. Duck Harness: Winning Solution for ARC-AGI-3 Milestone 1, 2026. URL https://tufalabs.ai/research/duck-harness/. Tufa Labs blog, July 1, 2026. Code: github.com/Tufalabs/duck-harness (commit 7652836).

B. H. Bjarnason, A. Silva, and M. Monperrus. On Randomness in Agentic Evals, 2026. URL https://arxiv.org/abs/2602.07150.

A. Blum and M. Hardt. The Ladder: A Reliable Leaderboard for Machine Learning Competitions, 2015. URL https://arxiv.org/abs/1502.04585.

borro1980. Paper Track team-up: I have a finished write-up but no leaderboard score, 2026. URL https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3/discussion/732932. Kaggle forum topic 732932, ARC Prize 2026 - ARC-AGI-3, post of 2026-08-05; the post reports the variance decomposition of 500 Milestone-1 game runs.

X. Bouthillier, P. Delaunay, M. Bronzi, A. Trofimov, B. Nichyporuk, J. Szeto, N. Mohammadi Sepahvand, E. Raff, K. Madan, V. Voleti, S. Ebrahimi Kahou, V. Michalski, T. Arbel, C. Pal, G. Varoquaux, and P. Vincent. Accounting for Variance in Machine Learning Benchmarks. In *Proceedings of Machine Learning and Systems 3 (MLSys 2021)*, 2021. URL https://proceedings.mlsys.org/paper_files/paper/2021/hash/0184b0cd3cfb185989f858a1d9f5c1eb-Abstract.html.

Y. Cai, X. Liang, X. Wang, J. Ma, H. Liang, J. Luo, X. Zuo, L. Duan, Y. Yin, and X. Chen. FastMTP: Accelerating LLM Inference with Enhanced Multi-Token Prediction, 2025. URL https://arxiv.org/abs/2509.18362.

C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper. Accelerating Large Language Model Decoding with Speculative Sampling, 2023. URL https://arxiv.org/abs/2302.01318.

Z. Chen, A. May, R. Svirschevski, Y. Huang, M. Ryabinin, Z. Jia, and B. Chen. Sequoia: Scalable, Robust, and Hardware-aware Speculative Decoding, 2024. URL https://arxiv.org/abs/2402.12374.

C. Colas, O. Sigaud, and P.-Y. Oudeyer. How Many Random Seeds? Statistical Power Analysis in Deep Reinforcement Learning Experiments, 2018. URL https://arxiv.org/abs/1806.08295.

J. Cottaar. Reply in "Two 0.00 scores since Oct 16, runs ending early. Anyone else?", 2026a. URL https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3/discussion/750173. Kaggle forum topic 750173, ARC Prize 2026 - ARC-AGI-3, post of 2026-10-17.

J. Cottaar. Reply in "Is everything back to normal, or is something else going on behind the scenes?", 2026b. URL https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3/discussion/753843. Kaggle forum topic 753843, ARC Prize 2026 - ARC-AGI-3, posts of 2026-10-29 and 2026-10-30.

DeepSeek-AI. DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models, 2025. URL https://arxiv.org/abs/2512.02556.

J. Dodge, S. Gururangan, D. Card, R. Schwartz, and N. A. Smith. Show Your Work: Improved Reporting of Experimental Results, 2019. URL https://arxiv.org/abs/1909.03004.

L. Feng, Z. Xue, T. Liu, and B. An. Group-in-Group Policy Optimization for LLM Agent Training, 2025. URL https://arxiv.org/abs/2505.10978.

D. Franzen. ARC-AGI-3 Milestone 2 Solution, 2026a. URL https://www.kaggle.com/code/dfranzen/arc-agi-3-milestone-2-solution. Kaggle notebook dfranzen/arc-agi-3-milestone-2-solution; public score 27.89 per the author's write-up.

D. Franzen. ARC-AGI-3 Milestone 2: Building on the Duck Harness, 2026b. URL https://github.com/da-fr/arc-agi-3-solution/blob/main/WRITEUP.md. Write-up in the author's repository (WRITEUP.md).

P. Gowtham. arc-skill: an agent skill that plays ARC-AGI-3, 2026. URL https://github.com/pbshgthm/arc-skill. GitHub repository pbshgthm/arc-skill and write-up at arc-skill.vercel.app, August 2026; README accessed 2026-11-01.

C. Gulcehre, T. L. Paine, S. Srinivasan, K. Konyushkova, L. Weerts, A. Sharma, A. Siddhant, A. Ahern, M. Wang, C. Gu, W. Macherey, A. Doucet, O. Firat, and N. de Freitas. Reinforced Self-Training (ReST) for Language Modeling, 2023. URL https://arxiv.org/abs/2308.08998.

guoyi12311125-netizen. Before Tuning the Harness: Repeat-Controlled Evaluation and Request-Level Accounting for ARC-AGI-3, 2026. URL https://github.com/guoyi12311125-netizen/arc3-repeat-controlled-eval. GitHub repository arc3-repeat-controlled-eval (author's name not shown; a 24-page PDF is listed). Read 2026-11-02 from the repository page and README only.

H. He and Thinking Machines Lab. Defeating Nondeterminism in LLM Inference, 2025. URL https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/. Blog post, September 10, 2025.

P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger. Deep Reinforcement Learning that Matters, 2017. URL https://arxiv.org/abs/1709.06560. AAAI 2018.

E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-Rank Adaptation of Large Language Models, 2021. URL https://arxiv.org/abs/2106.09685.

Intel. Qwen3.8-Flash-Next-W4A16-AutoRound (model card), 2026. URL https://huggingface.co/Intel/Qwen3.8-Flash-Next-W4A16-AutoRound. Hugging Face model card, accessed 2026-10-13.

G. Kamradt. OpenAI's GPT-6 Astra on ARC-AGI-3, 2026a. URL https://arcprize.org/blog/astra. ARC Prize blog, published 3 September 2026.

G. Kamradt. Re: Does the 1,500-word limit apply to a PDF submitted via the Public Project Link?, 2026b. URL https://www.kaggle.com/competitions/arc-prize-2026-paper-track/discussion/748431. Host reply of 2026-10-12 in Kaggle forum topic 748431, ARC Prize 2026 - Paper Track (topic opened by Ku Onoda).

S. Kapoor, B. Stroebl, Z. S. Siegel, N. Nadgir, and A. Narayanan. AI Agents That Matter, 2024. URL https://arxiv.org/abs/2407.01502.

M. Lasby, I. Lazarevich, N. Sinnadurai, S. Lie, Y. Ioannou, and V. Thangarasa. REAP the Experts: Why Pruning Prevails for One-Shot MoE compression, 2025. URL https://arxiv.org/abs/2510.13999.

J. Lehmann, A. Aioanei, and S. Vahdati. Tycho: Active Abstraction with Programmatic World Models for ARC-AGI-3, 2026. URL https://arxiv.org/abs/2607.28287.

R. Lencevicius. When the harness decides success: state that cannot be re-observed, 2026. URL https://arxiv.org/abs/2610.26632. Code and pre-registration: github.com/raimondasl/persistgym.

Y. Leviathan, M. Kalman, and Y. Matias. Fast Inference from Transformers via Speculative Decoding, 2022. URL https://arxiv.org/abs/2211.17192. ICML 2023.

Y. Li, H. Jiang, Y. Xu, J. Yang, Y. Zhang, Y. Cao, Y. Shen, F. Zhou, R. Men, J. Zhang, A. Yang, B. Yu, B. Zheng, F. Huang, J. Lin, D. Liu, and J. Zhou. Breaking Entropy Bounds: Accelerating RL Training via MTP with Rejection Sampling, 2026. URL https://arxiv.org/abs/2606.12370.

F. Liang. How noisy is an offline eval of the Franzen M2 harness? Plus two negative results (more streams, stall cutoff), 2026. URL https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3/discussion/750885. Kaggle forum topic 750885, ARC Prize 2026 - ARC-AGI-3, post of 2026-10-20.

X. Liu, J. Park, L. Hu, W. Kwon, Z. Li, C. Zhang, K. Du, X. Mo, K. You, A. Cheung, Z. Deng, I. Stoica, and H. Zhang. TurboSpec: Closed-loop Speculation Control System for Optimizing LLM Serving Goodput, 2024. URL https://arxiv.org/abs/2406.14066.

lovedheart. qsa: widen pending-ring stride to 2x ratio to stop spec-verify clobbering, 2026. URL https://github.com/lovedheart/sglang/commit/427e9380ade10c4bc6a30097879a544305787d2d. Commit 427e9380ade10c4bc6a30097879a544305787d2d in lovedheart/sglang, 2026-09-02.

S. Makarov. Harness or model? Notes from a week of controlled harness experiments, 2026. URL https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3/discussion/743723. Kaggle forum topic 743723, ARC Prize 2026 - ARC-AGI-3, post of 2026-09-26.

E. Miller. Adding Error Bars to Evals: A Statistical Approach to Language Model Evaluations, 2024. URL https://arxiv.org/abs/2411.00640.

N. Muennighoff, Z. Yang, W. Shi, X. L. Li, L. Fei-Fei, H. Hajishirzi, L. Zettlemoyer, P. Liang, E. Candès, and T. Hashimoto. s1: Simple test-time scaling, 2025. URL https://arxiv.org/abs/2501.19393. EMNLP 2025.

R. Pan, Z. Wang, Z. Jia, C. Karakus, L. Zancato, T. Dao, Y. Wang, and R. Netravali. Marconi: Prefix Caching for the Era of Hybrid LLMs, 2024. URL https://arxiv.org/abs/2411.19379. MLSys 2025.

J. Pezzulli. sglang-rtxpro6000 (Pennyroyal), release v2.5.3, 2026. URL https://github.com/jpezzulli/sglang-rtxpro6000. Source repository; commit d00d88e is the build used in Franzen's notebook.

S. Pham. Incredible notebook performance variation (Daniel Franzen's copies data analysis), 2026. URL https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3/discussion/745062. Kaggle forum topic 745062, ARC Prize 2026 - ARC-AGI-3, post of 2026-10-02.

W. T. Piriyakulkij, Y. Liang, H. Tang, A. Weller, M. Kryven, and K. Ellis. PoE-World: Compositional World Modeling with Products of Programmatic Experts, 2025. URL https://arxiv.org/abs/2505.10819. NeurIPS 2025.

J. Pourcel, C. Colas, and P.-Y. Oudeyer. Self-Improving Language Models for Evolutionary Program Synthesis: A Case Study on ARC-AGI, 2025. URL https://arxiv.org/abs/2507.14172. ICML 2025.

Z. Qiu, Z. Wang, X. Li, Y. Li, Y. Xu, Y. Wang, H. Zhang, R. Men, B. Mao, C. Zhang, F. Zhou, H. Luo, H. Huang, H. Lian, H. Huang, H. Chen, J. Zhang, J. Xu, J. Wang, L. Chen, L. Wang, L. Jiang, M. Yuan, M. Sun, P. Jin, S. Zhang, S. Wang, X. Ren, Y. Wang, Y. Zhang, Y. Dong, Y. Cao, Y. Ma, Y. Mao, B. Zheng, and D. Liu. On the Design of Qwen3.8-Next Architecture: Evaluation, Efficiency, and Training Stability, 2026. URL https://arxiv.org/abs/2608.30320.

Qwen Team. Qwen3.8-Flash-Next (model card), 2026. URL https://huggingface.co/Qwen/Qwen3.8-Flash-Next. Hugging Face model card, accessed 2026-10-13.

S. Rodionov. Do Coding Agents Need Executable World Models, Simplification, and Verification to Solve ARC-AGI-3?, 2026a. URL https://arxiv.org/abs/2607.15439.

S. Rodionov. Executable World Models for ARC-AGI-3 in the Era of Coding Agents, 2026b. URL https://arxiv.org/abs/2605.05138. AGI-2026, Lecture Notes in Computer Science vol. 16855, pp. 198-210 (venue per the arXiv comment; volume and pages per the Twin bibliography).

R. Roelofs, V. Shankar, B. Recht, S. Fridovich-Keil, M. Hardt, J. Miller, and L. Schmidt. A Meta-Analysis of Overfitting in Machine Learning. In *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/ee39e503b6bedf0c98c388b7e8589aca-Abstract.html.

E. Rudakov, J. Shock, and B. U. Cowley. Graph-Based Exploration for ARC-AGI-3 Interactive Reasoning Tasks, 2025. URL https://arxiv.org/abs/2512.24156. AAAI 2026 Workshop on AI for Scientific Research. Code: github.com/dolphin-in-a-coma/arc-agi-3-just-explore.

ryanbbrown. Retrodict: Frontier ARC-AGI-3 agent, 2026. URL https://github.com/ryanbbrown/Retrodict. GitHub repository ryanbbrown/Retrodict; title from the repository description, accessed 2026-11-02; contents not reviewed.

R. Sadhukhan, J. Chen, Z. Chen, V. Tiwari, R. Lai, J. Shi, I. E.-H. Yen, A. May, T. Chen, and B. Chen. MagicDec: Breaking the Latency-Throughput Tradeoff for Long Context Generation with Speculative Decoding, 2024. URL https://arxiv.org/abs/2408.11049.

A. Saxena, P.-A. Tsai, H. Taneja, A. Jaleel, and M. Qureshi. Utility-Driven Speculative Decoding for Mixture-of-Experts, 2025. URL https://arxiv.org/abs/2506.20675.

J. Schulman. Approximating KL Divergence, 2020. URL http://joschu.net/blog/kl-approx.html. Blog post.

Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models, 2024. URL https://arxiv.org/abs/2402.03300.

shiiin9. Kaggle notebook shiiin9/affectify-arc-dfranzen-form-d1, 2026. URL https://www.kaggle.com/code/shiiin9/affectify-arc-dfranzen-form-d1. Source of the D-prime priority formula; accessed 2026-10-13.

A. Skoutnev, K. Acharya, G. Longhitano, M. Udell, K. Ellis, and I. Drori. Twin: Playing an Unknown Game with a Test-Time Digital Twin, 2026. URL https://arxiv.org/abs/2608.14490.

H. Tang, D. Key, and K. Ellis. WorldCoder, a Model-Based LLM Agent: Building World Models by Writing Code and Interacting with the Environment, 2024. URL https://arxiv.org/abs/2402.12275. NeurIPS 2024.

P. A. Tsividis, J. Loula, J. Burga, N. Foss, A. Campero, T. Pouncy, S. J. Gershman, and J. B. Tenenbaum. Human-Level Reinforcement Learning through Theory-Based Modeling, Exploration, and Planning, 2021. URL https://arxiv.org/abs/2107.12544.

Tufa Labs. RE-ARC-3: an environment sampler for ARC-AGI-3, 2026. URL https://github.com/Tufalabs/re-arc-3. Source repository, version 0.1.2.

ursuciprian. Qwen3.8-Flash-Next-NVFP4-GDN-MSE (model card), 2026. URL https://huggingface.co/ursuciprian/Qwen3.8-Flash-Next-NVFP4-GDN-MSE. Hugging Face model card, accessed 2026-10-13; source of the D3 MTP refit weights.

A. Warrier, D. Nguyen, M. Naim, M. Jain, Y. Liang, K. Schroeder, C. Yang, J. B. Tenenbaum, S. Vollmer, K. Ellis, and Z. Tavares. Benchmarking World-Model Learning with Environment-Level Queries, 2025. URL https://arxiv.org/abs/2510.19788. ICML 2026; introduces the WorldTest protocol and the AutumnBench benchmark.

W. Wu. Kepler: Auditable World Models for ARC-AGI-3, 2026. URL https://arxiv.org/abs/2610.00834. Non-archival Interpreting Agent Behavior workshop at NeurIPS 2026. Code: github.com/Cveinnt/kepler.

xz. RTX PRO 6000 sessions: startup takes 10-60 min, and slow-starting instances run about 10% slower, 2026. URL https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3/discussion/752485. Kaggle forum topic 752485, ARC Prize 2026 - ARC-AGI-3, post of 2026-10-25 and replies of 2026-10-29 (Kaggle user xiaoz259).

J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering, 2024a. URL https://arxiv.org/abs/2405.15793. NeurIPS 2024.

S. Yang, J. Kautz, and A. Hatamizadeh. Gated Delta Networks: Improving Mamba2 with Delta Rule, 2024b. URL https://arxiv.org/abs/2412.06464. ICLR 2025.

Q. Yu, Z. Zhang, R. Zhu, Y. Yuan, X. Zuo, Y. Yue, W. Dai, T. Fan, G. Liu, L. Liu, X. Liu, H. Lin, Z. Lin, B. Ma, G. Sheng, Y. Tong, C. Zhang, M. Zhang, W. Zhang, H. Zhu, J. Zhu, J. Chen, J. Chen, C. Wang, H. Yu, Y. Song, X. Wei, H. Zhou, J. Liu, W.-Y. Ma, Y.-Q. Zhang, L. Yan, M. Qiao, Y. Wu, and M. Wang. DAPO: An Open-Source LLM Reinforcement Learning System at Scale, 2025. URL https://arxiv.org/abs/2503.14476.

J. Yuan, H. Li, X. Ding, W. Xie, Y.-J. Li, W. Zhao, K. Wan, J. Shi, X. Hu, and Z. Liu. Understanding and Mitigating Numerical Sources of Nondeterminism in LLM Inference, 2025. URL https://arxiv.org/abs/2506.09501. NeurIPS 2025.

Z. Yuan, H. Yuan, C. Li, G. Dong, K. Lu, C. Tan, C. Zhou, and J. Zhou. Scaling Relationship on Learning Mathematical Reasoning with Large Language Models, 2023. URL https://arxiv.org/abs/2308.01825.

E. Zelikman, Y. Wu, J. Mu, and N. D. Goodman. STaR: Bootstrapping Reasoning With Reasoning, 2022. URL https://arxiv.org/abs/2203.14465.

W. Zhao, T. Pan, X. Han, Y. Zhang, A. Sun, Y. Huang, K. Zhang, W. Zhao, Y. Li, J. Wang, Z. Liu, and M. Sun. FR-Spec: Accelerating Large-Vocabulary Language Models via Frequency-Ranked Speculative Sampling, 2025. URL https://arxiv.org/abs/2502.14856. ACL 2025.

L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng. SGLang: Efficient Execution of Structured Language Model Programs, 2023. URL https://arxiv.org/abs/2312.07104. NeurIPS.

A Components: inherited, borrowed, ours

Table 2 groups our 19 changes with their origin, and Table 3 lists every part of the configuration behind the 40.98 and 40.09 draws, called `l1a1y8sv2r3p` in our records (kernel version 1). Rows U1 to U7 are inherited parts that we did not re-measure; rows O1 to O19 are our changes, in build order, and their Origin cells say what each borrows. The notebook differs from the base fork in 5 of its 26 cells, through 29 asserted exact-string edits; 25 of them are active in the scored run.

For most changes the effect is a mechanism and not a level count: at two runs per arm a development comparison resolves shifts of about 23 levels (Sec. 3).

Changes since 2026-10-13. The pin moved to a draft-vocabulary variant (`l1a1y8sv2r3hm32p`, row O21) on 2026-10-14 after its gate passed, back to the configuration above for the 2026-10-20 draw, and on 2026-10-20 to that configuration plus the warmup keepalive (`l1a1y8sv2r3kp`, row O20). Neither change touches play. A fallback build that mounts our copies of the rival inputs (`l1a1y8sv2r3fbp`) passed its probe and stays unpinned. Later pins add 15 games in flight (2026-10-28) and the harness changes of Sec. 6: the stall check and corner-joined regions (2026-10-31), the tool fixes (2026-11-01) and the replay check (2026-11-02, the final draw).

Table 3: Upstream parts (U) and our changes (O) in the configuration behind the best draws (O20 and O21 came later). Third-party part: what inside the row is not ours. Tier: evidence class, as in Table 2. Levels are replica levels from single runs unless a count is given. r1, r2 and r3 are the last three serving runs: r1 lacks the drafter refit, r2 carries one unshipped overlay, r3 is the shipped set.

ID	Component	Origin and credit	Third-party part inside	Tier	Measured effect
Upstream parts, taken as given					
U1	Qwen3.8-Flash-Next. 48 layers, 3 Gated DeltaNet to 1 Qwen Sparse Attention; 512 experts, 10 routed and 1 shared active; 180B parameters, 6B active (125B core, 51B n-gram embedding, 4B multi-token prediction).	Qwen [Qwen Team, 2026, Qiu et al., 2026, Yang et al., 2024b]	Whole part	none	Franzen reports that each model switch "brought a substantial improvement"; not measured here.
U2	Intel W4A16 AutoRound weights. 4-bit weights, 16-bit activations; the BF16 n-gram table sits in host RAM.	Intel [Intel, 2026]; Kaggle copy by Franzen	Derivative of the Qwen weights	none	Franzen: "similar quality and throughput" to the NVFP4 build, with more memory for the KV cache (his claim).
U3	Multi-token-prediction drafter. Quantized draft checkpoint, 3 draft steps, 64K draft vocabulary.	albucino [albucino, 2026]; vocabulary map from the Pennyroyal build	Derivative of the Qwen weights	none	Accept length 2.70 when our work began.
U4	Pennyroyal SGLang 2.5.3 (commit `d00d88e`). Offline serving with a prefix cache that holds Mamba checkpoints, chunked prefill of 8,192 tokens and an FP8 KV cache.	J. Pezzulli [Pezzulli, 2026, Zheng et al., 2023]; patches by Franzen	Franzen's patches come from the Olympie fork (low-M BF16 GEMM; no licence file) and Ratsimbazafy's fork (memory-budget fix)	none	Franzen's earlier demo reused 93.4% of prompt tokens at 609 tokens/s on the harness (his report).

ID	Component	Origin and credit	Third-party part inside	Tier	Measured effect
U5	Duck harness. One `python` tool plus `action()` in a sandboxed REPL; thinking kept in history; a yield on a time threshold (60 s by default).	Tufa Labs (Cottaar), commit `7652836` [Bessis et al., 2026]	Whole part; no LICENSE file (Table 35)	none	Milestone 1 winner.
U6	Franzen's harness patch. 16 files, +8,405 and −197 lines: 10x board images with diffs, animation frames, UNDO, no-op and stale-state guards, block trimming, admission limit, priority scheduler, token-triggered yield (2,048 in his configuration).	D. Franzen [Franzen, 2026a,b]	Built on U5	none	His public score 27.89; copies of his notebook, assumed unchanged, average 25.77 (SD 3.93; n not stated) [Pham, 2026].
U7	Scored schedule. 110 games, concurrency 120, a 532 min ceiling per game.	Franzen	none	none	Not measured.
Our changes					
O1	Startup trims. Model precache starts at server launch (delay 0, not 60 s); a 30 s import is dropped.	Ours	Short anchor strings quote Franzen's cells	S, P	Boot until the server is ready 553 to 488 s (one pair, run with a 12-stream arm that failed). Lossless.
O2	REAP keep-448 experts (`p448_serve`). The loader drops 64 routed experts per layer, renumbers the rest and slices the router.	Method: REAP [Lasby et al., 2025]. Kept list: lee-chang-93's pruned checkpoint. Recovering the list from its router rows, the renumbering and the patch: ours.	The 48 by 64 table of dropped expert ids; the saliency was calibrated on another agentic deployment's traffic, not ARC.	R, S, T	Frees 7.31 GiB for the KV pool. Tokens/s +13.7%; levels 122 and 106 against controls 92 and 97, two runs per arm (SD 8.25), so the level gain is uncertain. Lossy.
O3	14 streams, 70 Mamba slots. Harness admission limit, server request cap, CUDA graphs for 11 to 14 requests, 5 Mamba slots per stream.	Ours. The value 14 also appears in Lord Han Solo's vLLM configuration.	none	R, S	Tokens/s +11.8%; levels tie (two runs per arm). Shipped on a tie with a verified mechanism.
O4	D-prime priority. Replaces Franzen's priority function and keeps his admission flow.	Formula: shiiin9 [shiiin9, 2026]. Choosing, embedding and simulating it: ours.	The whole formula, verbatim (252 lines, Japanese comments)	M, P	Simulator only (1,862 recorded attempts; 110 games, 14 slots): +2.81 points over Franzen's rule, paired SE 0.45, +2.6 to +3.3 across variants; +2.6 on the 1,114 attempts of eight replica runs; equal share beats it by 1.1 on the replica-like world. The replica cannot resolve it. Insurance, not a measured gain.

ID	Component	Origin and credit	Third-party part inside	Tier	Measured effect
O5	wastefix. Host action time no longer counts against the 30 s snippet deadline; a killed snippet no longer clears retained functions (24 timeouts had cleared 629); allowed standard-library modules are pre-bound (19 tracebacks).	Ours	Anchor strings quote Franzen's and Tufa's source	B, T	With O6 and O7: 111.0 levels against 112.0 for the control, two replica runs each, a tie. Its offline test has a file-name length defect in the composition step; the overlay's own checks pass.
O6	replycache. Re-sends the model's reasoning exactly as the server returned it, blank lines kept, so the prefix cache can serve the previous reply (a hybrid model needs an exact prefix match [Pan et al., 2024]).	Idea: face-of-agi's public verbatim re-send, which removes blank lines. Exact bytes, blank lines kept: ours.	none	S, T	Previous-reply cache reuse 38% to 93%.
O7	agentfix (with RESET exposed). RESET is skipped when its level is untouched; a caught stale-state refusal waives the next 6 no-ops; `level_starts` and `attempt_starts` globals.	Ours	none	B, T	With O5 and O6: burn-to-reset episodes 0 and 0 against 11 and 7 in an earlier arm; 39 to 45 manual RESETs, none lost a level.
O8	Yield threshold of 8,192 generated tokens. A yield ends one invocation after N generated tokens; the harness resumes the same turn with a re-sent opener and board image. Franzen's N is 2,048.	Duck: yield on a time threshold (60 s). Franzen: token-triggered, 2,048. Lord Han Solo: rarer yield (150 s, no token yield; public notebook). The value 8,192 and the measurement: ours.	none (one configuration line)	R, B	Runs at 8,192 cleared 12.5 more levels than runs at 2,048 (nine against 21 runs, Table 8). The runs were not interleaved and the nine generate about 15% more tokens, and an interleaved test removed the gain (Sec. 5). Resume openers 565, 634 and 572 against 151 to 215, about 70% fewer. The freed prefill is at most 2.3% of GPU time by arithmetic and 0.6 to 0.8% by the server's prefill timers, worth 0.4 to 0.5 levels by the timers and at most 1.4 by arithmetic.
O9	QSA ring fix (`qsaring_serve`). Sets the pending-ring stride to twice the ratio so a 4-token verify window cannot overwrite a boundary group's index keys.	Fix: lovedheart's commit `427e938`, authored 2026-09-02 and committed 2026-10-06 [lovedheart, 2026]; their draft-width guard is 2 times the ratio. Port to our wheel and the tighter guard (ratio plus 1): ours.	Derived from Apache-2.0 code; stays Apache-2.0	T, P, R	Replay on our wheel: 192 of 256 groups corrupted with the unpatched stride, 0 patched (a share of mid-group window alignments, not a traffic rate). The commit reports the same kind of sweep (90 of 118 groups, 0 after). The GPU probe was invalid by its own rule (1 and 2 of 16 prompts reused keys, 12 required). Effect unmeasured on the GPU.
O10	Asynchronous M-RoPE upload (`asyncmrope_serve`). A pinned, non-blocking copy of the speculative M-RoPE deltas.	Idea: Son Pham's team (repository public until 2026-10-11; no licence). Code: ours.	none	T, S	Lossless by design. No separate measurement; promoted together with O9.

ID	Component	Origin and credit	Third-party part inside	Tier	Measured effect
O11	Image hashes off the scheduler thread (`SGLANG_MM_PRECOMPUTE_HASH`).	Flag and idea: Son Pham's team. Finding, promotion test, setting it: ours.	An existing SGLang flag	S	Scheduler stall 13.4 to 1.7 ms per 10K cached tokens; small prefill at 60 to 120K context 194 to 90 ms; decode share 81.2% to 84.6% (one run each, before and after, server-log estimator). Same hash values.
O12	Memory fraction 0.955 (Franzen: 0.96). +0.47 GiB of runtime headroom for 2.3% of the KV pool.	Ours. Franzen's write-up lists 0.955 for an earlier demo configuration.	none	S	Near-miss warning in 23 of 29 runs before and 0 of 4 after. One unplanned CUDA OOM after the server was ready in 34 full-length runs (exact 97.5% upper bound 15.3% per run); none in r1 to r3. Not a level claim.
O13	Server watchdog. Relaunches a dead server (at most 3), reuses the precache, retries requests in place for 30 min.	Ours	none	P, T	Rehearsal on the GPU: server killed 60 s after it was ready, ready again after 346 s, and 1,810 of 1,862 actions and all 47 levels came after the relaunch. Never fired in a replica; unknown in scored runs.
O14	toolfast. Incremental runtime-state write and a per-frame memo.	Ours	none	S, T	In-batch host gaps 3,410 s to 563 s (r1), 485 (r2), 344 (r3); sandbox payload byte-identical at all 162 test steps; per action 99 to 18 ms at 100 history entries.
O15	Sandbox memory cap. An 8 GiB address-space limit and a raised kill priority in the python child, so a host OOM kills the snippet and not the server.	Idea shared with Son Pham's team. Code: ours.	none	S	0 MemoryError in 17,508 recorded calls before the cap and 0 in r1 to r3, so the guard has not been seen to act. A crash guard, no level claim.
O16	D3 drafter refit (`d3draft_serve`). At boot, overwrites the drafter's dense tensors (29 in albucino's file; the refit card says 24); shapes, dtypes, byte sizes and sha256 asserted.	Weights: ursuciprian's KL refit [ursuciprian, 2026], trained against a 4-bit NVFP4 build, not our W4A16 build; nearest prior art for the recipe is FastMTP [Cai et al., 2025]. Splice code and the measurement: ours.	The dense tensors (Qwen Community License; training mix of code, tool calls, chat and math, no ARC data)	S, T	Accept length 2.70 to 2.83: +0.130 $[+0.108, +0.151]$ (+4.8%); decode +50.5 tokens/s $[+39.7, +61.2]$ (+5.5%); step time +0.18 ms $[-0.04, +0.36]$. A probe pair isolating D3 gives +0.133 $[+0.076, +0.181]$; the r1 to r3 contrast also carries O17. Boot +40 s. Lossless by design: the target verifies every token.
O17	promptobs lite. Types the animation fields in the prompt, accepts U, D, L, R as action names, forwards history-view attributes.	Ours	none	B, T	Counters in r2: `int_len` 7 to 0, `unknown_udlr` 7 to 0, `view_attr` 2 to 0; r3 shows 3 `int_len` matches, one of them a raised error. We expect about 0 levels.
O18	Fail-closed at boot. In a scored run, a server that exits before it is ready raises at once.	Ours	none	T, X	Built after two 0.00 draws, both later shown to be field-wide Kaggle events (63 of 64 and 48 of 61 early endings in the 00:00 to 00:20Z wave). Never fired on a scored run. No score claim.

ID	Component	Origin and credit	Third-party part inside	Tier	Measured effect
O19	Fail-closed at the end. A scored run that ends under 20 levels raises, so Kaggle records an error, which does not spend the daily slot (shown 2026-09-30).	Ours	none	T, X	Never fired on a scored run; a run Kaggle kills from outside is not caught. No score claim.
O20	Warmup keepalive (`keepalive`). After the warmup RESET, a daemon thread re-sends RESET on the warmup game every 9 minutes while the server is not ready, at most 5 times, and stops at READY, the first reply or the game's first action.	Recipe: Jeroen Cottaar's forum post (a thread that resets one game every 10 minutes until startup) [Cottaar, 2026a]. Code and offline test: ours.	none	T, P	Offline: without it the scorecard closed 15.05 minutes after the warmup RESET; with it the card stayed open to 60.12 minutes. Kaggle probe: the thread stopped after 0 RESETs because the server was ready within 9 minutes. Pinned 2026-10-20, after the 40.98 draw. No score claim.
O21	Own-traffic draft vocabulary (`hm32`). A 32,768-id draft map built from our own transcripts replaces the served 64K map.	Idea: FR-Spec and FastMTP [Zhao et al., 2025, Cai et al., 2025]. Map and gate: ours.	none	S, P	Predeclared gate passed: decode $+1.2\%$ at 14 running requests (predicted $+1.4\%$ to $+1.7\%$), step 36.6 to 36.0 ms, accept length $+0.3\%$; one run against one control. Lossless by construction. Pinned 2026-10-14 to 2026-10-19; not in the 40.98 configuration.

Three more things sit in the notebook outside the table: a GPU check in the first cell that aborts unless the GPU has compute capability 8.9 and 80 GB, the wheelhouse mount path, and an attribution cell that we added. The inputs are read-only: the Pennyroyal build (6.70 GB), the harness source bundle, our D3 dataset (181 MB), Intel's weights (181.24 GB) and albucino's drafter (4.16 GB). Four of the five are Franzen's copies on Kaggle.

Where the plain credits point. Franzen's write-up [Franzen, 2026b] links the two patches in U4: Olympie's low-M GEMM patch (`github.com/gabrielolympie/sglang-flashnext-sm120`, patch 0004) and Ratsimbazafy's memory-budget change (`github.com/mratsim/sglang-qwen38fn-sm120-turbo`, patch 0002). The pruned checkpoint behind O2 is `huggingface.co/lee-chang-93/Qwen3.8-Flash-Next-NVFP4-REAP-k448`. face-of-agi and Lord Han Solo are credited for ideas from their public Kaggle notebooks, as the Origin cells say.

Table 2: The 19 changes of ours in six groups, with origin, evidence tier and the effect we can defend. Tiers: R replica levels, S server-log counters, B behaviour counters, M scheduler simulator, P short probe run, T offline test, X incident or code reading. Effects without level numbers are mechanisms only. Replica numbers are single runs unless a count is given.

Members	Origin	Tier	Effect we can defend
Capacity (3). Expert pruning to 448 of 512 per layer; 14 admitted streams with 70 Mamba slots; memory fraction 0.955.	REAP [Lasby et al., 2025]; kept-expert list from lee-chang-93's pruned checkpoint, recovered by us. Patch and stream cap ours. The value 14 also appears in Lord Han Solo's vLLM setup and 0.955 in an earlier Franzen demo.	R, S, T	Tokens per second +13.7% (pruning) and +11.8% (14 streams). Pruning is lossy: levels 122 and 106 against controls 92 and 97, two runs per arm, which is +19.5 on the arm means at +11.2% generated tokens (Table 15). The level gain is uncertain.
Host and serving path (5). Startup trims; image hashes off the scheduler thread; asynchronous M-RoPE upload; incremental host-state write with a frame memo; drafter refit.	Hash flag and upload idea: Son Pham's team. Refit weights: ursuciprian [ursuciprian, 2026]; drafter: albucino. Splice code and the rest ours.	S, T, P	Lossless by design. Scheduler stall 13.4 to 1.7 ms per 10K cached tokens; host gaps 3,410 s to between 344 and 563 s; accept length 2.70 to 2.83. Harness tokens per second 800 to 885 (+10.6%) from the hash flag, toolfast and the refit together; the upload is already in the 800. Single runs.
Context economy (2). Verbatim reasoning round trip; yield threshold of 8,192 generated tokens instead of 2,048.	Duck: reasoning kept in history, time-based yield (60 s). Franzen: token-triggered yield (2,048). face-of-agi: verbatim re-send. Lord Han Solo: rarer yield (150 s). Ours: blank lines kept in the re-send; the 8,192 value.	R, B, S, T	Previous-reply cache reuse 38% to 93%. About 70% fewer resume openers. Runs at 8,192 cleared 12.5 more levels than runs at 2,048 (nine against 21 runs); the cause is open (Sec. 5).
Agent interface (3). Waste fixes (host time off the snippet deadline, retained functions kept); RESET exposed but skipped on an untouched level; typed animation fields and U/D/L/R names.	Franzen's harness; edits ours.	B, T	Waste and error classes removed (counters). With the reply round trip, levels tie: 111.0 against 112.0, two runs each.
Scheduling (1). Priority formula D-prime replaces Franzen's priority value; his admission flow is kept.	shiiin9 (the whole formula, embedded verbatim) [shiiin9, 2026]; Franzen (admission flow); choice and simulation ours.	M, P	+2.6 to +3.3 points over Franzen's rule in the simulator, which the replica cannot resolve. Insurance, not a measured gain.
Robustness (5). Sparse-attention (QSA) index-key fix; server watchdog; sandbox memory cap; fail-closed checks at boot and at the end of the run.	lovedheart (the fix, commit 427e938 [lovedheart, 2026]; their guard is 2 times the ratio, ours is tighter); memory-cap idea shared with Son Pham's team; the rest ours.	R, S, T, P, X	No score claim. Watchdog rehearsal on the GPU: server loaded again after 346 s, games resumed. The effect of the QSA fix on levels is unmeasured.

B Claims and evidence

Table 4 lists the claims behind the numbers in the paper, each with its source (a script, a data file or a record), a tier, a strength and the hedge it carries. Rows without a prefix are the v2 map of 2026-10-13 (sources T, M, Y, S, E and B name its evidence notes); rows marked V3 come from the evidence dossier of 2026-10-21 and from rows added while writing this version (V3-W). Where a later row updates a v2 row, the v2 row says so. Our records call a development run a replica, and the ceiling of Sec. 4 a plateau; this table keeps the records' words. Every number in the body carries the ids of its rows in a source comment, and `paper/v3/analysis/build_claims_appendix.py --check` lists any id cited without a row.

Table 4: Claims and evidence map: v2 rows (ids without a prefix, through 2026-10-13) and V3 rows (2026-10-13 to 2026-10-21). Tier: measured = counted or fitted from recorded runs by a script; derived = arithmetic or a model on measured numbers; external = another party's statement; owner-reported = a project record we read and did not recompute. Strength: strong = exact or replicated; moderate = simulator or observational with stated limits; weak = single estimate, inference or model-dependent. Check: sources T, M, Y, S, E, B and L are evidence notes (scoring theory, measurement, yield, speculative decoding, system, board, time ledger), N1 to N3 later analyses, s a section; read = compared with the cited file or script output, recomputed = re-derived independently.

ID	Claim, source, tier and hedge
v2 A. Setting and score	
A1	Equation 1 restates the published scorer and matches its code on 1,425 of 1,425 recorded plays. *Source:* T C1; e5_scorer.py, read. *Tier:* measured; strong. *Hedge:* An algebraic restatement; completion share carries the score.
A2	On the first three replica runs at the 8,192-token yield, 94% of the points a run loses are levels never reached, and efficiency is 0.96. *Source:* T C3; e5_scorer.py; N3-10, read. *Tier:* measured; strong. *Hedge:* Three runs of one lineage, not the shipped set.
A3	The replica is the 25 public games on 14 streams in a shared 7,200 s window: about 257K generated tokens per game against an estimated 258K in the scored run, and 183 levels. The same program run twice cleared 123 and then 101 of 183. *Source:* T s7; M s2; e4_noise.txt, read. *Tier:* measured; strong. *Hedge:* Development set, never held out; the numbers may overstate fresh-game levels.
A4	The entry is Franzen's Milestone 2 notebook plus 19 changes of ours (O1 to O19) on 7 upstream parts (U1 to U7): 29 exact-string edits to 5 of 26 cells, 25 active in the scored run; the scored branch of the schedule cell is unchanged. *Source:* E s0, s3; e6_pin_components.py, read. *Tier:* measured; strong. *Hedge:* Mechanisms, not level gains: per-component level effects are ties within noise.
v2 B. Instrument: noise, power and gate (Sec. 3)	
B1, B2, I5, I6	Six byte-identical pairs differ by 3 to 22 levels (gaps 3, 4, 4, 6, 16, 22); the pooled SD is 8.25 (6 df, 95% CI 5.3 to 18.2). The second run was lower in all six (two-sided sign test p = 0.031). Around the shared first-minus-second shift (9.2, 62% of the sum of squares) the SD is 5.6; a median-based SD is 5.2; the three pairs planned in advance give 9.2. *Source:* M s3.1; e4_noise.py; N3-7, recomputed. *Tier:* measured; strong data, weak precision. *Hedge:* Six pairs; the CI is printed beside every use. The 8.25 includes an order shift of unknown cause. *Updated in v3:* V3-X3, V3-X4.
B4, I1 to I4, I8	SD table. At 80% power, two-sided 5%, two runs per arm resolve 11.5, 14.7, 15.5, 18.3, 23.1 and 50.9 levels at SD 4.1 (seven yield-8,192 runs), 5.2 (pairs, median-based), 5.5 (21 runs at 2,048), 6.5 (pairs with seven later runs), 8.25 (pairs) and 18.2 (its upper limit); nine runs against 21 resolve about 9. Estimating the SD from two runs gives power 0.35 at 23.1. *Source:* M s3.2; e4_noise.py; N3-1, recomputed. *Tier:* derived; strong. *Hedge:* Conditional on the SD printed beside each MDE. *Updated in v3:* V3-X12.
B5, I7	Null-promoted column: a one-run rule (the arm beats one control run by 10 levels) fires with nothing changed 4%, 9%, 10%, 14%, 20% and 35% of the time at those six SDs. Two of the six pairs differ by 10 or more (95% CI for the share 4% to 78%). *Source:* M s3.4; e4_noise.py, recomputed. *Tier:* derived; strong. *Hedge:* Normal model; empirically 0.167 on 12 labelled comparisons.
B6	Levels serve only as a veto: both of two runs must clear a floor 9 levels below the pooled incumbent mean. With SD 8.25 the floor passes a null change 74% of the time and a true -9 shift 25%. *Source:* M s4; e4_protocol.py, recomputed. *Tier:* derived; moderate. *Hedge:* Screens gross harm only.
B7	A one-run rule (win or loss at 10 levels) made 2 win, 6 loss and 9 tie calls over 17 first-run comparisons; all six loss calls were against three control runs, each the higher run of its identical pair. On the older stack the mean advantage of five promoted arms fell from +8.0 to +2.1 on replication. *Source:* M s3.4; e4_gates.py, read. *Tier:* measured; moderate. *Hedge:* Post hoc; our decisions used more evidence than the rule.
B8	Between-run CV: levels 7.31%; tokens per second 3.23% (1.29% without one host outlier); accept length 0.39%; cache hit 0.36%. *Source:* M s3.6; e4_serving.py, read. *Tier:* measured; strong. *Hedge:* Six pairs. Counters verify a mechanism, not a gain.
B10	Two code paths could bank a dead run as a completed 0.00 (Benchmark.run swallows a solver exception; the READY wait plays on a dead server). Two fail-closed checks now raise; neither has fired on a scored run. An errored submission does not spend the daily slot (two errors on 2026-09-30, then a completed run). *Source:* M s3.7; e4_incidents.py; code reading, read. *Tier:* measured (code reading); strong. *Hedge:* Test read, not rerun; cannot catch a run that dies after 20 levels or is killed from outside.
B11	Field-wide, early endings in the 00:00 to 00:20Z window were 48 of 61 (2026-10-03) and 63 of 64 (10-10) against a 7.4% base rate (140 of 1,880); two of five Saturdays were events. A dead draw is complete, 0.00 and 3,411 bytes (live: 3,810 to 3,907). Of our 11 fires in the window two banked 0.00, none of the 9 later ones did; we now submit from 02:06Z. *Source:* M s3.7; e4_wave_recount.py, e4_board.py; board_draws.csv, read. *Tier:* measured; strong counts. *Hedge:* Cause unknown; the 01:00Z rule has not met a Saturday. *Updated in v3:* V3-B7.
B12	A CUDA out-of-memory error killed the server after READY in 1 of 34 full-length runs (68.6 play-hours): 2.9% per run, exact 97.5% upper bound 15.3%, 12.3% extrapolated to nine hours. A low-memory warning appeared in 23 of 29 runs before memory fraction 0.955 and 0 of 4 after. Watchdog rehearsal: READY again after 346 s. *Source:* M s3.7; e4_incidents.py; E O13, recomputed. *Tier:* measured; moderate. *Hedge:* One event; 12.3% assumes a constant hazard. Only the warning can show 0.955 works.
v2 N3. Noise model and stuck mass (Sec. 3, Appendix L)	
N3-1, N3-3, N3-14	Observed run SDs of total levels are 3.9 (nine yield-8,192 runs), 5.5 (21 controls) and 8.25 (six pairs). A model that clears each level independently at its recorded rate under the replica's token budget predicts 5.2, 6.4 and 5.7 (SE 0.3 to 0.8). *Source:* N3 (new_noise_model.json: observed, pred, tests), recomputed. *Tier:* measured; derived (model output). *Hedge:* Assumes independence across games.
N3-7, N3-8	The model underpredicts the pairs by a factor 1.44 in SD (one-sided p = 0.05 to 0.09). The gap is the pairs' shared shift of 9.2 levels (62% of their sum of squares); around it the SD is 5.6 against the model's 5.7. Tokens per second and slot contention do not explain it. *Source:* N3 s2, s4, read. *Tier:* derived; moderate. *Hedge:* One pair (123 vs 101) carries the dependence; the cause is unidentified.
N3-9	A forum post reports that 84.7% of repetition variance in 500 Milestone 1 runs is whether level n clears (two games hold 65%). On our replica completion explains 84% to 90% of within-game score variance; the two largest games hold 27% to 32%. Another post finds that identical builds differ widely (forum 743723). *Source:* N3 s5; forum topic 732932 as posted, read. *Tier:* external (post); measured (ours). *Hedge:* That our measure matches the post's definition is a reading.

Table 4, continued

ID	Claim, source, tier and hedge
N3-10 to N3-12	A level is never cleared when no attempt in the pool cleared it (35 of 183 for the eight yield runs, 31 for all 30); counting every level any attempt cleared would give 70.9, not the plateau 60.2. Of the 40.5 points a game the nine yield runs lose to unreached levels, 57% sit on levels none of 30 runs cleared, 24% where the run had spent at least the longest recorded clear, 18% where it stalled with fewer tokens. *Source:* N3 s6 (new_noise_model.json: stuck), read. *Tier:* measured; moderate. *Hedge:* At recorded depths; what deeper search would clear is open.
v2 C. Budget curve and plateau (Sec. 4, Appendix L)	
C1	The curve resamples 1,114 level attempts recorded in eight replica runs (seven at yield 8,192). Score is proportional to budget up to rho = 0.7, bends near 0.8 and flattens at a plateau. *Source:* T C8; e5_conversion.py, read. *Tier:* derived (simulator); moderate. *Hedge:* Resamples development-game attempts.
C2, N2, C9	At rho = 1 a replica run earns 94% of the plateau, partly by construction: attempts were recorded at the replica's own budget, so a level no run cleared stays stuck. The plateau rises from 60.2 at eight runs to 61.8 with all 34 replicas pooled, and the share falls with it; extrapolating puts it at 85 to 94%, so we say about 90%. *Source:* N2 (new_plateau_pool.json; plateau table); T C9, read. *Tier:* derived; moderate. *Hedge:* Resampling model with the repo censoring rule. The comment tag C9 in plateau rows is this row. *Updated in v3:* the extrapolated share is 85% to 91% (Table plateau); 94% is the eight-run value, not an extrapolation.
N2-C9	On the older September stack the same measure gives 78% at eight pooled runs (59% at 31), against 94% for ours. *Source:* N2 s3, read. *Tier:* measured; moderate. *Hedge:* Lever experiments, not replicates; run share only.
C3	The elasticity at rho = 1 is 0.31 on levels and 0.22 on score; it averages 0.31 to 0.33 for pools of two to ten runs and spans 0.18 to 0.49 across the 70 four-run subsets (central 95%). Fig. 1a's dashed world has levels costing 2.5 times the tokens. *Source:* T C10; e5_conversion.py B0; new_plateau_pool.json, read. *Tier:* derived; moderate. *Hedge:* Levels and score are quoted separately; the world is a construction.
C4	+11% tokens is worth 4.2 of 124 levels (+5%: 1.9; +26%: 9.9). Seeing 4.2 levels at run SD 4.1 and 8.25 takes 12 and 48 runs per arm one-sided (15 and 61 two-sided), against two in our promotion tests. *Source:* T C22; e5_detect.py, recomputed. *Tier:* derived (model output); moderate. *Hedge:* A prediction: unresolved, not zero. *Updated in v3:* v3 quotes only the two-sided values, at the identical-run SDs 5.95 and 8.25 (V3-W12).
C5, C6	In leave-one-run-out checks the curve is within 1.8 levels at matched tokens (se 1.4 to 2.6) but 4.9 levels low end to end (se 1.5). At the deadline the simulator spends 62.2% of tokens on levels it clears, 30.9% on levels no attempt cleared and 6.9% on clearable levels left unfinished. *Source:* T C7, C21; e5_curvecheck.py; e5_conversion.py B4, read. *Tier:* derived; moderate. *Hedge:* Inside the development set; the end-to-end bias is as large as the 4-level price.
C7	Two matched pools of three runs (yield 2,048 and 8,192, equal token rate) have plateaus 5 to 6 points apart (53.59 and 58.75, +5.16, bootstrap CI +3.2 to +11.2). +11% tokens is worth +0.58 and +0.85 points; one 25-game run has SD 5.0 points. *Source:* T C26; e5_yield_ceiling.py; f1_curves.py; N2 s6, read. *Tier:* derived; weak to moderate. *Hedge:* Classifies the effect in the model's units; three runs per pool.
C8	D-prime (shiiin9's public formula) is 2.6 points above Franzen's earlier rule and 1.1 below equal shares on the eight-run pool. *Source:* T C15; e5_ablation.py, e5_equiv.py; E O4, read. *Tier:* derived; moderate. *Hedge:* Simulator only; insurance, not a measured gain.
v2 N1. Past token-changing steps (Sec. 4, Appendix L)	
N1, N1.1, N1.2	Twelve past changes to tokens per second (generated tokens -15.6% to +11.6%) are set against the curve: each is an arm against its control, with levels and the curve's prediction (0.31 x token change x control levels). *Source:* N1 (new_token_steps.py; steps table rows S01 to S18), read. *Tier:* measured; moderate. *Hedge:* Steps share runs; the curve was built after all of them.
N1.3, N1.4, N1.5, N1.8, N1-s9	Fitted through the origin the twelve steps give an elasticity of 0.45 (95% interval 0.13 to 0.77 at SD 8.25), about 2 to 10 levels for +11% tokens. Expert pruning carries the fit (+19.5 levels at +11.2% tokens against 3.3 predicted); without it 0.02. One step's noise (8 to 12 levels) swamps the curve's prediction (4.5 at most), and none of the 29 promotion-test records predicts levels from tokens. A token-only cut to 70% of the budget predicts about -20 levels; we have not run it. *Source:* N1 s0, N1.3 to N1.5, N1.8, s9; E O2; e5_conversion.py, read. *Tier:* measured; derived. *Hedge:* The steps neither confirm nor refute the curve; pruning is lossy; the cut is a prediction.
v2 D. Yield (Sec. 6, Appendix I)	
D1, YN1	Nine runs at yield 8,192 average 124.0 levels (SD 3.9) and 21 at 2,048 average 111.5 (SD 5.5): +12.5, exact p 2.4e-6 over run labels, bootstrap CI +9.2 to +15.8; at the six-pair SD 8.25 the interval is +4.5 to +20.6 (6 df). The runs were not interleaved. *Source:* Y s3, s5.2; yield_stats.py, recomputed. *Tier:* measured; strong (difference), moderate (effect). *Hedge:* Not interleaved or randomized; p assumes exchangeable run labels. *Updated in v3:* V3-H5.
D2	Eight (without the run whose promotion test is open): +12.3, CI +8.8 to +15.8, p 7.9e-6. Six (also without the 16K-cap and four-draft runs): +13.5, +9.7 to +17.1, 2.4e-5. Later six: +11.5, +7.8 to +15.6, 1.05e-4. Later seven: +12.0, +8.3 to +15.7, 2.7e-5. *Source:* Y s5.2; framing_stats_check.py, recomputed. *Tier:* measured; strong. *Hedge:* No claim rests on the open run. *Updated in v3:* V3-H5.
D3	Slices of the comparison: by UTC day (3 vs 6) +10.9, p = 1/84; against same-stack controls +12.6, pinned-code controls +11.1, Kaggle controls +12.7; per game 22 up, 2 down, 1 tied (p = 3e-4); score gap +6.9 points. We chose the sets, subsets, horizons and outcome. *Source:* Y s5.2, s5.3, s8; yield_stats.py, recomputed. *Tier:* measured; moderate. *Hedge:* Slices of one confounded comparison.
D4, YN5	The first two runs against their two controls: +15.0, p 0.33; that test cannot detect less than 23 levels. Against all 21 controls they are +14.5 (p 0.004), one of 17 first-run comparisons on this stack: if all 17 arms did nothing, the best pair would reach +14.5 about once in ten at SD 8.25 and once in 600 at SD 5.5. *Source:* Y s5.1; yield_stats.py, recomputed. *Tier:* measured; strong (counts), derived (selection). *Hedge:* The gate could not show the effect; the evidence is the pooled family.
D5, D6	Resume openers fell about 70%, from 565 to 634 per run at 2,048 to 151 to 215 at 8,192, while turns, requests per turn and tokens per turn held. Replaying the recorded turns reproduces each run's resume count at its own threshold (546, 587, 523 at 2,048; 185 at 8,192); re-cut at 8,192 the 2,048 traces give 135 to 154 against 151 to 215 observed. *Source:* Y s4; yield_mech.py, read. *Tier:* measured; strong counts. *Hedge:* A yield ends one invocation; the harness resumes the same turn. The cross-yield miss (+20% to +37%) is reported.
D7	The freed prefill is at most 2.3% of GPU time by arithmetic and 0.6% to 0.8% by the server's timers, worth at most 0.4 to 0.5 levels. *Source:* Y s5.4; time_ledger F10, recomputed. *Tier:* derived; moderate (inference). *Hedge:* An upper bound at a generous 11.2 levels per million tokens.
D8, D9, YN2, YN8	The nine runs generate 14.8% more tokens than the 21 controls (the first two match theirs, 5.61M against 5.62M; the later seven generate 17% more). Regressing levels on yield and run tokens (30 runs) gives +12.9 (se 3.1) and -0.5 per million tokens (se 2.9): tokens account for -5 to +4 levels (95%). At the curve's elasticity they are worth about 5 of the 12.5 levels, up to 9 at 11.2 per million tokens. The controls ran 2026-10-03 to 10-10, the nine 10-10 to 10-12. *Source:* Y s5.2, s5.4; yield_stats.py; framing_final_check.py s4, recomputed. *Tier:* measured; derived (token share). *Hedge:* The token slope cannot exclude token volume; the share is a joint effect.

Table 4, continued

ID	Claim, source, tier and hedge
D10	At equal tokens per game the first two runs lead the controls by 9.1 levels at 100K, while the later seven match the controls to 150K and lead only after 200K, once they have generated more: two patterns, not one. *Source:* Y s5.5, s7; yield_stats.py, read. *Tier:* measured; strong descriptive. *Hedge:* Two signatures; not merged into one story.
D12, D13	After a resume the next request acts more often at both thresholds (+0.113 at 2,048 and 400 tokens; +0.396 at 8,192, one run); removing resumes removes the nudge, so it does not explain the gain. Clearance per generated token has a Mantel-Haenszel ratio of 1.073 for the nine against the 21 controls (CI 1.029 to 1.138). *Source:* Y s5.6, s8; yield_rd.py, yield_pertoken.py, read. *Tier:* measured; weak. *Hedge:* Low power; indicative, not a measure of decision quality.
D14	Yielding is in Duck as a time threshold (60 s) and Franzen made it token-triggered at 2,048; Lord Han Solo's public notebook yields rarely (150 s, no token yield). Ours is the 8,192 value and this measurement. *Source:* Y s1; audit_cites C1, C2; E O6, O8, read. *Tier:* external; credit. *Hedge:* Credit is for the rarer yield, not the idea of yielding.
YN3, YN4	Three runs at 8,192 and three at 2,048 from one lineage, with 5.67M and 5.62M tokens, average 123.7 and 110.3 levels (+13.3, exact p = 0.05, the least possible). One pair started within three minutes (04:08 and 04:11 UTC, 2026-10-10): 128 and 109. *Source:* Y s5; yield_runs.json, recomputed. *Tier:* measured; weak. *Hedge:* Confounded by a cloud VM, a thinking-budget change and two serving fixes.
YN6	The settling test, 8 runs per arm at SD 8.25, detects +12.5 with power 0.86 and +7.4 (the gap left after the token share) with power 0.43; +7.4 at 80% power takes 20 runs per arm at SD 8.25, 13 at 6.5 and 9 at 5.5: about $160 for 40 runs at $4 each. We have not run it. *Source:* Y s7; framing_final_check.py, recomputed. *Tier:* derived; moderate. *Hedge:* Not run; the $4 per run is owner-reported (H1). *Updated in v3:* V3-X13.
YN7	Of 34 full-length replica runs 30 enter the comparison. Left out: three older-stack runs (92, 97, 90 levels), the run without a token yield (115 levels; out of memory at 6,893 s) and one 8,192 run that died at start (0 levels; counted, the ten-run mean is 111.6). *Source:* Y s2; yield_runs.json; e4_incidents.txt, recomputed. *Tier:* measured; strong. *Hedge:* The choice of runs is ours; both exclusions are listed. *Updated in v3:* V3-H5.
v2 E. Speculative decoding (Sec. 5, Appendix J)	
E1	Under sampling the server accepts a drafted token with the probability the target gives it, which caps acceptance at the target's maximum probability (Bebop, Proposition 1). We read the rule in the shipped wheel and the kernel on SGLang main, not the binary; we draft a chain; we did not measure the gap to the cap. *Source:* S s2 P1, read. *Tier:* external (rule); measured (code reading). *Hedge:* The cap is Bebop's and the SGLang docs'; ours is the code reading. *Updated in v3:* V3-T7.
E2, E3	A_1 to A_3 = 0.7734, 0.5917, 0.4521 from 479 replies and 374,855 verify steps of 5 of 25 games (conditional 0.773, 0.765, 0.764). Greedy accept length is 3.221 against 2.59 to 2.61 at temperature 0.7. The vendor reports 4.06 to 4.07 at four draft steps on benchmark text; ours is 2.83 at three on agent traffic. *Source:* S s3, s4a; spec_hist.py, spec_greedy.py, read. *Tier:* measured (ours); external (vendor). *Hedge:* One unadjudicated run, counts only; greedy and sampled text differ.
E4	Step time is 20.92 [20.29, 21.45] + 1.120 [1.082, 1.168] R ms over 8,132 clean intervals (R = 8 to 14); with context 18.21 + 1.163 R + 0.280 ms per 10K tokens (+7% from under 20K to 80K to 100K at R = 14). A fourth step adds 3.58, 3.59, 4.61 and 4.93 ms at R = 11 to 14. *Source:* S s4b; spec_step.py, read. *Tier:* measured; strong. *Hedge:* Decode-line arithmetic, not kernel timers.
E5	A fourth draft step raises accept length 9.8% (+0.276 [0.246, 0.303]) and the step 4.93 ms [4.76, 5.12] (+13.5%), and lowers decode throughput 4.2% (-40.9 tok/s [-49.1, -32.3]); harness tokens per second fall 2.8% (861 against 885). *Source:* S s4c; spec_contrasts.py; gate record (harness tok/s), read. *Tier:* measured; moderate to strong. *Hedge:* One run against two controls at one operating point (14 streams, temperature 0.7).
E6, E11, E12	Break-even for a fourth draft is 2.814 x 4.93/36.6 = 0.379; the measured A_4 is 0.275, 21% below the geometric 0.350. Given three accepted drafts the fourth is accepted with probability 0.61 against 0.77 at positions 1 to 3; it would need 0.838. We declared a prediction of +2 to +3% decode rate before the run. A per-position count above 0.379 would reverse the verdict. *Source:* S s4c, P3; spec_model.py; gate-predeclared file (before the push), recomputed. *Tier:* derived; moderate. *Hedge:* 0.275 is a difference of two runs; per-position four-draft counts were not collected. *Updated in v3:* V3-T7.
E7	ursuciprian's refit of the drafter's dense tensors (29 in albucino's file, 24 in the card), transplanted at boot, raises accept length from 2.699 to 2.830 (+0.130 [0.108, 0.151], +4.8%) and decode 50.5 tok/s [39.7, 61.2] (+5.5%) at an unchanged step (+0.18 ms [-0.04, 0.36]); a probe pair isolating the refit agrees (+0.133 [0.076, 0.181]). *Source:* S s4e; spec_contrasts.py, read. *Tier:* measured; strong. *Hedge:* The replica pair also changes the prompt format. The refit is ursuciprian's.
E8	The served 64K draft vocabulary misses 1.416% of 41,560,633 generated tokens; a leave-one-game-out rebuild misses 0.106% (32K) and 0.038% (64K). We estimate the gain at 0.4% to 1.0% of accept length and claim none. *Source:* S s4d; spec_vocab.py, read. *Tier:* measured coverage; derived effect. *Hedge:* The idea is FR-Spec's and FastMTP's; tokens are re-tokenized text, a proxy. *Updated in v3:* V3-H2, V3-H4.
E9	BF16 draft KV: probe-pair accept length +0.010 [-0.040, 0.062] at 8 or more running requests (+0.027 at 14; null pair -0.012); it costs 8% of the KV pool, and a full-size run hit out-of-memory seven seconds into play. *Source:* S s4f; spec_contrasts.py, read. *Tier:* measured; moderate. *Hedge:* Null at probe scale; the probes carry the unadjudicated 32K map.
E10	The QSA index-key fix is lovedheart's. Replayed on our wheel, 4-token verify windows corrupt 192 of 256 groups unpatched and 0 patched (3/4, the share of mid-group alignments, not a traffic rate); windows up to 5 rows are clean and 6 wrap. The GPU probe was invalid by its own rule (1 and 2 of 16 prompts reused keys, 12 required), so the effect on the GPU is unmeasured. *Source:* S s4g; test_franzen_qsaring.py; commit 427e938, read. *Tier:* measured (code replay); strong. *Hedge:* Unmeasured on the GPU, not absent.
E13, E14	The step-time fit spans R = 8 to 14 and the four-draft run 11 to 14. Three cost models for the added row cross the needed gain at R of about 3, 8 and 11, so we do not say where the fourth step pays. A coverage model predicts +2.6 ms for the fourth step at R = 14 against 4.9 measured; expert bytes alone give 2.2 to 2.3 ms. *Source:* S s3, s4c; spec_model.py; fig_spec.py, recomputed. *Tier:* derived (extrapolation); weak. *Hedge:* No four-draft data below R = 11; the coverage model is rejected as the whole account.
v2 F. System and time (Sec. 5, Appendix K)	
F1, F2, F12	Host-path changes: scheduler stall 13.4 to 1.7 ms per 10K cached tokens, decode share 81.2% to 84.6%; with the refit, harness tokens per second rise 11% (800 to 885). REAP keep-448: levels 122 and 106 against controls 92 and 97 (two runs per arm), tokens per second +13.7%; 14 streams: +11.8%, levels tie. *Source:* E O2, O3, O11, O14; time_ledger.py; N1 S02, S03, read. *Tier:* measured (single runs); strong counts. *Hedge:* Single runs; the level gain of pruning is uncertain.
F3	Tried and not shipped, with their numbers: relaxed acceptance (levels 113 vs 114), 16 streams with a host-RAM KV tier (115 vs 114; later line 104 vs 123), 12 streams without pruning (cache hit 72.2%, 90 levels), strict thinking budget (109), 16K output cap (120, 26 cut replies vs 20), keepdata (2 of 6 counters failed), solved-level memory, strategy audit, early handover, opener port (111, 110, 108, 105 vs 122, 122, 122, 114), 71K history (113 vs 114). *Source:* E s5; e4_gates.py, read. *Tier:* measured (read); moderate. *Hedge:* Single runs; levels screen and do not settle. *Updated in v3:* V3-L9, V3-L10, V3-T3.
F4, F5, F6	In run 3 decode steps take 6,146 s of 7,200 (85.4%), new-token prefill 722 s, batch setup 298 s, scheduler stall 38 s; in the 2,048 control decode is 5,867 s (81.5%) and the stall 306 s. Prefill-side GPU time is 1,287 s in the control, 1,217 s in an 8,192 run before the later changes and 1,005 to 1,058 s in three runs with them. *Source:* L s0 to s3; time_ledger.py, read. *Tier:* measured; moderate. *Hedge:* Log-line arithmetic on single runs, no kernel timers; that image hashing causes the stall rests on the flag that removes it.

Table 4, continued

ID	Claim, source, tier and hedge
F7	At 14 streams the decode ceiling is 1,083 tokens per second (14 running requests x accept length 2.830 over the 36.6 ms step) and run 3 reaches 892; zero non-decode time would add at most 21% tokens, about 8 levels at the curve's elasticity. *Source:* L F7; time_ledger.py, recomputed. *Tier:* derived (bound); weak. *Hedge:* An upper bound (inference).
F8, F9	Stream time, run 3: GPU decoding 79.9%, waiting for non-decode GPU work 13.2%, own prefill 1.1%, queued 0.4%, no request on the server 5.4% (5.1% not logged). No-prefill intervals hold -22 to +42 s of extra time against a noise of about 20 s, so idle GPU time cannot be told from model error below about 40 s. *Source:* L s0.5, s0.6; time_ledger.py, read. *Tier:* measured; strong (server phases), weak (stalled row). *Hedge:* The not-logged time has no timer.
F10, F11	Direct prefill seconds are 43 to 58 s lower (0.6% to 0.8% of the window) in the 8,192 pair than in the 2,048 controls, against 163 s by arithmetic; the two cloud controls differ by 109 s. From the 8,192 run before the later changes to run 3, tokens per second rise by 1.108x = decode share 1.033, accept length 1.046, running requests 1.055, step cost 0.972. *Source:* L s0.4, F10, F11; time_ledger.py, read. *Tier:* measured; derived (identity). *Hedge:* Inside the control spread; the factors locate the gain, they do not attribute it.
v2 G. Leaderboard (Sec. 7, Appendix C)	
G1	Eleven draws have completed since 2026-10-03: nine live and two dead (10-03 and 10-10, 0.00). The seven live draws before the yield change are 27.15 to 31.90; the two after are 34.34 and 40.09. Before that: two Kaggle errors on 2026-09-30 (slot not spent) and three September-stack draws. *Source:* B s2; e7_board.py; board_draws.csv, recomputed. *Tier:* measured; strong. *Hedge:* All eleven are listed. *Updated in v3:* V3-B1, V3-B7.
G2	The seven live draws before the yield change average 29.25 (SD 1.58, 95% interval 1.02 to 3.47). Copies of the base notebook average 25.77 (SD 3.93) in one forum snapshot, assumed unchanged, n not stated. *Source:* B s3, s4; board_stats.json, recomputed. *Tier:* measured; moderate. *Hedge:* One snapshot of unverified copies (forum 745062).
G3, G5	34.34 and 40.09 exceed all seven earlier live draws (largest 31.90): exact one-sided p = 1/36; the two average 37.2. If all nine live draws shared one mean, the best would sit about 1.5 SD above it: 2.3 points at SD 1.58, 5.8 at 3.93. *Source:* B s4, s5; e7_board.py; framing_stats_check.py, recomputed. *Tier:* derived; moderate. *Hedge:* A trend across different configurations, not a selection test or a correction. *Updated in v3:* V3-B5.
G4, G6	The 5.75-point step is 1.8 to 3.8 SD of a two-draw difference at the replica-implied draw SD of 1.1 to 2.2 (observed 1.58, replica scale 0.55), and 1.0 SD at 3.93. *Source:* B s5; e4_board.py, recomputed. *Tier:* derived; moderate. *Hedge:* The configurations differ; no lever is credited.
G7	Our best public score is 40.09 (submission (withheld), our rank withheld on 2026-10-13). It covers half of the 110 hidden games; it is the best of nine live draws and one draw of its configuration. It is verified by submission ID: outside the scored path the notebook plays 300 s per game on the 25 public games. *Source:* B s1, s8; e7_board.py; e6_arcagi3_rules.md, read. *Tier:* measured (score, rank). *Hedge:* A rank is meaningful only with its date; the private half is unknown. *Updated in v3:* V3-B1, V3-B2.
G8	Choosing two Final Submissions by public score is a mild form of holdout reuse. On the replica assumption, public and private scores of draws of one configuration correlate 0.00 (later runs) to 0.53 (six pairs). *Source:* M s3.3; e4_varcomp.py, read. *Tier:* derived; weak. *Hedge:* An inference; the shared component cannot be bounded with six pairs.
v2 H. Components, credit and release (Appendices A, B and F)	
N5, N6, H1, H6	Previous-reply cache reuse rises from 38% to 93% with the verbatim reasoning round trip; waste fixes, reply round trip and agent fixes together tie in levels (111.0 against 112.0, two runs each). A replica run costs about 2 h 10 min and $4 (GCP G4 spot). Inputs: Pennyroyal build 6.70 GB, harness bundle, D3 dataset 181 MB, Intel weights 181.24 GB, albucino drafter 4.16 GB. *Source:* E O5 to O7, s0, s2; gcp-bench-2026-10-06.md; CLAUDE.md, read. *Tier:* measured (read); owner-reported (cost, inputs). *Hedge:* One gate record each; the $4 is a 2026-10-06 spot price. *Updated in v3:* V3-X1, V3-X8, V3-X11.
H2, H3, H4	Plain credits without a cite: Son Pham's team (image-hash flag, memory-cap idea), lee-chang-93 (expert list), face-of-agi (verbatim re-send), Olympie and Ratsimbazafy (Pennyroyal patches), lovedheart (QSA fix). 14 streams and memory fraction 0.955 also appear in others' configurations. *Source:* audit_cites C6, C13, C16, C23; E O2, O3, O9 to O12, read. *Tier:* external; credit. *Hedge:* Plain-text credit, as checked by the citation audit; the implementation is ours.
H5	Component table: each row U1 to U7 and O1 to O19 states its origin, mechanism and numbers (for example REAP frees 7.31 GiB; 0 MemoryError in 17,508 calls; D-prime is a 252-line third-party file). *Source:* E s3; gate records, build_franzen_arms.py, e6_stack_numbers.json, read. *Tier:* measured (read); external (upstream). *Hedge:* Upstream numbers are theirs; most rows state a mechanism, not a level gain.
H7, H8	The Paper Track allows a Writeup of at most 1,500 words with a cover image and a public notebook, and publishes private attached resources after the deadline. Whether Qwen Community License weights are eligible is unanswered by the hosts (forum 745079, 745837); we make no claim. Our private D3 dataset inherits the refit's licence (our reading). *Source:* E s6.1, s6.2; e6_arcagi3_rules.md; e6_license_probe.py, read. *Tier:* external (rules, licence); owner-reported (our reading). *Hedge:* Rules as fetched on 2026-10-13; not legal advice.
H9	The 12 overlays hold 1,316 lines; ten (988) are ours alone (CC0 or MIT-0 intended), p448_serve (214) carries third-party expert ids, qsaring_serve (114) stays Apache-2.0; D-prime (252) and Franzen's patch are not ours to relicense. On 2026-10-13 the kernel and D3 dataset are private and the repository has no licence file. *Source:* E s6.4, s6.5, read. *Tier:* owner-reported. *Hedge:* Status as of 2026-10-13.
H10	AI coding assistants wrote or co-wrote most of the code, scripts and drafts: 165 of 166 commits carry a Claude co-author trailer (12 Opus 4.8, 76 Opus 5, 77 Opus 5.5). *Source:* E s7; git log, recomputed. *Tier:* owner-reported. *Hedge:* Counts commits that carry the trailer, not lines written by each author. *Updated in v3:* not recounted; the AI-use line in Appendix K replaces it.
V3 Board: draws, zeros and rank, 2026-10-13 to 2026-10-21	
V3-B1	Draws since 10-13 (submission id, UTC submit, config, public score, output bytes): (withheld) 10-13 02:06 r3p 40.09 (3,878); (withheld) 10-14 02:06 r3p 34.22 (3,882); (withheld) 10-15 02:06 hm32p 31.63 (3,881); (withheld) 10-16 02:06 hm32p 30.97 (3,838); (withheld) 10-17 02:06 hm32p 33.05 (3,886); (withheld) 10-18 06:56 hm32p 0.00 (3,411); (withheld) 10-19 02:06 hm32p 28.97 (3,868); (withheld) 10-20 02:06 r3p 40.98 (3,847). No errors in the period. *Source:* `data/board_draws.csv`; `kapi.sh raw competitions/submissions/list/...`, recomputed. *Tier:* measured; strong. *Hedge:* All eight listed, the zero included.
V3-B2	Best public score 40.98 (submission (withheld), config r3p, kernel version 1), our rank withheld in the 10-21 00:06Z snapshot (rank 10 43.94, rank 5 49.85, rank 3 52.82, rank 1 68.91). The previous best, 40.09, was our rank withheld in the 10-14 00:06Z snapshot. *Source:* `daily/board-2026-10-21.csv`, `board-2026-10-14.csv`; `data/rank_history.csv`, recomputed. *Tier:* measured; strong. *Hedge:* Public score on half of the 110 hidden games; a rank only with its date; best of 15 live fork-era draws, one draw of its configuration's third.
V3-B3	Our rank history is withheld in the anonymised copy.*Source:* `data/rank_history.csv`, recomputed. *Tier:* measured; strong. *Hedge:* Snapshot times are 00:06Z; the field's best-of-N rises with its draw count.

Table 4, continued

ID	Claim, source, tier and hedge
V3-B4	Same agent, two serving variants: r3p's 3 draws average 38.43 (SD 3.67); hm32p's 4 valid draws 31.15 (SD 1.70). The three r3p draws are the top three of the seven: exact one-sided permutation p = 1/35 = 0.029; Welch t 3.19. The r3p-first hypothesis was formed on 10-19 from 2 against 4 draws (p = 1/15 = 0.067); the one draw taken after it (40.98) sits above all four hm32p draws, which has probability 1/5 under no difference. *Source:* `data/board_draws.csv`; v3_board_trim.py, recomputed. *Tier:* measured (draws); derived (test); weak. *Hedge:* Not interleaved in time (r3p 10-13, 10-14, 10-20; hm32p 10-15 to 10-19), so a calendar or Kaggle-node effect is not excluded; the draft map is lossless by construction (the target verifies every token), so we do not attribute the gap to the map. Post hoc for 6 of 7 draws.
V3-B5	Yield-family draws (10-12 onward, configs that carry the yield change): 8 live, mean 34.28 (SD 4.25), against 7 pre-yield live draws, 29.25 (SD 1.58): exact one-sided p = 18/6,435 = 0.0028; Mann-Whitney U 50.5 of 56. Family A (r3p and play-identical variants, finals rule v4) valid draws average 34.27 (n 7). *Source:* v3_board_trim.py; `solution/kaggle/generated/finals-rule-predeclared-2026-10-12.json` (v4 note, 10-20 11:45Z), recomputed. *Tier:* measured; derived (test); moderate. *Hedge:* A trend across different, time-ordered configurations, each also carrying other changes; ranks the stack, attributes nothing.
V3-B6	40.98 is the best of 15 live draws since 10-04 and of 8 in the yield family; the expected maximum of 8 normal draws is 1.42 SD above their mean (+6.1 at SD 4.25, so about 40.3 for a family mean of 34.28); of 15, 1.74 SD. *Source:* v3_board_trim.py (simulated, 200,000 sets), recomputed. *Tier:* derived; moderate. *Hedge:* Normal model across configurations; 40.98 is consistent with a maximum, not evidence of a higher mean.
V3-B7	The 10-18 draw is complete, 0.00 and 3,411 bytes, like the 10-03 and 10-10 zeros (15 live fork-era draws are 3,810 to 3,893 bytes). The 02:06Z and 04:06Z fires never ran (the launchd log jumps from `2026-10-18T00:06:03Z daily_loop end` to `06:53:06Z daily_loop start`); the 06:56Z submission fell inside a field cluster of early endings (05:00 to 07:59Z: 18 of 101 runs short on 10-18, 15 of 93 on 10-17, 2 to 3 of 69 to 88 on 10-13 to 10-16). *Source:* `data/board_draws.csv` (bytes recomputed); `~/Library/Application Support/arc3/daily_loop.log` lines 5065 to 5124 (read); field counts from `docs/research/2026-10-19-top/field.md` s5 (read; its `shortruns.py` scratch copy is no longer on disk), check marked inline. *Tier:* measured (bytes, log); external (field counts as computed by the research pass); moderate. *Hedge:* Cause unknown and unknowable: a scored run returns no log. The 15-minute idle rule (topic 4) is a candidate, not a finding; an early outside kill fits the bytes as well.
V3-B8	Saturday midnight waves continue: 00:00 to 00:59Z, 73 of 118 runs ended early on 10-17 (68 of them 23 to 34 minutes in), against 8 to 18 of 140 to 148 on 10-13 to 10-16; the third Saturday in a row (10-03, 10-10, 10-17). Our 02:06Z fire drew 33.05 that day. The submit-hour default moved from 1 to 2 on 10-20 so the post-DST 01:06Z fire skips. *Source:* field.md s5 (read); PROGRESS 2026-10-20 "40.98 draw" entry (read); `solution/kaggle/daily_submit.sh` (owner-reported change, test_submission_guard 20 of 20), check marked inline. *Tier:* external (counts); owner-reported (change); moderate. *Hedge:* Field counts are the research pass's, from a third-party monitor; cause unknown.
V3-B9	Finals rule v4 (adopted 10-20 07:30Z under owner delegation, veto until 10-28): family A = r3p, hm32p, r3kp, r3fbp; valid draws exclude errors, 0.00 and field-wide failure days; with no eligible adapter, mark family A's two best valid draws (now 40.98 and 40.09). *Source:* `finals-rule-predeclared-2026-10-12.json` amendments (read), check marked inline. *Tier:* owner-reported; strong (record). *Hedge:* A decision rule, not a result; the private half is unknown.
V3 hm32: the own-traffic draft vocabulary gate	
V3-H1	Predeclared bars (control r3: 975.3 tok/s at 14 running, clean step 36.6 ms, accept 2.830 at 8+ running): throughput at least x1.008 (983.1), step at most 36.45 ms (identical stacks agree within 0.1 ms), accept not below 2.827 (identical stacks differ by 0.003), buckets 60 to 80K and 80 to 100K each at least C - 0.005, health clean, levels at least 105. Predicted +1.4 to 1.7% decode. *Source:* `gate-predeclared-2026-10-12-franzen-hm32.json` (read; declared before the push), check marked inline. *Tier:* owner-reported (record); strong. *Hedge:* Predeclared; one replica against one control.
V3-H2	Result: 987.1 tok/s at 14 running against 975.3 (+1.2%; r2, an identical-stack reference, 973.3, 0.2% from r3); clean step at 14 running 36.0 ms against 36.6 (at 12 and 13 running 33.5 and 35.2 against 34.3 and 35.5); accept at 8+ running 2.8385 against 2.8300 (+0.3%); 126 of 183 levels against 129, mean score 56.95 against 58.46; generated tokens 6.379M against 6.324M (+0.9%). Decode share 84.2% in both. *Source:* serve.log and benchmark.json in `generated/kernel-outputs/franzen-l1a1y8sv2r3hm32-v1` and `...r3-v1`, recomputed. *Tier:* measured; moderate (one run each). *Hedge:* One run per arm; +1.2% is 6 times the identical-stack spread on this counter but rests on single runs. The 3-level gap is inside replica noise (SD 3.9 in the family).
V3-H3	Gate items: every readable item passes; the 60 to 80K accept bucket sits exactly on its floor (2.833); the per-transcript `spec:` lines (report-only per-game accept) were not in the downloaded output and were not checked. Probe `l1a1y8sv2r3hm32p` v1: map install line (bitmap sha256 ad21d436929f), `--speculative-token-map ...arc-traffic-32768.pt` in the server argv, READY after 550 s, accept 2.721 at 8+ running on probe contexts, 0 tracebacks. *Source:* gate-results 10-14 (read); `kernel-outputs/franzen-l1a1y8sv2r3hm32p-v1/kernel.log` (install and READY lines read), check marked inline. *Tier:* measured; strong (install), moderate (counters). *Hedge:* The judgment calls are in the record; no per-game accept exists.
V3-H4	Measured gain is below the prediction: +1.2% decode against +1.4 to 1.7% predicted, from +0.3% accept and a 0.6 ms (1.6%) shorter step. *Source:* gate-predeclared and recomputed values above, recomputed. *Tier:* derived; moderate. *Hedge:* Single runs; the map's coverage gain (1.416% to about 0.05% missed) is a proxy, not the cause of the step change.
V3-H5	hm32 completes the yield family: 9 Kaggle replicas 128, 124, 119, 120, 128, 122, 129, 120, 126 (mean 124.0, SD 3.9), every one now adjudicated (hm32 PROMOTE). *Source:* `docs/research/2026-10-19-top/ledger.md` (yield family list, read); gate records (read); mean and SD recomputed (1,116/9 = 124.0; SD 3.91), check marked inline. *Tier:* measured; strong. *Hedge:* The v2 "+12.3 without one unadjudicated arm" row can lose the word "unadjudicated"; the 8- and 6-run sensitivity rows stay.
V3 LoRA r16 rejection-sampling fine-tune	
V3-L1	On 127 RE-ARC-3 games the pin cleared 548 of 601 levels (91.2%) and won 106 games (best per game over three attempts). The same pin clears 67.8% of the public 25's levels (124.0 of 183). *Source:* VM `~/lora_data/runs/a0{2,3,4}/benchmark.json`, recomputed. *Tier:* measured; strong. *Hedge:* One play per game, merged over Spot attempts; RE-ARC-3 is a generated imitation, not the hidden set.
V3-L2	13,652 windows (train 10,810, val 2,206, test 636, split by game family); 6,806 train windows (63.0%) carry a target; 99% are cut suffixes, mean 12,384 estimated tokens. Logged prompts at play time had medians of 21K to 53K tokens. *Source:* `wins.py` run on the VM over `~/lora_data/windows_15000.jsonl`, recomputed. *Tier:* measured; strong. *Hedge:* Estimated tokens, not tokenizer counts; the prompt medians are lora.md's.
V3-L3	Selection was weak: of 7,426 supervised train turns, 17.0% come from levels cleared at or under the human baseline's actions, 50.9% from levels at over 2x baseline and 20.4% over 4x; 69% come from won games. *Source:* `eff.py`, `wins.py` on the VM, recomputed. *Tier:* measured; strong. *Hedge:* Efficiency is actions over baseline per level; it is not the reward the board uses per se (that is its square, capped).
V3-L4	The training sample: 1,400 windows from 100 games (top 10 games hold 34%), 1,507 supervised replies, 15,996,575 tokens. *Source:* `wins.py`; `train_summary.json`; `adapter_full/log.jsonl`, recomputed. *Tier:* measured; strong. *Hedge:* About 21% of one epoch of the eligible windows.

Table 4, continued

ID	Claim, source, tier and hedge
V3-L5	Training ran 350 steps at 977 tok/s (train time), peak 78.4 GiB, 4.55 h of train time and 4.78 h wall: $7.89 at $1.65 per G4 Spot hour. *Source:* VM `~/lora_data/adapter_full/train_summary.json`, `~/lora_train_report.md`, read. *Tier:* measured (time); derived (cost); strong. *Hedge:* Cost is wall x list Spot price, not a bill.
V3-L6	Val NLL (32 fixed windows from the 15 validation families) fell from 0.7853 to 0.6966 at step 40 and 0.6759 at step 350: -13.94%, 81% of it by step 40, never rising. Train NLL per 20-step block stayed 0.62 to 0.70. *Source:* `train_summary.json` history; `lora_train_curve.csv`, recomputed. *Tier:* measured; strong. *Hedge:* Val NLL measures how well the adapter reproduces the pin's own sampled RE-ARC-3 replies, not decisions.
V3-L7	Two step-0 val passes of the same base on the same 32 windows (crashed first start and the final run) read 0.7830 and 0.7853 (0.29% apart), and step-1 losses 0.8599 and 0.8580 at identical token counts. *Source:* VM `~/lora_data/train_full.crashed1.log`, `train_full.log`, read. *Tier:* measured; moderate. *Hedge:* Two readings; the same forward nondeterminism that GRPO round A measured (topic 7). It is 2% of the 13.9% drop.
V3-L8	Merged-served against trained adapter, 30 requests and 1,854 tokens: mean abs logprob difference 0.0290, argmax agreement 97.8%, signed mean -0.0050 (gate: at most 0.07, at least 95%, at most 0.03). The gate was revised on 10-18 because two collects of one server differ by 0.0403 and agree on about 96.6% of greedy tokens. *Source:* VM `~/lora_data/compare_adapter.log`, `driver.log`; `solution/lora/T0_REPORT.md`, read. *Tier:* measured; strong. *Hedge:* Parity of the merge, not of play; the revised gate is a judgment call recorded before training.
V3-L9	At scale 2.0 the two replicas cleared 109 and 115 levels (mean game score 46.69 and 48.80; 8 and 8 games won; no zero-level game). The nine pin runs cleared 119 to 129, mean 124.0, SD 3.9 (3.7 with ddof 0). Both LoRA runs sit below every pin run: exact one-sided rank p = 1/C(11,2) = 0.018. The 2-run mean is 12 levels below: 3.1 run SDs; z = 3.9 at SD 3.9 and 1.9 at the six-pair SD 8.25. *Source:* `~/rep_results/rep{1,2}.json`, `~/runs/loraR{1,2}/working/benchmark.json` (levels_completed sums 109, 115); `yield_runs.json`, recomputed. *Tier:* measured; strong (counts), moderate (effect). *Hedge:* Not interleaved: LoRA on the GCP VM 10-19, controls on Kaggle 10-10 to 10-12. A same-card, no-adapter control of the base arm was added 2026-11-03 on Vertex AI (132 and 130; V3-W44), still not interleaved. At the paper's SD 8.25 the gap is under 2 SE.
V3-L10	At scale 1.0 one replica cleared 117 levels (mean score 49.72, 9 won), still below the pin minimum of 119. All three LoRA runs below all nine pin runs: 1/C(12,3) = 0.0045 if both scales are treated as one adapter class. 7 levels below the mean: 1.8 run SDs (z 0.8 at SD 8.25). *Source:* `~/rep_results/half1.json`; `loraH1/working/benchmark.json`; serve.log line 675 `lora: merged 300 tensors, scale 1.0` (R1: scale 2.0), recomputed. *Tier:* measured; moderate. *Hedge:* One run; the class pooling is our choice.
V3-L11	Decision rules: 130 levels in each full-strength replica to proceed; at half strength, 130 or more earns a second replica, 121 to 129 an entropy check, 120 or less stops the line. 109, 115 and 117 all fall in the stop region. *Source:* PROGRESS 10-19 ("predeclared gate of 130 each"); `docs/research/2026-10-19-top/PLAN.md` s2.7 (written while H1 ran, before its 00:08Z end); lora.md line 21, read. *Tier:* owner-reported. *Hedge:* No timestamped predeclaration file for the 130 gate was found in the repo; cite it as the project's stated rule.
V3-L12	Serving was not the cause: harness tok/s 916.8, 917.2 (scale 2.0) and 930.2 (scale 1.0) against 885.5 for r3 and 894.9 for hm32; generated tokens 6.64M, 6.66M, 6.73M against 5.54M to 6.49M for the pin runs; cache hit 94.8 to 95.4% against 94.6 to 95.3%. Accept length rose from 2.837 (hm32) to 3.036 and 3.032 (+7.0%) and 2.951 (+4.0%). Tokens per action fell to 845 and 840 (scale 2.0) against 886 to 1,021 for the pin, 913 at scale 1.0. *Source:* `rep*.json` and `half1.json` (franzen_readout, the same readout as the pin rows in `yield_runs.json`); `lora_replicas.csv`, recomputed. *Tier:* measured; strong. *Hedge:* More tokens and fewer levels; the accept rise is read as more predictable output [inference]. hm32's gate has passed (topic 2), so its accept may be quoted, labelled one run; r3 (2.832) is the alternative control.
V3-L13	At scale 2.0 play changed: actions 7,859 and 7,927 against 6,108 to 7,082 (pin mean 6,511, +21%); RESET actions 129 and 138 against 52 to 87 (mean 65, 2.0x); actions on never-cleared levels 2,965 and 1,980 against 777 to 1,592; GAME OVER echoes 26 and 27 against 8 to 13 (eight runs with transcripts); reasoning characters per action on cleared levels 1,729 and 1,772 against 2,021 to 2,346; NO-OP share 0.073 and 0.081 against 0.042 to 0.058; median cleared-level actions over baseline 0.706 against 0.606 to 0.689. Tracebacks per 100 replies (7.71, 7.00 against 6.95 to 8.72) and tokens per reply (2,031, 1,962 against 2,097 to 2,277) did not rise. *Source:* `playstats.py`, `split.py` run on the 8 local pin replicas and on the VM's 3 LoRA runs; RESETs and open-level actions from benchmark.json histories; `lora_behaviour.csv`, recomputed. *Tier:* measured; strong (counts). *Hedge:* Counters, not causes. hm32 (the arm's exact base) has 7,082 actions and 87 RESETs and only 5 of 25 transcripts locally, so its style counters are missing.
V3-L14	Half strength moved most counters partway back: actions 7,375, RESETs 100, never-cleared-level actions 1,770, reasoning per cleared action 1,979, NO-OP share 0.058, tokens per reply 2,152, accept 2.951. Deaths returned fully to the pin's range (10 GAME OVER echoes), yet levels (117) stayed below every pin run. *Source:* Same scripts on `~/runs/loraH1`, recomputed. *Tier:* measured; moderate. *Hedge:* One run. Deaths alone do not explain the half-strength loss; actions and RESETs stay elevated.
V3-L15	The loss is concentrated: per game the scale-2.0 mean is up on 6 games, down on 12 and tied on 7 (sign test two-sided p 0.24); five game-runs fall below the pin's nine-run minimum at scale 2.0 (ft09, g50t, vc33 in R1; cn04, ls20 in R2) and two at scale 1.0 (cn04, m0r0). Six games are constant across the pin family. *Source:* `lora_pergame.csv`, recomputed. *Tier:* measured; moderate. *Hedge:* Per-game tests are low power; the down games are mid-difficulty games.
V3-L16	Mean completion length on 20 RE-ARC-3 prompts (T 0.6, one sample each) went 652.4 to 843.8 tokens, but the median fell 194 to 147.5; one 5,663-token reply carries the rise (without that prompt: 638 to 590). *Source:* VM `~/lora_data/lenprobe_{base,adapter}.json`, recomputed. *Tier:* measured; weak. *Hedge:* RE-ARC-3 prompts, not the public 25; one sample per prompt. Do not write "longer outputs".
V3-L17	Greedy-token confidence on 30 T0 prompts is flat (mean logprob -0.150 base against -0.142 adapter). *Source:* lora.md s3 (`sharp.py`), read. *Tier:* measured (read); weak. *Hedge:* Small sample; argues against pure sampler sharpening.
V3-L18	Diagnosis: the filter kept almost everything (91% of levels cleared, 63% of windows supervised, half the targets from levels at over 2x baseline), and the adapter moved the pin's public-game style toward its RE-ARC-3 style (more actions per reply, less reasoning per action, more deaths and RESETs). *Source:* V3-L1 to L3, L13; RE-ARC-3 style column of lora.md s2 (966 to 1,347 reasoning chars per action, 4.28 to 4.52 actions per reply, 97 to 100 automatic RESETs), read. *Tier:* derived; moderate (facts), weak (cause). *Hedge:* The cause is an inference. Truncated contexts (V3-L2) and the T 1 loss on T 0.7 targets are untested alternatives; over-strength is ruled out as the whole story by V3-L10.
V3-L19	Replica wall times: 2.13, 2.14 and 2.13 h for the three completed runs; H1 lost two earlier attempts (from 18:37Z and 21:10Z, preempted). LoRA-line G4 time: datagen about 13.5 h wall (18:10Z 10-16 to 07:43Z 10-17, gaps included; estimated $17), training 4.78 h ($7.89), replicas at most 9.6 h (at most $16). *Source:* VM `~/runs/lora*/status`, preempted-dir timestamps, `~/datagen/queue.log`; `solution/lora/DATAGEN_STATUS.md`, recomputed. *Tier:* derived; moderate. *Hedge:* Wall time includes preemption gaps that were not billed; no ledger rows for 10-16 to 10-19 were read here.

Table 4, continued

ID	Claim, source, tier and hedge
V3 The 10-19 research verdict, the 15-minute idle rule and the keepalive	
V3-R1	Verdict: nothing public from any top-10 team explains or transfers its edge (no notebook, dataset, model, Hugging Face upload or ARC commit from about 50 member accounts since 10-05); the best public ARC-3 notebook is Franzen's 34.30. The top two (Tufa Labs 68.91, Yi-Chia Chen 67.49) also lead ARC-AGI-2 2026 (88.06 and 84.72, best public notebook 33.89), which fits a general model-level advantage plus large compute (post-training, mature serving) rather than an ARC-3 harness trick. *Source:* `docs/research/2026-10-19-top/leaders.md` Answer 1 to 3 (read; marked [verified] for the counts and [inference, medium] for the reading). Not rechecked: the raw API and web evidence in the session scratch is gone, check marked inline. *Tier:* external (board, posts); inference (the reading); weak to moderate. *Hedge:* "Fits", never "they post-trained". The leaders have published no method.
V3-R2	Compute behind the reading: Yi-Chia's public repo carries on-policy distillation (OPD v2) and DFlash drafter training code and names "the 64x B200 OPD run"; miles' public Flash-Next RL recipe (PR #2777, merged 09-25) needs 32 GPUs and 102 GB of host RAM for the PLE table. Neither fits one GPU by 11-02. *Source:* leaders.md Answer 3 (read). Not rechecked, check marked inline. *Tier:* external; moderate. *Hedge:* Public artifacts of the leaders, not their submitted systems.
V3-R4	The idle rule: Jeroen Cottaar (Tufa Labs), 2026-10-17 08:35Z, forum topic 750173: "If you don't do any move for 15 minutes, the run will end with the score so far", with "a thread that issues a RESET move to one game every 10 minutes until startup" as the fix. CPMP (NVARC3) replied 10-20 05:23Z that slow dataset loading can cause it. *Source:* Kaggle forum topic 750173, fetched anonymously 2026-10-21 by `scratchpad/v3ev/forum_topic.py` (read; text quoted verbatim), check marked inline. *Tier:* external; moderate. *Hedge:* A competitor's statement, not a host rule; the hosts have not confirmed it.
V3-R5	Our exposure: the pin waited up to `SERVER_STARTUP_TIMEOUT = 12` minutes for READY, then sent one warmup RESET (`ARC3_WARMUP_ACTION_GAMES = '1'`); the harness comment reads "Kaggle requires an action within 15 minutes of the run starting". A server not serving by about 27 minutes therefore banks the score so far, 0.00. *Source:* `generated/candidates-franzen-l1a1y8sv2r3hm32p/submission.ipynb` (grep, read), check marked inline. *Tier:* measured (code reading); strong. *Hedge:* Code reading; whether any of our zeros came from it is unknown (V3-B7).
V3-K1	Keepalive design: after the warmup RESET a daemon thread re-sends RESET on the warmup game only, every 9 minutes while /health is not 200, at most 5 times; it stops at READY, the first analyzer reply or the game's first action; it logs and never raises. The arm differs from r3p only in cell 7. *Source:* gate-results-2026-10-20-franzen-l1a1y8sv2r3kp.json `arm.change`; `scratchpad/keepalive-run1.txt` (builder check) (read), check marked inline. *Tier:* measured (code); strong. *Hedge:* Guards a slow boot only.
V3-K2	Offline test (`solution/sim/test_keepalive.py`, taaf's localhost competition arcade on the arc_agi 0.9.8 wheel, STALE_MINUTES=1, clock x15, the real ToolAgent; 8 checks, 4 min 07 s): without the keepalive the scorecard closed 15.05 minutes after the warmup RESET; with it RESETs at 9.05, 18.05, 27.05, 36.06 and 45.06 minutes (gaps 9.001 to 9.004), only on the warmup game, and the card lived to 60.12; READY at 38 minutes after the release (50 after notebook start): 4 RESETs, none after READY, every game played; READY before the release: none; an injected RESET failure is logged and play continues. Negative control (ARC3_HTTP_RETRIES=3, no watchdog): every game gives up ("analyzer failed 11 times consecutively"). *Source:* `scratchpad/keepalive-run1.txt` and `scratchpad/negative-control.txt` (test output, read); PROGRESS 2026-10-20 early-UTC entry (read; it gives 19.6 minutes for the control's last give-up, which the saved output's truncated log does not reach), check marked inline. *Tier:* measured (offline simulation); strong. *Hedge:* A scaled-clock simulation of the gateway's idle close, not the scored gateway.
V3-K3	Kaggle probe `arc3-franzen-l1a1y8sv2r3kp-a` v1 (2026-10-20 15:00Z): install lines present (GPU gate, keepalive, watchdog with 360 retries, D3 spliced, D-prime); warmup RESET at 732 s with the server loading; thread stopped at 966 s "after 0 RESET(s): the game is playing"; 25 games, 64 actions, 1 level (READY near 960 s left about 70 s of a 300 s per game probe); 0 tracebacks, 0 OOM. Promoted and pinned 10-20 15:23Z; first live draw due 10-21 15:06Z. *Source:* `kernel-outputs/franzen-l1a1y8sv2r3kp-v1/kernel.log` (keepalive lines at 732.06 s and 965.70 s read) and benchmark.json (64 actions, 1 level, recomputed); gate-results 10-20 r3kp (read); `kit/pinned/evidence.json` created 15:22:42Z (read), check marked inline. *Tier:* measured; moderate. *Hedge:* The live RESET path did not fire on Kaggle (READY came within 9 minutes); only the offline test exercises it. No score effect is claimed.
V3-K4	Expected value: about 0 to the private final; it converts a slow-boot day from a banked 0.00 into lost play time. Of our 3 fork-era zeros, 2 fell in midnight waves the 02:06Z fire now avoids. *Source:* PLAN.md s2 row 5 and field.md s7 (read), check marked inline. *Tier:* derived (inference); weak. *Hedge:* A guard, not a lever.
V3 trimtest: a paired decision test of leaner history	
V3-T1	Inputs: 117 post-trim states from 24 games in the r3 replica's request logs; slimming cuts their text 31% and images from 1,526 to 615 (the median state keeps 68% of its text). *Source:* PROGRESS 2026-10-12 "Trim test inputs ready" (read); 117 states recomputed from the replies, check marked inline. *Tier:* measured (count); owner-reported (cut sizes); strong. *Hedge:* One replica's states; post-trim first replies only.
V3-T2	Gate (any miss kills): coverage at least 90%; acts lean minus full at least -0.05 (lower bound at least -0.12); first-action agreement full-vs-lean minus full-vs-full at least -0.05 (lower bound at least -0.15); median reasoning ratio 0.80 to 1.25; a blind judge on 40 states. *Source:* gate-predeclared-2026-10-12-trimtest.json (read), check marked inline. *Tier:* owner-reported (record); strong. *Hedge:* Predeclared.
V3-T3	Result: coverage 117 of 117; acts full 0.372 [0.308, 0.436], lean 0.436 [0.368, 0.504], lean minus full +0.064 [+0.004, +0.124] (pass); first-action agreement full-full 0.778 [0.709, 0.838], full-lean 0.707 [0.652, 0.765], difference -0.071 [-0.132, -0.004] (FAIL: the point must be at least -0.05); reasoning ratio median 0.993 [0.851, 1.251] (pass). Blind judge not run. 90% state-bootstrap intervals. *Source:* `~/.cache/arc3/trimtest-2026-10-20/trimtest_replies.jsonl`; `solution/analysis/trim_compare.py`, recomputed. *Tier:* measured; moderate. *Hedge:* The fail is on the point estimate; the interval contains the -0.05 tolerance.
V3-T4	The difference is marginal as a test: paired per state (mean of the 4 cross pairs minus the full-full match), 24 states lower, 25 higher, 68 equal; a two-sided sign-flip permutation gives p = 0.093 (20,000 flips), consistent with the one-sided 95% bound just below 0. *Source:* v3_board_trim.py on the replies; `data/trimtest_by_state.csv`, recomputed. *Tier:* derived; moderate. *Hedge:* The gate was a tolerance rule, not a significance test; quote both.
V3-T5	"Agreement" mostly means agreeing not to act: in 63 of 117 states both full replies only investigate (no `action(` call), and two non-acting replies count as agreeing. Lean acts 6.4 points more often, which lowers agreement mechanically. Among replies that both act, agreement is 0.848 full-full (n 33) against 0.812 full-lean (n 133), post hoc. *Source:* v3_board_trim.py (63 recomputed from `first_action`); post-hoc figures from the gate record (read), check marked inline. *Tier:* measured; moderate. *Hedge:* The measured change is mainly acting sooner, not picking a different action; the gate counts both.
V3-T6	Consequence: leantrim stays killed for the freeze; no lt replicas; about 15 Kaggle GPU-h earmarked for four lt replicas and controls are freed. *Source:* gate-results trimtest `consequence` (read); PLAN.md row 6 (read), check marked inline. *Tier:* owner-reported; strong. *Hedge:* The test does not show leantrim would lose levels.

Table 4, continued

ID	Claim, source, tier and hedge
V3-T7	K0 sampling ceiling (new, from the same kernel's logprob tail): over 33,364 generated positions in 30 full-history replies, the mean post-temperature, top_p-cut maximum token probability is c = 0.912 (per reply 0.857 to 0.974, median 0.932). A one-hot chain drafter of 3 steps is capped near $1 + c + c^2 + c^3 = 3.500$ accepted tokens per step (4.190 at 4 steps), against 2.830 live: headroom +0.670 (the D4 plan asked for +0.40). Our measured first-position acceptance, 0.773, is 85% of c. *Source:* `trimtest_logprobs.jsonl`; `solution/analysis/k0_headroom.py` (recomputed; not reported anywhere before this dossier), check marked inline. *Tier:* measured (c); derived (ceiling); weak to moderate. *Hedge:* 30 replies, positions not independent, replies capped at 2,048 tokens; the ceiling assumes independent positions and that the logged logprobs are post-temperature (the overlay's docstring, not checked against the wheel). The ceiling idea is Bebop's (v2 E1).
V3-T8	Parser defect, verdict unchanged: the gate's `first_action` takes the first quoted identifier inside `action(`, which for the dict form `action([{'action': 'UP'}, ...])` is the key, so 77 of 189 acting replies parse as "ACTION" whatever the move (43 more are variables, "VAR"). Reading the dict value instead (61 of the 77 are MOUSE clicks) gives full-full 0.778, full-lean 0.697, difference -0.081 [-0.145, -0.015]: still a fail, slightly stronger. *Source:* `paper/v3/analysis/v3_trim_parser.py` on the replies (new, post hoc), recomputed. *Tier:* measured; moderate. *Hedge:* Post hoc, not part of the gate. The action name ignores click coordinates, so MOUSE against MOUSE counts as agreement.
V3 Vertex AI replication	
V3-X1	One Vertex replica of the shipped configuration matched its Kaggle run within 1% on every serving number: harness 886.9 vs 885.5 tok/s (+0.2%), decode mean 950.6 vs 950.7, accept 2.809 vs 2.830 (-0.7%), cache hit 95.08 vs 95.03%, clean step at 14 running 36.3 vs 36.6 ms, decode share 84.6 vs 84.2%, prefill 109.5 vs 108.6 ms per 1K new tokens, READY 563 vs 558 s. *Source:* `docs/research/2026-10-19-top/vertex.md`; run dirs `solution/kaggle/generated/kernel-outputs/vertex-cal1`, `.../franzen-l1a1y8sv2r3-v1`, recomputed. *Tier:* measured; strong (serving). *Hedge:* One run per platform. Weight load differs (308.8 vs 258.1 s) and is absorbed before READY.
V3-X2	The same pair cleared 126 and 129 levels (mean game score 58.44 and 58.46, no game at 0). *Source:* Same dirs, recomputed. *Tier:* measured; weak (levels, n = 1 each). *Hedge:* A 3-level gap is inside one run's spread (V3-X3); levels do not calibrate a platform.
V3-X3	Eight byte-identical replicas run concurrently on one day cleared 115, 117, 124, 125, 126, 128, 130, 132 levels: mean 124.6, SD 5.95 (7 df, 95% CI 3.9 to 12.1). On the 17 bench games they read 90 to 98, SD 2.32. Over the 35 four-against-four splits, the gap between halves reaches 8.75 levels on 25 games (mean 3.38; 29% of splits at 5 or more) and 3.0 on the bench (mean 1.36; 37% at 2.0 or more). *Source:* Readouts `scratchpad/rq/readout-rq1020-{1,2,3,4,7,8}.json`; `gs://arc3-rep-uc1/runs/rq1020-{5,6}/kernel.log` (130, 128), recomputed. *Tier:* measured; strong (counts), moderate (SD, 7 df). *Hedge:* Same notebook with request logging; rq5 and rq6 ran 2.5 h after the other six (after us-east4 preemptions), so "one day", not one start.
V3-X4	The new same-day SD (5.95) sits inside the six-pair CI (5.3 to 18.2, FRAMING B1) and below its point estimate 8.25; pooled with the six pairs (13 df) it is 7.1. The eight show no order effect: the two later runs scored 130 and 128, above the first six's mean. *Source:* V3-X3; FRAMING row B1, recomputed. *Tier:* derived; moderate. *Hedge:* Different stacks and conditions; pooling is a reading, not a test. Do not replace 8.25 in the paper without saying which set it is.
V3-X5	No platform offset is visible: 9 Vertex runs (cal1 + 8 rq) average 124.8 levels (SD 5.59) and 9 Kaggle y8-family runs 124.0 (SD 3.91), exact permutation p = 0.77; on the bench 94.56 vs 94.89, p = 0.86. The 16-run pool behind the gate has bench SD 2.61 (2.47 with rq5 and rq6). *Source:* `scratchpad/gate_k/pool.json` (copied to `paper/v3/analysis/gate_sim/gate_k/pool.json`) plus rq5, rq6, recomputed. *Tier:* measured; moderate. *Hedge:* The Kaggle nine are different configurations of one family, not identical bytes; this bounds an offset, it does not prove none.
V3-X6	Request logging does not visibly change play: rq mean 124.6 against cal1 126 and the Kaggle family 124.0. *Source:* V3-X3, V3-X5; PARALLEL.md "rq replicas read 115 to 132", recomputed. *Tier:* derived; weak. *Hedge:* No paired test of logging on and off.
V3-X7	Capacity: the Vertex training quota is 14 Spot RTX PRO 6000 per region (all 40 locations), outside the Compute Engine global cap of 1; two regions hold our inputs (us-central1, us-east4), so 28 concurrent jobs. The most we ran at once was 8 (07:48 to 09:21Z on 10-20). Other regions cap Vertex hyperdisk at 450 GB, below the 600 GB job disk. *Source:* `docs/research/2026-10-19-top/compute.md` s1 (Cloud Quotas API, read 10-19); `docs/research/2026-10-20-ft/ledger.csv`, read. *Tier:* measured (quota, read); measured (peak). *Hedge:* Quota is not capacity: G4 Spot stocked out in both regions on 10-20 (V3-X9). 28 at once has not been run.
V3-X8	Timing: submit to running 2.0 to 10.1 min in us-central1 (14.7 min for the first job), 9.5 to 41.7 min in us-east4 (one stockout retry); staging 173 to 186 s; READY 563 s; a replica takes 2.23 to 2.25 running hours and 2 h 17 min from submission to end. *Source:* `data/vertex_job_waits.csv` (from `gcloud ai custom-jobs list`, both regions, read-only, 10-21 00:58Z); vertex.md, recomputed. *Tier:* measured; strong. *Hedge:* Waits are one day's sample; cancelled-while-PENDING jobs show startTime equal to submit time.
V3-X9	Spot: us-central1 placed every job and preempted none (about 26 job-hours); us-east4 preempted two replicas in the same minute after 1.54 h each, and ended all 7 WC trainer attempts by SIGTERM before a first checkpoint (longest 69 min, step 7). us-central1 held WC PENDING for 21 min with 13 of 14 GPUs of quota free. *Source:* `ledger.csv` rows rq1020-5, -6, wc; `gs://arc3-rep-ue4/grpo2/runs/wc-ue4-preempted/progress.txt` (7 ATTEMPT and 7 SIGTERM lines), read. *Tier:* measured; moderate. *Hedge:* One day; Vertex documents up to six restarts per preempted job, each from scratch.
V3-X10	Two entrypoint traps, both fixed: Vertex omits the driver mount from PATH (no nvidia-smi), and staged inputs left in the container's page cache (inside a 168.5 GiB cgroup) slowed the target weight load to 702.9 s, so READY missed the notebook's 12-minute deadline; evicting them (posix_fadvise DONTNEED, no CAP_SYS_ADMIN) gave 308.8 s. A non-zero exit is restarted 3 to 4 times then FAILs, so the entrypoint exits 0 once DONE is written. *Source:* vertex.md "Gotchas"; `solution/gcp/image/entrypoint.sh` lines 23, 75 to 82, 138 to 151; serve.log of `scratchpad/vtx/probe2` and `vertex-cal1`, read. *Tier:* measured; strong. *Hedge:* Engineering detail for an appendix.
V3-X11	Cost per replica: $3.88 to $5.45 (2.23 to 2.25 h at $1.74 to $2.42 per job-hour). $1.74 = Spot G4 $1.663 (GPU 0.450, 48 cores 0.837, 180 GiB 0.376) + 600 GB hyperdisk $0.076; $2.42 adds the listed G4 training fee $0.675, which may not apply to Spot. *Source:* vertex.md "Cost" (Cloud Billing catalog, 10-20); ledger.csv, recomputed. *Tier:* derived; moderate. *Hedge:* List prices; Billing not reconciled (lags a day; fee on Spot unsettled).
V3-X12	Detectable shift (two-sided 5%, 80% power, 2.802 x SD x sqrt(2/n)): at the paper's SD 8.25, 2 vs 2 resolves 23.1 levels and 12 vs 12 resolves 9.4; at the same-day SD 5.95, 16.7 and 6.8; at the family SD 3.9, 10.9 and 4.5. On the 17-game bench at SD 2.61 a 12 vs 12 resolves 3.0 bench levels, about 5.6 on the 25-game scale at the gate model's bench ratio 0.53. *Source:* FRAMING B4 formula; `data/mde_by_runs.csv`, recomputed. *Tier:* derived; strong (arithmetic), weak (the 0.53 ratio). *Hedge:* Conditional on the SD printed beside each MDE; the bench ratio is a model of where an adapter's gain lands.
V3-X13	What one wave buys: a 12 vs 12 wave is 24 jobs, one 2 h 17 min round at $93 to $131, inside one region pair's quota. The paper's settling test for the yield (8 per arm, FRAMING YN6) is one concurrent 16-job wave at $62 to $87 and resolves 11.6 levels at SD 8.25. Seeing the curve's +4.2 levels for +11% tokens (FRAMING C4) at 80% power takes 13.5, 31 and 60 runs per arm at SD 3.9, 5.95 and 8.25. *Source:* V3-X8, V3-X11, V3-X12, recomputed. *Tier:* derived; moderate. *Hedge:* Not run. Concurrent arms remove the day and order confounds of the earlier sequential tests; they do not remove run noise.

Table 4, continued

ID	Claim, source, tier and hedge
V3-X14	The round-A gate design (one concurrent wave 1 for k members against shared controls, a fresh wave 2 for the best survivor, bar 2.0 at k = 3, wave-2-only Delta at least 0) promotes a null family with probability 0.038 (whole-run resampling, bench SD 2.6) or 0.111 (games drawn independently, bench SD 3.5), and a lone true +6 member 0.82 or 0.69, using 34 to 39 bench jobs. One adapter alone under the single rule: 0.054 and 0.110. *Source:* `docs/research/2026-10-20-ft/PARALLEL.md` gate table; `paper/v3/analysis/gate_sim/par/cmp2.py`, recomputed. *Tier:* derived (simulation); moderate. *Hedge:* The noise model resamples 16 recorded runs; the effect model (one level per non-saturated game) is an assumption.
V3-X15	The GRPO trainer also runs as a Vertex job: smoke `trsmoke2` matched WA's VM train.log step for step (tokens 329,179, 314,062, 311,225; peak 85.0 to 86.8 GiB; gnorm 0.048, 0.036, 0.013) and resumed from its checkpoint after a cancel; it runs about 14% slower than the VM (723 to 745 vs 855 to 867 tok/s). *Source:* PROGRESS.md 10-20 19:15Z; vertex.md "Training jobs", read. *Tier:* owner-reported; moderate. *Hedge:* Throughput from one smoke run.
V3 GRPO round A	
V3-G1	**WA data.** 900 train windows came from 264 plays (200 R1, 64 replica). All are mixed-clear: 658 replica windows from the 8 public train-half games and 242 R1 windows from 11 RE-ARC-3 games. The set spans 19 games and 32 game-levels, with a top game-level share of 0.05. Positive share 0.581, net/gross advantage -0.197, mean abs advantage 0.374, 17.85M est tokens. 21 R1 plays were cut by the 280K cap, and the 17 bench games' plays (8 each) were skipped. Groups: R1 had 227 groups, of which 158 were skipped (reward range under 0.1), 3 had too few plays and 66 were kept (54 death-only, 12 mixed). The replicas had 53 groups: 23 skipped for range, 7 with too few plays, 23 kept (20 mixed, 3 death-only), 12 plays censored. Hashes: portable windows sha256 40ffa0b3...; trainer file d3660e4f...; images 19d05592... over 2,663 images. *Source:* VM `~/arc3r2/wa/windows_review.json`, `phase_windows.log`; GATES.md data record; Vertex check in `gs://arc3-rep-uc1/grpo2/runs/wb/progress.txt` ("925 windows, 2663 images, 0 missing", sha256 as expected), read. *Tier:* measured; strong. *Hedge:* The windows are the pin's own play. 27% are RE-ARC-3, the family both r16 adapters drifted toward.
V3-G2	**WC data.** 882 windows (858 mixed, 24 death-only), all from the 64 replica plays of the 8 public train games. 23 game-levels, top share 0.073, positive share 0.663, net/gross -0.031, mean abs advantage 0.291, 17.55M est tokens. Its 25 val windows are byte-identical to WA's. Hashes: portable sha256 6cb9eda3...; source 94238582... (equal to PARALLEL.md's `nor1t18.jsonl`); 2,356 images. *Source:* VM `~/arc3r2/wc/prep.log`, `windows_review.json`; GCS `runs/wc/progress.txt` ("907 windows, 2356 images, 0 missing"), read. *Tier:* measured; strong. *Hedge:* 8 games and 23 game-levels, so memorization would show only as no bench gain.
V3-G3	**Recipe.** Reward per (play, level) is 1 if cleared within budget, minus 0.1 x min(deaths, 5). Advantage is (reward - group mean) x 2i/(n+1), with no supervision where abs A $<$ 0.05. Loss is the token mean over 16 windows of -sg(rho) A m log pi + 0.01 k3, where: rho = pi / pi_old; pi_old = the base, from a no-grad LoRA-off pass before each window; m = 0 outside [0.8, 1.28] or outside either keep set; log pi uses logits / 0.7 renormalized over SGLang's top-k 20 / top-p 0.95 keep set. Optimizer: AdamW at 3e-5, constant after 3 warmup steps, betas (0.9, 0.95), wd 0, clip 1.0; gnorm skip at 3x median. LoRA r16, alpha 16 (scale 1.0) on attention, GDN and the shared expert; routed experts and router frozen. One epoch is 57 steps for WA and WB, 56 for WC. Stops: val k3 $>$ 0.02 or entropy $>$ 10%. Null below 0.002. T-kill at 90 GiB or under 550 tok/s. *Source:* GATES.md "Round A (WA)"; `train.py` `ra_loss` (lines 120 to 136) and the loop (lines 300 to 390), code read; `args.txt` of runs wb and wc in GCS (the flags match GATES), read. *Tier:* owner-reported (recipe); measured (code reading); strong. *Hedge:* The band, beta and clip come from others' recipes (agent-factory-3, DAPO, GiGPO per DECISION.md); the thresholds are our judgment.
V3-G4	**The defect.** At step 0 the adapter is zero, so the reading is base vs base. WA's first attempt read raw val k3 12.909 with k1 -0.0026, on 66,949 val tokens in 24 windows. One token moving ln(12.909 x 66,949) = 13.67 nats explains it. The predeclared stop (k3 $>$ 0.02) would have fired at step 8 with nothing kept. Raw step-0 k3 in the other base-vs-base readings: 0.1357 (WA), 0.1238 (WB), 0.1267 (WC), 23.14 (smoke). *Source:* VM `wa/train.log.k3bug-1752`, `wa/train.log`; GCS `runs/{wb,wc,trsmoke2}/out/train.log`, read. *Tier:* measured; strong. *Hedge:* Raw k3 is unusable as a stop on this stack. It spans 0.12 to 23 for the same comparison.
V3-G5	**Pass-to-pass noise, recomputed.** Five independent base passes of the same 24 val windows were compared: three VM processes (17:52Z, 17:55Z, the final WA run) and two Vertex hosts (WB, WC). Across all 10 pairs: only 12.1% to 12.4% of tokens match exactly; 45.7% to 46.0% differ by more than 0.01 nats; 16.9% to 17.1% leave the [0.8, 1.28] ratio band; 2.9% to 3.1% move more than 1 nat; 336 to 384 tokens move more than 2 nats. Max abs 7.0 to 18.4 nats, RMS 0.365 to 0.373 nats. Clipped k3 is 0.0629 to 0.0658 in both directions (mean 0.0644, SD 0.0008, n = 20), while raw k3 is 0.099 to 1,525. VM-VM, VM-Vertex and Vertex-Vertex pairs cannot be told apart. The within-run floors (0.0643, 0.0641, 0.0644, 0.0650; SD 0.0004) sit inside the same range. *Source:* , recomputed. *Tier:* measured; strong. *Hedge:* Cause not identified. That routing or sparse-attention selection flips amplify over 20K-token windows is an inference we did not test. These are full-vocabulary log-probs at logits / 0.7 on the pin's own tokens.
V3-G6	**Training-side floor.** At step 1 the adapter is still zero, and the grad pass and the no-grad pass disagree. Masked shares at that step: WA's first attempt 16.56% (4.95% keep-set misses, 11.61% by the band); WA final 16.64% (4.95%, 11.68%); WB on Vertex 16.33% (11.34% by the band). Training k3 reads 0.027 to 0.029. Smoke steps 1 to 3 on Vertex match WA's tokens (329,179, 314,062, 311,225), gnorm and peaks exactly, with loss within 0.0025. *Source:* VM `wa/adapter.k3bug-1752/log.jsonl`, `wa/adapter/log.jsonl`; GCS `runs/wb/out/log.jsonl`, `runs/trsmoke2/out/train.log`, recomputed. *Tier:* measured; strong. *Hedge:* The "two identical passes" are one grad pass and one no-grad pass of the same weights, so this floor mixes nondeterminism with a code-path difference. Amendment 2's "11.6%" is the first attempt (11.61%); the restart reads 11.68%.
V3-G7	**The fix.** `--k3-clip 2` clamps each token's log ratio to [-2, 2] in the val k3, so a 13.7-nat token adds (e^2 - 3)/66,949 = 6.6e-5. Both stops and the null rule then read k3 minus step 0's clipped reading (the run's own floor F). This is amendment 2, which drops amendment 1's max(0.002, 2F) null. The loss's KL keeps raw d. Smoke check: clipped val k3 at steps 0 to 3 read 0.06497, 0.06556, 0.06556, 0.06545, a spread of 0.00059 against the 0.02 margin, across two Vertex hosts (a resume after step 1). Both amendments came after the first base-vs-base reading and before any trained-adapter reading. *Source:* GATES.md amendments 1, 2 and family; `train.py` `kl_to_base` and `net_k3`; GCS `trsmoke2/out/train.log` and `progress.txt` (two hosts), read. *Tier:* owner-reported (rule); measured (spread); strong. *Hedge:* The rule changed after seeing noise data, not signal. The floor's own SD (0.0008) is 40% of the 0.002 null margin, so the null rule is near its noise.
V3-G8	**WA curve.** Each reading gives net k3 / entropy / val NLL against step 0: step 8 +0.0001 / +1.6% / +0.75%; step 16 +0.0116 / +5.4% / +1.86%; step 24 +0.0260 / +11.1% / +3.92%. Step 24 broke both stops, and step 16 was selected (not null: 0.0116 $>$ 0.002). Run: 24 of 57 steps, 3.09 h, 878 tok/s, peak 87.05 GiB, 0 skipped steps, T-kills false. Selected sha256 fa80a6db... (equal to ckpt_16), uploaded 21:06Z; md5 57e8a481... identical in both buckets. *Source:* VM `wa/adapter/train_summary.json`, `train.log`, `uploaded.md5`, `sha256sum` of selected and ckpt_16; `gcloud storage ls -L` of both dataset copies, recomputed. *Tier:* measured; strong (readings), weak (meaning). *Hedge:* One seed. The val set is 24 windows of the pin's play on held-out RE-ARC-3 games, so it measures distance from the pin, not quality.

Table 4, continued

ID	Claim, source, tier and hedge
V3-G9	**WB curve.** Step 8 +0.0136 / +6.25% / +1.82%; step 16 +0.0658 / +25.0% / +8.24%, which broke both stops, so step 8 was selected. Run: 16 of 57 steps, 2.15 h wall (job 2.55 h with staging), 862 tok/s, peak 86.99 GiB, 0 skipped. Selected sha256 8f63a946..., published 22:28Z, md5 matches the dataset. *Source:* GCS `runs/wb/{progress.txt,DONE,PUBLISHED.sha256}`, `out/train_summary.json`; VM `sha256sum wb/selected`, recomputed. *Tier:* measured; strong (readings). *Hedge:* As V3-G8.
V3-G10	**WC so far.** Step 8 +0.0022 / -0.08% / -0.37% (val NLL fell). No stop. ckpt_8 synced at 00:49Z; 8 of 56 steps, 847 tok/s, peak 85.7 GiB. Preempted again at 01:04:49Z (SIGTERM, last checkpoint ckpt_8); a new attempt started 01:06:24Z on the same flags (re-read from `runs/wc/progress.txt` at 01:07Z). Earlier us-east4 attempts: 7, none checkpointed (val every 8 steps); the longest ran from 22:10Z to 23:15Z. *Source:* GCS `runs/wc/out/train.log`, `progress.txt`; `runs/wc-ue4-preempted/progress.txt` (7 ATTEMPT lines), read. *Tier:* measured; strong (readings). *Hedge:* Mid-run, one checkpoint. Not comparable to WA beyond step 8.
V3-G11	**Drift tracks lr x steps.** WA at step 16 and WB at step 8 both reach lr x steps = 4.8e-4, and they read alike: net k3 0.0116 vs 0.0136, entropy +5.4% vs +6.3%, val NLL +1.86% vs +1.82%. Entropy rises faster than linearly in lr x steps: +1.6%, +5.4% and +6.3%, +11.1% and +25.0% at 2.4, 4.8, 7.2 and 9.6 (x 1e-4). PARALLEL.md predicted that WB's selected net k3 would be 2 to 4x WA's; it is 1.18x, because WB stopped earlier. WC at 2.4e-4 moves less in entropy than WA at the same dose. *Source:* Derived from `data/grpo_val_curves.csv` (lr_x_steps column), check marked inline. *Tier:* derived; weak. *Hedge:* Two runs, one seed. WB at step 8 saw only the first 128 windows WA saw.
V3-G12	**Supervision mass is net negative.** net/gross advantage over in-band tokens is negative on 17 of 24 WA steps (median -0.42; 13 at -0.4 or below; range -0.85 to +0.55) and 11 of 16 WB steps (median -0.44). WC is 5 of 8 (median -0.06, mean +0.04). The window-level positive share is 0.581 for WA, so failed plays carry more tokens. Masked share rises as the adapter moves: WA averages 0.173, 0.184 and 0.212 over steps 1-8, 9-16 and 17-24; WB 0.176 and 0.226. Keep-set misses stay near 5.5%. gnorm median 0.041 (WA), 0.047 (WB), 0.024 (WC); max 0.075; no step skipped. WA and WB tokens per step are equal at all 16 shared steps. *Source:* , recomputed. *Tier:* measured; strong. *Hedge:* That negative-dominated updates raise entropy is an inference, not a test. val NLL rise equals k1 on the same tokens (WA at 24: dNLL 0.0272 vs k1 0.0283), so "entropy and val NLL rising together" is two readings, not three.
V3-G13	**Gate (a') state.** 400 states from 136 request logs of the 17 bench games. Base and H have 800 replies each (0 errors, 4,022 s and 4,027 s). The W server boot failed at 00:26Z ("p448: qwen4_exp.py does not match the Pennyroyal v2.5.3 source"), and the GPU was idle at 00:58Z (still idle, no new gate file, at 01:07Z). No verdict. **Interim, information only:** H (the full-strength r16 adapter at merge scale 2.0) vs base gives G (geometric-mean reasoning chars) 0.831, outside the base's 90% band [0.934, 1.073], and C (action calls per reply) 1.035, inside [0.870, 1.078]. So (a') is a live gate on G: a W is killed on G when its ratio is at or below 0.915. Round-1 metrics, base vs H: reasoning chars per action 8,598 vs 7,846 (0.912), tokens per reply 2,067 vs 1,892, action calls per reply 0.506 vs 0.524. *Source:* VM `wa/phase_gate_a2.log{,.draftview}`, `gate_a2/boot_W.log`, `run.log`, `nvidia-smi`, recomputed. *Tier:* measured; moderate. *Hedge:* Infrastructure failure, not a model result. The H-vs-base ratio is not the gate's output, because the gate restricts to states all three arms answer.
V3-G14	**Waves and probe.** None run. k is fixed at 07:00Z on 10-21. Simulated bar: P(promote) with every member null is 0.038 / 0.111 at k = 3; a lone true +6 member is promoted 0.82 / 0.69 (run model / stress model). *Source:* GATES.md family amendment; PARALLEL.md (`par/cmp.py`), recomputed. *Tier:* owner-reported; weak. *Hedge:* A simulation on a 16-run pin pool, bench SD 2.6.
V3-G15	**Odds stated before any result.** P(an adapter with a true +6 levels ships) is 0.05 to 0.12; the expected private-board gain is +0.1 (0 to +0.3); rank 3 needs +13 to +20 board. *Source:* DECISION.md "Decision" 8 and "Expected effect", read. *Tier:* owner-reported; weak. *Hedge:* Judgment, predeclared.
V3-G16	**Trainer cost so far.** Smoke runs: 1.07 job-h, $2.59. WB: 2.55 job-h, about $6.17 at $2.42/h; the ledger row is still blank. WC in us-east4: 3.30 h, $7.99. WC in us-central1 is running. WA trained on the VM from 17:34Z to 21:06Z, inside the VM row ($1.66/h). The forecast was $23 to $27 per member (PARALLEL.md). *Source:* `docs/research/2026-10-20-ft/ledger.csv`; WB hours recomputed from progress.txt (19:52:00Z to 22:25:05Z), check marked inline. *Tier:* owner-reported (rates); derived (WB). *Hedge:* Billing lags a day, and the Spot fee is unsettled. Topic 8 owns the totals.
V3 Costs and compute	
V3-C1	The round-A ledger holds 31 rows, 42.06 hours and $91.86 at its own rates ($2.42 per Vertex job-hour, $1.66 per VM-hour): replicas $56.45 (15 rows, 23.34 h, including two preempted attempts and cancels), setup and probes $3.25, trainer smoke runs $2.59, trainer members $7.99 (WC's us-east4 attempts only), VM $21.58 (13 h, 07:00 to 20:00Z 10-20). Every Vertex row's dollars equal hours x 2.42. *Source:* `docs/research/2026-10-20-ft/ledger.csv`, recomputed. *Tier:* owner-reported (ledger); strong (arithmetic). *Hedge:* A local ledger at list rates, not billed spend.
V3-C2	Filled to 01:00Z 10-21 (WB 2.55 h to 22:25:33Z, WC us-central1 1.47 h and running, VM 5 h more), the program's jobs and VM come to $87.46 to $109.95 (51.08 h), and $92.47 to $114.96 with the one-off items vertex.md lists (seed $3.85, builds $0.65, pulls $0.25, one day of storage $0.26). *Source:* `data/ledger_by_category.csv`; job API (`data/vertex_jobs_api_2026-10-21.csv`); vertex.md "Cost", recomputed. *Tier:* derived; moderate. *Hedge:* WC is still running; the low end assumes no Spot fee.
V3-C3	The VM ran 95.94 hours in the window, $159.54 at $1.663/h; it was off from 10-08 05:11Z to 10-16 17:45Z, so all of it falls 10-16 to 10-21 (LoRA data generation, the r16 fine-tune and its replicas, GRPO rollouts and WA's training). It was preempted 6 times and refused 45 start attempts for zone stockouts. *Source:* `gcloud compute operations list` for arc3-bench-1 (read-only, 99 operations); intervals in `data/vm_intervals_since_1013.csv`, by day in `data/compute_by_day.csv`, recomputed. *Tier:* measured; moderate. *Hedge:* Operation times, not billing; boot disk and snapshot storage are not costed.
V3-C4	Vertex used 33.0 job-hours (first job 10-20 01:41Z), $57.45 to $79.90. *Source:* ledger.csv plus filled rows; job API, recomputed. *Tier:* measured; moderate. *Hedge:* List rates.
V3-C5	Kaggle development GPU-hours since 10-13: 1.02 of the free 30 per week. The week to 10-17 already stood at 29.56 h at 10-13 00:06Z and did not move to the reset; after it the meter read 0.00 until 10-20 06:06Z, 0.81 at 15:06Z (the trimtest kernel) and 1.02 at 10-21 00:06Z (the keepalive probe). Daily scored submissions ran 10-17 to 10-20 with the meter at 0.00, so scored runs do not draw on this quota. *Source:* `data/kaggle_quota_readings.csv` (40 readings from daily_loop.log); `kapi.sh quota` live (1.02 h used, free 28.98 h, refresh 10-24 00:00Z), read. *Tier:* measured; strong. *Hedge:* Which kernel used which hours is inferred from timing.
V3-C6	All lanes, 10-13 00:00Z to 10-21 01:00Z: about 130 GPU-hours on the RTX PRO 6000 (95.9 VM, 33.0 Vertex, 1.0 Kaggle) and about $222 to $244 at list rates (VM $159.54, Vertex $57.45 to $79.90, one-off items about $5). *Source:* V3-C3 to V3-C5, recomputed. *Tier:* derived; moderate. *Hedge:* Excludes disks, snapshots, bucket storage beyond one day and the unsettled Spot fee; not reconciled with Billing.
V3-C7	Against the caps: the GRPO program (budget from 10-20 07:00Z) stops at $700, starts no round at $500 or more, and expected about $380 (DECISION.md s5); PARALLEL.md puts the peak near $650 after adding WB and WC (round A about $450 in all). At 01:00Z 10-21 the program ledger reads about $110 to $115. The September ceilings (USD 750 gross, 140 physical GPU-hours, USD 150 released) appear only in 09-12 to 09-16 notes and are not restated since; G4 spend had reached about $276 by 10-19 (compute.md), so they are not the operating limit now. *Source:* DECISION.md s5 lines 125 to 136; PARALLEL.md decision 6; `docs/research/2026-09-15-pre-gpu-readiness/README.md` line 92; compute.md line 109, read. *Tier:* owner-reported; moderate. *Hedge:* Which ceiling governs is the owner's statement; the $276 is not checked.

Table 4, continued

ID	Claim, source, tier and hedge
V3-C8	Cost per experiment class at list rates: a Vertex replica $3.88 to $5.45 (2.23 to 2.25 h); a 300 s probe $0.44 to $0.92 (0.25 to 0.38 h); a trainer member WB $4.45 to $6.18 (2.55 h, stopped by its guard at step 16); a full 57-step member about 9.5 to 11 h, $23 to $27 expected, $38.72 at the 16 h cap; three smoke jobs $2.59; the LoRA r16 RFT on the VM 4.8 h, about $8; a Kaggle replica commit about 2.2 to 2.4 free GPU-hours. Spot waste: $7.46 for the two preempted replica attempts, $7.99 for WC's checkpoint-free us-east4 attempts. *Source:* ledger.csv; job API; PROGRESS.md 10-19 and 10-20 20:20Z; CLAUDE.md (Kaggle commit), recomputed. *Tier:* derived (Vertex); owner-reported (others). *Hedge:* List rates; the expected member cost is the owner's estimate.
V3 W. Writer-derived rows	
V3-W2	The agent's seven valid board draws average 34.27, which is 0.60 of the nine-run development mean game score of 56.65 (yield-8,192 family); earlier configurations scored 0.53 to 0.63 of theirs (G4, G6). Corrected after review round 1: the 0.69 once quoted as the top of that range is this configuration's own 10-13 draw (40.09/58.46 = 0.686), not an earlier configuration. *Source:* V3-B5; D3 (56.65); G4; `paper/v3/review/numbers.md` row 17, recomputed. *Tier:* derived; moderate. *Hedge:* A ratio across different configurations, not a forecast; the hidden games may be harder.
V3-W3	The 25 demonstration games have 6 to 10 levels each, 183 in all. *Source:* `paper/v2/analysis/out/yield_runs.json` field per_game_nlevels, recomputed. *Tier:* measured; strong. *Hedge:* The hidden games' level counts are not known to us.
V3-W4	Franzen's harness patch shows the board as an enlarged image with the changed cells marked (10x images with diffs), adds animation frames and UNDO, keeps reasoning in the history and serves a 128K-token context. *Source:* Component row U6 (H5); `CLAUDE.md` "What we ship", read. *Tier:* owner-reported; moderate. *Hedge:* Description of inherited code, not a measurement.
V3-W5	The leaner history rule would save about 3.5% of tokens if behaviour held, worth about 1.3 levels at the curve's elasticity (0.31 x 0.035 x 124 = 1.35). *Source:* dossier section 5 "What was done" and "What it shows" (`docs/research/2026-10-12-next/PLAN.md` 5.1), read. *Tier:* derived (inference); weak. *Hedge:* The 3.5% is a planning estimate, not a measurement.
V3-W6	The paired decision test ran as one Kaggle kernel: 468 replies, 0 errors, 2,597 s of kernel time (0.72 GPU-hours), server READY after 448 s. *Source:* dossier section 5 "What was done"; gate record `gate-results-2026-10-20-trimtest.json`, read. *Tier:* measured; strong. *Hedge:* One kernel; cost excludes the replica whose states it replays.
V3-W7	The clipped val k3 floor has an SD of 0.0008 (unrounded 0.00083) across the 20 directed pass comparisons, about 24 times smaller than the 0.02 stop margin; its whole range (0.0629 to 0.0658, width 0.0029) is about seven times smaller. *Source:* V3-G5, V3-G3, recomputed. *Tier:* derived; strong. *Hedge:* The 20 readings come from five passes compared in ten pairs both ways, so they are not independent. The floor's SD is 40% of the 0.002 null margin, so the null rule sits near the noise.
V3-W8	At the 2.3% arithmetic bound the freed prefill would be worth at most 1.4 levels (0.0226 x 5.29M tokens x 11.2 levels per million = 1.34); at the timers' 0.6% to 0.8% it is 0.4 to 0.5. *Source:* D7, recomputed. *Tier:* derived; moderate. *Hedge:* An upper bound at a generous 11.2 levels per million tokens.
V3-W9	Eighteen development runs of the yield-8,192 family without an adapter (nine on Kaggle, the nine compared in V3-L9; nine on Vertex, cal1 and rq1020-1 to 8) cleared 115 to 132 levels, mean 124.4, SD 4.7. Against them the three adapter runs (109, 115, 117) give a one-sided exact rank-test p of 0.0038 (5 of 1,330 splits as extreme; ties counted half): 109 is below all 18, and 115 and 117 tie the two lowest. *Source:* `paper/v3/evidence/data/bench_pool18_levels.json` (field all25) and `lora_replicas.csv`, recomputed. *Tier:* derived; moderate. *Hedge:* The control set mixes configurations of one family and two platforms; the adapter ran on a third (GCP VM) on other days. A same-card, no-adapter control of the adapter's base arm was added on 2026-11-03 (V3-W44: 132 and 130, above all three adapter runs, rank p 0.10 among the five); it was on Vertex AI, not the VM, and later.
V3-W10	Resume openers at the 8,192 yield are 151 to 215 per run in the eight runs with transcripts (mean 188, median 194), so "about 190"; at 2,048, 565 to 634 (mean 590). *Source:* D5; `paper/v2/analysis/out/yield_mech.json`, read. *Tier:* measured; strong. *Hedge:* Counts per run, not per game.
V3-W11	Counting the 2026-10-18 zero (cause unknown), the agent's eight draws average 29.99 (7 x 34.27 / 8). *Source:* V3-B5, V3-B7; `board_draws.csv`, recomputed. *Tier:* derived; strong (arithmetic). *Hedge:* Finals rule v4 treats the zero as invalid; this row only shows how much the exclusion matters.
V3-W12	Runs per arm to see a level shift at 80% power, two-sided 5%, normal approximation, 2 x $((z.975 + z.80) \times SD / d)^2$: for the curve's +11% token price (d 4.19) 31.6 at SD 5.95 and 60.8 at SD 8.25 (so about 32 and 61); for the 7.4 levels left after the yield runs' token share, 10.2 and 19.5 (so about 11 and 20). *Source:* C4 (4.19 from `e5_detect.json`), V3-X3 (5.95), B1 (8.25), YN6 (7.4), recomputed. *Tier:* derived; strong (arithmetic). *Hedge:* Conditional on the SD; every detection number in v3 is two-sided at an identical-run SD.
V3-W13	In the last 720 s of the 7,200 s window (its last tenth), 34 pin-family development runs on the 25 demonstration games cleared 5.5 of 123.8 levels per run (4.4%) and earned 4.39 of 59.30 points (7.4%) with 632K of 6,236K generated tokens (10.1%). Read as a marginal, that is an elasticity of 0.44 on levels (0.0444 / 0.1013) and 0.73 on points (0.0740 / 0.1013). Clears per 720 s bin fall from 17.4 to 5.5 while points per bin stay 4.39 to 7.52, so points per clear rise from 0.28 in the first bin to 0.80 in the last: late clears are deep levels. *Source:* `paper/v3/analysis/clock_marginal.py` section 4; output `paper/v3/analysis/out/clock_marginal_2026-10-26.txt`; `docs/research/2026-10-23-push/clock.md`. Reproduced 2026-10-28 (identical output); elasticities and points per clear recomputed, check marked inline. *Tier:* derived; moderate. *Hedge:* A marginal only if the late rate would hold on a stretched clock; the clear rate was still falling (8.8, 8.4, 6.1, 5.5 over the last four bins). Points weight each cleared level by its depth at a per-level score of 1, so they sum to 59.30, not the 56.44 mean game score. Pin-family runs, not the eight that built the curve.
V3-W14	On levels the clock's elasticity (0.44, V3-W13) sits inside the curve's range across four-run subsets (0.18 to 0.49, C3) and next to the 0.45 fitted on twelve past serving steps (N1.3). On score it is 3.3 times the curve's 0.22 (C3). At +11% tokens the clock gives 0.11 x 0.73 x 59.30 = 4.8 points (about 5 of 59) and 6.0 levels, against about 1.4 points and 4.2 levels from the curve (C4). One 25-game run has an SD of 5.0 points (C7), so a difference of two runs per arm has an SD of about 5.0 and cannot resolve 4.8. *Source:* V3-W13; C3; C4; C7; N1.3, recomputed. *Tier:* derived; moderate. *Hedge:* The curve's elasticities are for the 110-game hidden window and the clock's for the 25-game development window at about the same slot-seconds per game; the two use different runs. Verified only that the curve's marginal levels weigh less than its average (score elasticity 0.22 under levels 0.31) and the clock's weigh more (0.73 over 0.44); why the curve's are shallower (its stuck-level rule, or its 110-game world) is untested.
V3-W15	Read at 70% of the window (5,040 s of 7,200 s; 4,345K of 6,236K generated tokens, 69.7%), the 34 runs of V3-W13 had cleared 103.8 of 123.8 levels per run (20.0 fewer, -16.2%) and earned 44.92 of 59.30 points (-24.2%). The curve's full pool at rho 0.7 against 1.0 gives 437.2 against 522.6 levels (-16.3%, about 20 of 124) and 49.59 against 56.00 points (-11.4%). On levels the two agree; on score the timeline loses about twice the curve's share. *Source:* `paper/v3/analysis/out/clock_marginal_2026-10-26.txt` section 4 (bins 1 to 7 of 10); `paper/v3/analysis/figs/data/budget_band.json` (full_pool_levels, full_pool_S at rhos 0.7 and 1.0), recomputed. *Tier:* derived; moderate. *Hedge:* A truncated timeline is not a run given 70% of the tokens: the scheduler ran with the whole window ahead of it. The curve is for the 110-game hidden window, the timeline for the 25-game development window; different runs. The token-only cut itself was not run.

Table 4, continued

ID	Claim, source, tier and hedge
V3-W16	Board refresh to 2026-10-28. Since the fork, 26 completed draws (23 live, 3 dead); the best live draw is still 40.98. The agent's family (finals rule v4: r3p, hm32p and the boot-keepalive variants r3kp and kp12, which act only before the server is ready) has 12 valid draws, mean 34.59, SD 3.59, range 28.97 to 40.98; with the 10-18 zero, 31.93 over 13. Ratio to the 56.65 development mean: 0.611. The five keepalive draws (35.05, 36.03, 37.56, 33.09, 33.45) average 35.04, between r3p's 38.43 (3) and hm32p's 31.15 (4); the adapter's three (35.90, 39.79, 35.98) average 37.22; the yield family's 16 live draws (10-12 on, adapter included) average 35.07. Resampled expected best of 12: 40.33 (normal 40.43); of 23: 40.82 (normal 41.51). Daily snapshot 2026-10-28 (04:34Z): our rank withheld; rank 1 75.56, rank 3 56.02, rank 5 51.04, rank 10 45.41, rank 20 38.20. The family's record closes on 10-28: from 10-29 the pin is kp12 plus two harness overlays, a different agent. *Source:* `paper/v3/evidence/data/board_draws.csv` (Kaggle submissions API, 8 rows added 2026-10-28 13:55Z); `solution/kaggle/generated/daily/board-2026-10-22.csv` to `board-2026-10-28.csv`; `fig_noise_draws.py` and `fig_selection.py` output (asserted), recomputed. *Tier:* measured (draws, ranks); derived (means, expected maxima); strong. *Hedge:* Public scores on half of the hidden games; the family pools play-identical configurations drawn on different days; snapshots after 10-21 were taken at the daily job's run, not at 00:06Z.
V3-W17	Round B adapter WE (GRPO on levels the agent clears 20% to 80% of the time, served by stochastic-rounding merge) was pinned for the daily draw on 2026-10-23 by the owner's override before its predeclared waves read, and drew 35.90, 39.79 and 35.98 (10-24 to 10-26). Stage 1 (6 valid runs against 34 pooled controls): Delta -1.37 levels on the trained half of the demonstration games (+0.92 on the bench half); Round B rule: KILL. Round C (WE3, WE served at three times its scale): stage 1 trained half +1.59, bench -1.41; stage 2 (7 valid runs against 36 pooled controls) bench -0.79 against a +1.5 bar; fails, and reinforcement learning ends for the competition. Under finals rule v4 an unpromoted adapter's draws are not marked. *Source:* `solution/kaggle/generated/gate-results-2026-10-23-franzen-l1a1y8sv2r3wesrp.json` (override record); `solution/lora/grpo/GATES.md` Rounds B and C; `docs/PROGRESS.md` 2026-10-26 16:30Z and 20:00Z; `G/wave-we1-2026-10-26-pooled.txt`, `G/wave-we3-2026-10-26-{pooled,concurrent}.txt`, read. *Tier:* measured; moderate. *Hedge:* Deltas are levels per run against pooled controls at 6 to 7 runs; the adapter's board draws carry no level evidence (one agent's draws span 12 points).
V3-W18	Diagnosis of stuck plays (2026-10-30): eight paired plays of the same level, cleared in one run and stuck in another, read from four Kaggle development runs of the r3 agent (562 level plays, 63 never cleared). 27% of never-cleared levels hold 3 or more idle turns in a row (a turn that ends at a yield or the length cap with no action executed), against 8% of cleared levels; 40% of their clock goes to such turns, against 29%. Of the turns that are the 3rd, 5th or 7th idle turn in a row, 54% then act (42 of 77). Stuck plays also wrote off an object type or an action untested (tn36 L2, sp80 L2) and misread what the harness shows (bp35 L3 X marks, bp35 L2 scroll). *Source:* `docs/research/2026-10-30-unstick/FINDINGS.md` (tags [verified] and [inference] there); `solution/analysis/unstick_counters.py`; `docs/PROGRESS.md` 2026-10-30 00:40Z, read. *Tier:* measured by read; moderate. *Hedge:* Four runs and four reading agents; the case table is eight plays; the 54% is the reference for V3-W20.
V3-W19	The harness program's design and control. Each change has a predeclared gate (`gate-predeclared-2026-10-30-franzen-l1a1y8sv2r3lf15uns2.json`, `...-2026-10-31-...us8.json`, `...-usc.json`; written 2026-10-30 03:40Z, 05:39Z and 2026-10-31 12:05Z, before the first run of each build): mechanism counter, model response, actions (at most 1.10 times the control, median ratio at most plus 0.05) and no harm (mean levels at least control minus 2.5, mean score at least minus 1.5); kill on any miss; no runs added after reading. Control: seven runs of the 15-stream build (129, 120, 125, 122, 135, 123, 110 levels; mean 123.4, SD 7.76; mean score 54.85, SD 3.17; 7,105 actions; median actions over baseline 0.674), five on Vertex AI and two on Kaggle; all arm runs on Kaggle. Three runs against seven at SD 8.25 resolve 2.80 x 8.25 x sqrt(1/3 + 1/7) = 15.9 levels; a 95% interval on a lead is the lead plus or minus 11.2 (13.0 for two runs). *Source:* `paper/v3/analysis/harness_read.py` (output `paper/v3/analysis/figs/data/harness_runs.csv`); the gate files above, recomputed. *Tier:* measured (runs), derived (intervals); strong. *Hedge:* Arms and control differ in platform and day; arms are nested, so results are not independent.
V3-W20	Stall check, coverage and step checking (build `unstick2`): three Kaggle runs, 141, 126 and 118 levels (mean 128.3, +4.9 on the control), mean score 58.48 (+3.63), 6,716 actions, median ratio 0.620; 41 of 47 third, fifth and seventh idle turns were followed by an action (0.87; reference 0.54); streaks of 5 or more idle turns 0.33 per run (reference 2.75); stall lines 20, 17, 10, untested lists 46, 45, 38, new-type lists 107, 96, 88; no unused-action line named an engine action. Every predeclared item passed; pinned 2026-10-31 11:43Z and replaced at 14:33Z by the build with corner-joined regions. The lead is inside the interval of V3-W19 (-6.3 to +16.1). *Source:* `paper/v3/analysis/harness_read.py`; `solution/kaggle/generated/gate-results-2026-10-30-franzen-l1a1y8sv2r3lf15uns2.json`; `docs/PROGRESS.md` 2026-10-31 11:45Z, recomputed. *Tier:* measured; strong for the mechanism, none for the level lead. *Hedge:* Three runs cannot resolve 4.9 levels; no level gain claimed.
V3-W21	The first stall build (`unstick`) was killed by its own m1 item on 2026-10-30: the untested-list counter read 0 against a bar of 5 because engine action names (ACTION1 to ACTION7) never matched the history's names; its two runs cleared 131 and 127 levels (mean 129.0, score 58.57), 27 of 37 stalled turns acted (0.73), 1.50 streaks per run. *Source:* `gate-results-2026-10-30-franzen-l1a1y8sv2r3lf15unst.json`; `harness_read.py`; `docs/PROGRESS.md` 2026-10-30 03:05Z, recomputed. *Tier:* measured; strong. *Hedge:* Two runs; the killed arm's levels equal the fixed build's, which shows levels from so few runs do not separate a half-built mechanism from a built one.
V3-W22	Corner-joined regions (build `seg8`, in `us8` on top of the stall check): each segmentation node gains `group`, the lowest id among same-colour nodes joined by corner contacts. Three Kaggle runs, 132, 128 and 127 levels (mean 129.0, +5.6), mean score 60.66 (+5.81), 6,431 actions, median ratio 0.657; `group` read in 39, 64 and 33 code blocks (0 of 5,880 in the two earlier runs); 33 of 39 idle turns act (0.85). Every item passed; pinned 2026-10-31 14:33Z. bp35 L3 X marks: 15 one-pixel nodes edge-joined, 3 objects of 3 by 3 as groups. 76 of 184 human level starts in 13 of 22 replayed games hold a group of 2 or more nodes with a bounding box of at least 3 by 3 (73 with sparse dot lattices dropped). The gate record's 71 in 13 of 21 games (unstated dot rule) is not reproduced and not used. *Source:* `harness_read.py`; `gate-results-2026-10-31-franzen-l1a1y8sv2r3lf15us8.json`; `paper/v3/analysis/figs/fig_glyph.py` and `data/glyph_corpus.json` (real patched module); `solution/sim/test_franzen_seg8.py`, recomputed. *Tier:* measured; strong for the mechanism, none for the level lead. *Hedge:* Three runs cannot resolve 5.6 levels; the 76 describes boards, not play.
V3-W23	Scroll detector (build `scroll`, in `usc` on top of us8) and the clock read. On the 22 human replays the detector fires on 103 of 363 changed same-level pairs in bp35 and 40 of 772 in lf52, and on 0 of 5,178 in the other 20 games (6,170 silent of 6,313). In play, 21, 29 and 21 scroll openers per run in bp35 and lf52 (mean 23.7), none elsewhere. (Gate verdict and promotion: V3-W27.) Clock of the six runs of the first two builds (150 game plays): 76% of clock on levels cleared (13.4 minutes per cleared level, 772 cleared levels), 24% on the level open at the end (13% to 33% by run); turns with no executed action hold 27% of the clock (27.5% in the stall-check runs; 20% the first idle turn of a streak). *Source:* `harness_read.py`; `paper/v3/analysis/figs/fig_scroll.py`, `fig_clock.py`, `clock_tail.py`, `data/scroll_corpus.json`, `data/clock_by_game_run.csv`; `solution/sim/test_franzen_scroll.py`; `gate-predeclared-2026-10-31-franzen-l1a1y8sv2r3lf15usc.json`, recomputed. *Tier:* measured; strong (mechanism, clock), moderate (levels at n = 3). *Hedge:* Wall-clock stamps include queueing; the last turn of a game is dropped; the scroll arm's final result is in V3-W27.

Table 4, continued

ID	Claim, source, tier and hedge
V3-W24	Board refresh to 2026-11-01 18:35Z. 80 scored submissions in all; 30 completed draws since the fork (27 live, 3 dead); best live draw still 40.98. Draws since 10-28: 38.17 (10-29), 29.63 (10-30), 35.25 (10-31), all of the 15-stream build kp12lf15 (mean 34.35), and 33.56 (11-01, submission (withheld)), the first draw of the harness build (kp12lf15us8: stall check plus corner-joined regions); one draw says nothing about that build's mean. Valid draws of the family (12, mean 34.59) plus these four: 16, mean 34.48, SD 3.47, range 28.97 to 40.98; with the 10-18 zero 17 draws, mean 32.45. Live draws before the yield change 7 (mean 29.25), after it 20 (mean 34.89, adapter draws included). Ratio of the 16 to the 56.65 development mean: 0.609. Live read 2026-11-01 about 18:35Z: our rank withheld; rank 1 75.56, rank 3 58.59, rank 5 56.02, rank 10 46.82, rank 20 40.75; last daily snapshot 16:06Z: our rank withheld. Daily-job snapshot of 11-01: our rank withheld. Earlier snapshots: our rank withheld (10-29), our rank withheld (10-30), our rank withheld (10-31). The family's twelve-draw record (Figure 12) closed on 10-28 and is not redrawn. *Source:* `solution/kaggle/kapi.sh subs` and `leaderboard 40` (read 2026-11-01); `paper/v3/analysis/figs/data/board_draws_all_2026-11-01.csv`, `board_snapshots_2026-11-01.csv`, `kapi_subs_2026-11-01.txt`; `solution/kaggle/generated/daily/board-2026-10-29.csv` to `-11-01.csv`, recomputed. *Tier:* measured; strong. *Hedge:* Public scores on half of the hidden games; the final private score is not known until after 2026-11-02; refresh after the last draw.
V3-W25	As of 2026-10-31 no Final Submission was marked by hand (counter 0 of 2, after two manual marks were removed at the owner's request; marked again 2026-11-02, see V3-W35), and Kaggle's automatic pick of the two highest public draws would apply: the 2026-10-20 draw (40.98) and the 2026-10-13 draw (40.09). Both predate every harness change. *Source:* `docs/PROGRESS.md` 2026-10-31 18:05Z (hand check in the UI), read. *Tier:* owner-reported; moderate. *Hedge:* Not re-read by the paper's scripts; re-check after the 11-01 and 11-02 draws.
V3-W26	Earlier arms on the 15-stream build, appendix only: kept-variables overlay v1 and v2 killed on the NameError ratio (0.51 and 0.52 against 0.50; pooled levels 127.8 against 125.7 and 125.3 against 123.4); ThinkingCap against the build beside it on Kaggle: 122 against 129 levels, score 49.19 against 57.79, 11,523 against 6,780 actions, killed. The 15-stream build's own gate: 125.5 levels against 124.0, pinned 2026-10-28. *Source:* `docs/PROGRESS.md` 2026-10-28 18:33Z and 22:57Z, 2026-10-29 08:40Z and 20:56Z; `gate-results-2026-10-29-franzen-l1a1y8sv2r3lf15kv*.json`, read. *Tier:* measured; moderate. *Hedge:* Single waves; no level effect claimed.
V3-W27	Scroll arm, final (read 2026-10-31 19:00Z): three Kaggle runs, 111, 122 and 130 levels (mean 121.0, -2.4 on the control, -8.0 on the build it extends, whose mean is 129.0), mean game score 54.10 (-0.75; us8 60.66), 6,813 actions, median ratio 0.654; 42 of 58 idle turns act (0.72); `group` read in 49, 41 and 49 blocks; scroll openers 21, 29, 21 (mean 23.7) in bp35 and lf52, 0 elsewhere; bp35 at 1 level in all three runs, lf52 at 2, 3 and 4. Gate verdict PASS on every item (no-harm levels 121.0 against a bar of 120.9, score 54.10 against 53.35). Not promoted (record `promote: false`): its mean is below the control and the build it would replace and it showed no gain in the games it targets; the gate text makes a pass necessary, not sufficient. The tool2 result is V3-W28. *Source:* `paper/v3/analysis/harness_read.py`; `solution/kaggle/generated/gate-results-2026-10-31-franzen-l1a1y8sv2r3lf15usc.json`; `paper/v3/analysis/figs/fig_gates.py`, recomputed. *Tier:* measured; strong (mechanism, verdict), none for any level effect. *Hedge:* Three runs cannot resolve 8 levels, so this does not show that scroll hurts, only that it showed no benefit.
V3-W28	Tool-fix arm `tool2` on the corner-joined build (`ust` = us8 + tool2; read 2026-11-01 20:49Z): three Kaggle runs, 123, 129 and 115 levels (mean 122.3, -1.1 on the control, -6.7 on the build it extends), mean game score 54.63 (-0.22), 7,099 actions, median ratio 0.660; 54 of 68 idle turns act (0.79); `group` read in 46, 53 and 47 blocks; `inspect.getsource` failures ('could not get source code') 0, 0, 0 per run against 7.8 per run in the six uns2 and us8 runs (4, 3, 14, 15, 2, 9); SnippetTimeout results 6, 5, 6; 'Tool timed out' 0, 0, 0 against 6.0 (3, 8, 9, 7, 3, 6); streaks of five or more idle turns 3.00 per run (reference 2.75). Gate verdict PASS on every item; probe v1 0 tracebacks in kernel.log and serve.log. Promoted 2026-11-01 20:51Z (`l1a1y8sv2r3kp12lf15ust` v1). *Source:* `solution/analysis/harness_2026-10/gates/ust_gate.py`, `ust_record.py`; `solution/kaggle/generated/gate-predeclared-2026-10-31-franzen-l1a1y8sv2r3lf15ust.json`, `gate-results-2026-10-31-franzen-l1a1y8sv2r3lf15ust.json`; `paper/v3/analysis/figs/fig_gates.py`, recomputed. *Tier:* measured; strong (mechanism, verdict), none for any level effect. *Hedge:* The 6.7-level gap to the corner-joined build is about 1.0 standard error of a three-against-three difference (SD 8.25) and is not a measured regression; the gate compares to the pooled control by design.
V3-W29	Replay-check arm `verify` on the tool-fix build (`ustv` = ust + verify; read 2026-11-02 00:27Z). Motivation from research workflow wf_27428c3d-32e (11-01): clock per level is set by discovery; the model replays a simulator against recorded steps on about 6% of cleared levels and tests a goal guess against history about 0.3 times per run. Three Kaggle runs, 128, 124 and 128 levels (mean 126.7, +3.2 on the control, +4.3 on the tool-fix build), mean game score 56.74 (+1.89), 6,983 actions, median ratio 0.679; 45 of 58 idle turns act (0.78); helper calls (code blocks calling check_model or check_goal) 33, 50 and 18 per run, in 6, 6 and 5 games; `check_model` printed 85 reports (23, 43, 19), 33 matching every checked transition and 52 naming a mismatch (42 a wrong prediction, 10 only exceptions raised by the model's own step or observe code), 8 stopped at the time limit; `check_goal` printed 25 (19, 5, 1), none with a counterexample; `group` read in 59, 39, 59 blocks; getsource failures 0, 0, 0; SnippetTimeout 6, 3, 3; streaks of five or more idle turns 1.33 per run. Gate verdict PASS on every item, including mean levels at least 117.3 (the tool-fix build's 122.3 minus 5); probe v1 0 tracebacks, 8 levels, 247 actions, a helper called in 1 of 25 games (tu93; 10 probe transcripts empty). The research pass also flagged that some replica runs start play before the server is ready (measured mechanism and counts: V3-W32); the gate's reported-only readings (aligned-window levels, rule redefinition after a mismatch) were not computed. Promoted 2026-11-02 00:28Z (`l1a1y8sv2r3kp12lf15ustv` v1) for the final draw. *Source:* `solution/analysis/harness_2026-10/gates/ustv_gate.py` (argument 122.3), `ustv_record.py`, `verify_outcomes.py`; `solution/kaggle/generated/gate-predeclared-2026-11-01-franzen-l1a1y8sv2r3lf15ustv.json`, `gate-results-2026-11-01-franzen-l1a1y8sv2r3lf15ustv.json`; `docs/PROGRESS.md` 2026-11-01 19:40Z (research verdict); `solution/sim/test_franzen_verify.py`, recomputed. *Tier:* measured; strong (mechanism, use, verdict), none for any level effect. *Hedge:* A mismatch report counts wrong predictions and exceptions raised by the model's code (10 of the 52 are only exceptions); it does not show that the model then fixed its rule or that a level cleared because of it; the research figures (6%, 0.3) are transcript counts from earlier runs.

Table 4, continued

ID	Claim, source, tier and hedge
V3-W30	Board refresh to 2026-11-02 about 12:15Z, after the last daily draw scored. 81 scored submissions in all; 31 completed draws since the fork (28 live, 3 dead); best live draw still 40.98. The final draw, submitted 2026-11-02 03:06Z by the daily job (submission (withheld)), of the four-change harness build (kp12lf15ustv: stall check, corner-joined regions, tool fixes, replay check; pinned 00:28Z), scored 36.38; the two harness-build draws are 33.56 (11-01, kp12lf15us8) and 36.38. Valid draws: 17, mean 34.59, SD 3.39, range 28.97 to 40.98; with the 10-18 zero 18 draws, mean 32.67. Live draws before the yield change 7 (mean 29.25), after it 21 (mean 34.96, adapter draws included). Ratio of the 17 to the 56.65 development mean: 0.611. Live read 2026-11-02 about 12:15Z: our rank withheld; rank 1 75.56, rank 3 58.59, rank 5 56.02, rank 10 50.37, rank 20 41.73; daily-job snapshots of 11-02: 01:06Z our rank withheld; 07:06Z our rank withheld. 36.38 is below 40.09, so the two highest public draws are unchanged (40.98 on 10-20, 40.09 on 10-13) and Kaggle's automatic pick, with no Final Submission marked by hand, still selects them. *Source:* `solution/kaggle/kapi.sh subs` and `leaderboard 40` (read 2026-11-02 12:11Z to 12:15Z); `paper/v3/analysis/figs/data/board_draws_all_2026-11-02.csv`, `board_snapshots_2026-11-02.csv`, `kapi_subs_2026-11-02.txt`, `leaderboard_live_2026-11-02T1215Z.txt`; `solution/kaggle/generated/daily/2026-11-02T*.json`, recomputed. *Tier:* measured; strong. *Hedge:* Public scores on half of the hidden games, read before the 23:59Z deadline: ranks can still move as others' last runs score, and the final private score is not known until the private board is posted. The hand-marking state is V3-W35.
V3-W31	What sets the clock per level, reproduced from transcripts (research pass of 2026-11-01, re-read 2026-11-02). Baseline: 11 Kaggle replica runs of the 15-stream harness family before the replay check (uns2 x3, us8 x3, usc x3, ust-1031-1 and -2), 16,750 turns, 275 game plays, 1,387 cleared levels. Levels over 15 minutes are 26.8% of cleared levels from level 2 on (272 of 1,016 by the transcript rule) and hold 65.8% of their clock; on them the winning code first appears with a median 12.9% of the level's clock left (code-similarity proxy, matched on 94 of 272). 17.3% of the clock on cleared levels from level 2 on passes before the level's first action (median 83 s over all later-level starts, 84 s on cleared ones). Turns with no action: 14.0% of turns, 30.0% of clock, 20.4% of them cut at the output cap. Of 7,494 responses running 3 or more actions, 7.2% were stopped by the no-op guard and 0.9% by game over. Median actions over the human baseline 0.644. Tool errors 8.2% of 31,111 calls. 73.4% of level openings call a function written on an earlier level (the digest's 81% did not reproduce). A simulator was checked against recorded history by hand on 6.0% of cleared later levels (61 of 1,016), against 4.0% (11 of 275) in three earlier runs before the stall check. Replay-check build (ustv, 3 runs; readings, not claims): long levels 29.7% of clears and 71.4% of clock, winning code median 12.8% left (27 of 83), hand-written history checks 4.7% (13 of 279), a call to check_model or check_goal on 7.2% (20 of 279; check_model alone 14), both on 5, so hand-written only 2.9%, both 1.8%, helper only 5.4%, either 10.0% (28 of 279). *Source:* `paper/v3/analysis/discovery_read.py` (stdlib; writes `figs/data/discovery_levels.csv`, `discovery_summary.json`); `paper/v3/analysis/figs/fig_discovery.py`; research digest and scratch scripts in `docs/research/2026-11-01-harness/`, recomputed. *Tier:* measured; strong (counts), moderate (code-similarity proxy). *Hedge:* The transcript 'cleared' rule misses levels cleared on a game's last turn (1,016 against 1,113 in benchmark.json); the hand-written detector needs a loop over transitions in the model's own code and cannot see helper calls; the groups are different builds on different days.
V3-W32	Start-time confound, measured. Franzen's notebook releases the benchmark when the server reports READY or at a deadline 720 to 722 s after notebook start with the server still loading ('DEADLINE at 720s from notebook start; releasing the benchmark with the server still loading'). Of 23 Kaggle replica runs of the 15-stream family (kaggle-lf15*), 13 were released at the deadline; 6 started 11.5 to 21.3 minutes before serve.log's ready line: control lf15-1029-2, stall check uns2-1030-3, corner-joined us8-1031-1 and -3, scroll usc-1031-1, tool fixes ust-1031-1; the three replay-check runs started at most 1.0 minute early. Size of the effect on levels not estimated. *Source:* `paper/v3/analysis/start_time_read.py` (writes `figs/data/start_times.csv`); kernel.log and serve.log of each run, recomputed. *Tier:* measured; strong. *Hedge:* Vertex control runs not read; server ready from serve.log wall time mapped to kernel time through the launch line (about a second of slack).
V3-W33	Public harnesses and literature behind the replay check, verified against saved copies 2026-11-02. Twin (arXiv 2608.14490): 93.3 on the 25 public games with GPT-5.6 Sol; the ablation keeps the agent, model, file bridge and sandbox and disables the whole harness (executable world model, replay validation, exploration, planning, halt-on-mismatch executor) and scores 61.1, so the gap is not a replay-only effect; during the reported runs the goal check was asked for by prompt (its Appendix D); one scored run per game. Rodionov (arXiv 2607.15439): 4 nested variants by 4 model settings, one run each; the verification variant ranks first in every setting, margins 0.62 to 8.56 points over simplification; verification is prompted and tool-supported, not enforced, and the author does not separate it from the bundle. Kepler (arXiv 2610.00834): backtesting is instructed, not forced; the first mismatch voids the remaining plan. Tycho (arXiv 2607.28287): replays the latest attempt per level; route given as advice, no halting executor. arc-skill: every press needs a written prediction, plans halt at the first miss; the full-history rules check is optional. WorldCoder and PoE-World originate the program-world-model pattern (secondhand via Twin). Graph exploration (Rudakov et al., arXiv 2512.24156) reaches deep states at near-zero action efficiency per Twin. persistgym (arXiv 2610.26632): truncation costs little when facts can be re-observed. Rivals: no top-12 team published its changes after Franzen's notebook (read 2026-11-01); xz (rank 3) ported its solution to Kaggle 10-25, validates on public-25 and copycat-30, starts games at the 10-minute mark and RESETs every 7 minutes (forum 752485, 753843); Tufa reports scored notebooks lose about 40 minutes (forum 753843). Not verified: the digest's EMPA and AutumnBench specifics; Turducken's heal fine-tune and prompt claims (repo records only). *Source:* Saved pages and full texts in the session scratchpad (pubharness/, pdfs/twin.txt, forum JSON); fact sheet copied to `docs/research/2026-11-01-harness/LITERATURE.md`; BibTeX in `paper/v3/tex/refs.bib`, read. *Tier:* source-verified; strong for quoted numbers. *Hedge:* Frontier-model magnitudes do not transfer to a 6B-active open model; every comparison above is one run per game or per setting.
V3-W34	The research pass, as run 2026-11-01 (workflow wf_27428c3d-32e): five research angles (public ARC-AGI-3 harnesses, agent literature, code read, transcripts, competitive meta), one synthesizer (26 proposals, 5 ranked, 15 rejected) and four skeptics, one per top candidate, 10 agents. All four skeptic-reviewed candidates were refuted as level levers a three-run gate could see; adjusted effects per 25-game run: verify 0 to +1, codefirst about 0, coverage v3 about 0 (at most +0.2), costmodel about 0 (-6 to +3); groupdiff had no skeptic. Facts behind the verdicts (agents' own counts): coverage v3's back-test 9% of listed items later useful against a 20% bar (2% on stuck levels); after a cut reply the next first call already holds code or an action in 66 of 79; objective-driven thinking 0.7% of the clock. Only verify was built, with 10 of the skeptic's 12 build pitfalls fixed (one left, one not checked). Predicted against measured: the skeptic predicted low use (precedent GG6, a validation helper on Flash-Next, 1.5% of calls); the gate measured 33, 50 and 18 helper calls per run, 1.14% of all code blocks (101 of 8,842); level effect predicted 0 to +1, measured +3.2 on the control inside the skeptic's 4.5-level SE (no claim); check_goal found no counterexample, as predicted. *Source:* `docs/research/2026-11-01-harness/DIGEST.txt` and `LEDGER.md`; `solution/analysis/harness_2026-10/gates/ustv_gate.py`, `verify_outcomes.py`, recomputed. *Tier:* measured (gate, use share); agents' own counts (skeptic facts). *Hedge:* The skeptic facts were not rerun; the GG6 denominator (calls) may not equal code blocks.
V3-W35	Final Submissions marked by hand 2026-11-02 about 18:50Z (5 h before the 23:59Z deadline), after the last daily draw: the 2026-10-20 draw (40.98, submission (withheld)) and the 2026-10-13 draw (40.09, (withheld)); the submissions page counter read 2 of 2 after a reload. The same two are the two highest public draws. Earlier, on 2026-10-31, two marks were set and removed at the owner's request while draws continued. *Source:* `docs/PROGRESS.md` 2026-11-02 18:50Z and 2026-10-31 18:05Z (hand check in the UI), read. *Tier:* measured; strong. *Hedge:* The selection can be changed until the deadline; the page does not show the private score.

Table 4, continued

ID	Claim, source, tier and hedge
V3-W36	The interleaved yield wave (launched 2026-10-21 02:50Z to 02:54Z; read 2026-10-24 by its predeclared rules). Sixteen Vertex AI jobs were launched, eight per arm, four per region per arm in us-central1 and us-east4, submitted in A B B A order inside each region; four never ran (y8 c-2 and c-4 and y2 c-2 sat PENDING in us-east4 from 03:21Z and were cancelled at 06:40Z to free capacity for the adapter wave; y2 e-1 failed with 'Resources are insufficient in region: us-east4'), and no replacement ran. Twelve valid runs, six per arm, all complete with READY seen and no server death or out-of-memory. Levels per run on the 25 demonstration games, yield 8,192 (build l1a1y8sv2r3rq): yw-y8-c-1 114 (us-central1), c-3 129 (us-central1), e-1 113 (us-central1), e-2 127 (us-east4), e-3 120 (us-central1), e-4 121 (us-central1); mean 120.67, SD 6.53. Yield 2,048 (build l1a1y8sv2r3rqy2): yw-y2-c-1 123 (us-east4), c-3 125 (us-central1), c-4 119 (us-central1), e-2 124 (us-central1), e-3 123 (us-central1), e-4 125 (us-central1); mean 123.17, SD 2.23. Difference 8,192 minus 2,048: -2.50 levels, exact run-label permutation p 0.42, Welch p 0.41, run-bootstrap 95% interval -7.5 to +2.5 (narrow by construction with six runs); by region -3.8 (us-central1) and +4.0 (us-east4); at matched tokens -3.17 (p 0.34); tokens per game 256.8K against 252.9K (+1.5%); resume openers per game 8.97 against 26.95 (a third as many). Interval used in Figure 3 and the text: -2.50 plus or minus t(0.975, 6 df) x 8.25 x sqrt(2/6), with t 2.4469 and the six identical pairs' SD 8.25, the paper's convention for every interval at the pairs' SD: -14.2 to +9.2, so the test excludes the across-days +12.5 and any gain above about 9 levels, and does not exclude smaller gains; at the SD of the eight same-day runs, 5.95 (7 df, t 2.3646) it is -10.6 to +5.6. The two builds differ in one line of cell 7 (the yield token count); notebook sha256 prefixes a8e2e846639b (8,192) and e695102c96ba (2,048). The across-days comparison it was designed to settle carried 17% more tokens in seven of nine treated runs, worth about 5 of the 12.5 levels at the curve's elasticity (D9), and was not interleaved (D8): drift and those tokens are candidate causes of the non-reproduction, and this test cannot apportion them. *Source:* `paper/v3/analysis/out/yield_wave/read_2026-10-24.json` (per-run levels, tokens, region, validity; read by `paper/v3/analysis/yield_wave_read.py`); predeclared design `solution/kaggle/generated/gate-predeclared-2026-10-21-yield-interleaved.json` (declared_at 2026-10-21 about 02:48Z, before the first submission at 02:49:55Z; decision rules amended 02:55Z while all 16 jobs were PENDING); `docs/PROGRESS.md` entries 2026-10-21 02:51Z, 06:45Z and 2026-10-24 04:00Z (launch, the four missing runs, the verdict); interval recomputed by `paper/v3/analysis/figs/fig_yield.py` (asserted: -14.2 and +9.2), check marked inline. *Tier:* measured (levels, tokens); derived (interval); strong. *Hedge:* Six runs per arm, one build, Vertex AI Spot instances, regions not balanced after the four lost jobs; the design cannot show whether a resume changes decisions; the interval rules out gains above about 9 levels, not small ones; it ran on Vertex AI, not Kaggle.
V3-W37	Start-time confound in the six identical pairs, measured 2026-11-02. Per run, play start against server READY (kernel.log and serve.log, the method of V3-W32), early minutes with negative meaning play began after READY: p448 run 1 9.5 and run 2 -0.3 (122 and 106 levels); p448s14 -0.6 and -0.7 (117, 111); l1a1y8 -0.7 and 2.4 (128, 124); dpctl -0.5 and -0.3 (123, 101); L1 -0.4 and -0.3 (116, 113); l1a1 -0.3 and -0.3 (113, 109). Only 1 of 12 runs began play more than 3 minutes before READY (a first run, p448 run 1, which scored higher); ten began 0.3 to 0.7 minutes after READY. Without the p448 pair the second run is still lower in the other five (gaps 6, 4, 22, 3, 4); the largest gap, 22 (dpctl, GCP), is a pair in which both runs began after READY. Start time does not explain the first-minus-second shift (it may add to the 16-level gap of p448); the shift stays unexplained. Timing of the pairs, from the start_time field of `paper/v2/analysis/out/yield_runs.json`: the second run began 18.9 (p448), 16.6 (p448s14), 2.3 (l1a1y8), 26.6 (dpctl), 26.7 (L1) and 26.1 (l1a1) hours after the first, so the pairs ran one after another, 2.3 to 26.7 hours apart, three of the six within one UTC day (p448, p448s14, l1a1y8). *Source:* `paper/v3/analysis/start_time_pairs.py` (stdout), reading kernel.log and serve.log of the 12 runs under `solution/kaggle/generated/kernel-outputs/` and `paper/v2/analysis/out/e4_noise.json`, recomputed. *Tier:* measured; strong. *Hedge:* Same mapping as V3-W32 (about a second of slack); the reading is of when play began, not of an effect on levels; six pairs only.
V3-W38	Dated pre-declaration files, read 2026-11-02. (a) Interleaved yield test: `solution/kaggle/generated/gate-predeclared-2026-10-21-yield-interleaved.json`, declared 2026-10-21 about 02:48Z before any job (first submission 02:49:55Z); its decision rules were amended at 02:55Z with all 16 jobs still pending and no run started (the 02:48Z rules are kept in the file); file modified 2026-10-21 02:56Z; sha256 ed5fed72386c6f91. (b) Harness kill rules: five files, uns2 (declared 2026-10-30T03:40Z), us8 (2026-10-30T05:39Z), usc (2026-10-31T12:05Z), ust (2026-10-31T18:09Z) and ustv (2026-11-01T19:33Z), each stating it was written before any run of its build was pushed; the uns2 wave schedule, not its gate, was amended at 07:52Z after two runs were read. (c) Final Submission rule: `solution/kaggle/generated/finals-rule-predeclared-2026-10-12.json`, declared 2026-10-12T13:45Z before any draw of the final pin existed and amended as v4 on 2026-10-20T07:30Z, after the 2026-10-20 draw (40.98) was submitted at 02:06Z; sha256 0524253bd1092ecb. The two marks (40.98 and 40.09) equal Kaggle's automatic pick (V3-W35). *Source:* The three kinds of file above; `paper/v3/evidence/data/board_draws.csv` for the draw time, read. *Tier:* read; moderate. *Hedge:* Dates are the files' own headers, supported by file modification times and not externally timestamped. The Writeup uses (a) and (b) only.
V3-W39	The closest public work, described without numbers: "Before Tuning the Harness: Repeat-Controlled Evaluation and Request-Level Accounting for ARC-AGI-3" (GitHub repository arc3-repeat-controlled-eval, 24-page PDF), which repeats identical-code submissions before comparing changes and accounts cost per request. The Writeup says only that it repeats identical runs and accounts cost per request, and that we price tokens in levels from logged level attempts; "new here, as far as we found" is hedged in the text. *Source:* Public repository page, by the Paper Track research pass (its notes are in the session scratchpad, not in the repository); no number from it is used, read. *Tier:* read; moderate. *Hedge:* Described from the repository page and README, not from a full read of its PDF; nothing is claimed about what it lacks.
V3-W40	Release state on 2026-11-02: nothing is public. An anonymous request for the submitted kernel (`(withheld)/arc3-franzen-l1a1y8sv2r3p-a`) and for the D3 dataset (`(withheld)/flashnext-mtp-d3`) each returned not found, while Franzen's public notebook returned found as a control. The project repository has no remote and no LICENSE file; 25 overlay files under `solution/kaggle/franzen_overlays/` are untracked, among them memcap, toolfast, promptobs, qsaring, asyncmrope, d3draft, watchdog and failclosed (the eight behind the 40.98 configuration listed in Appendix M); `solution/kaggle/build_franzen_arms.py` is modified and uncommitted. *Source:* Repository state read locally 2026-11-02 (`git remote -v` empty, no `LICENSE*` file, `git status --short solution/kaggle/franzen_overlays` shows 25 untracked files, `git status --short solution/kaggle/build_franzen_arms.py` shows `M`). The anonymous requests are the planning review's of 2026-11-02 (Paper Track plan section 1, session scratchpad), not repeated by the figure and PDF pass, which made no network call, check marked inline. *Tier:* measured (local git); reported (anonymous requests); strong for the repository, moderate for the requests. *Hedge:* Every line changes the moment the owner releases; recheck both anonymous requests from a logged-out browser before the PDF or the Writeup is posted.
V3-W41	Harness leads depend on the control. The pooled control (seven runs, 123.4 levels) is five Vertex AI runs (129, 120, 125, 122, 135; mean 126.2) and two Kaggle runs (123, 110; mean 116.5), a 9.7-level gap between the two sets. Against the pooled control the four shipped changes lead by +4.9 (stall check, three runs 141, 126, 118), +5.6 (corner-joined, 132, 128, 127), +3.2 (replay check, 128, 124, 128) and -1.1 (tool fixes, 123, 129, 115); the scroll build (111, 122, 130) trails by 2.4. Against the two Kaggle controls alone the same arms read +11.8, +12.5, +10.2, +5.8 and +4.5 (scroll), that is, about +6 to +13 for the four shipped changes. In the 18-run fine-tuning control set (V3-X5, V3-W9) the nine Vertex and nine Kaggle runs differ by under one level (124.8 against 124.0). *Source:* Arm means from the three-run readouts in V3-W20, V3-W22, V3-W27, V3-W28 and V3-W29 and `paper/v3/tex/sections/i2_harness.tex` (table of every run), recomputed. *Tier:* derived; moderate. *Hedge:* Two Kaggle control runs are too few to say whether the 9.7-level gap is platform or noise, and the arms ran on Kaggle one after another, so neither control isolates a platform effect; no lead is claimed.

Table 4, continued

ID	Claim, source, tier and hedge
V3-W42	Release update 2026-11-02 about 21:00Z (supersedes V3-W40 for the repository). Public GitHub repository (withheld)/ arc3-decisions-or-tokens, tag v1, commit c77243c (598 files, 17 MB, history started fresh with the GitHub noreply identity); anonymous requests for the repository and its LICENSE returned 200. LICENSE: paper, figures, data CC BY 4.0; code CC BY 4.0 or MIT-0 at the reader's choice. Not included (licence unread or unsettled, per the 2026-11-02 audit): the Tufa Duck bundle (franzen-bundle), the fork candidate, shiiin9's D-prime file, p448_serve.py (lee-chang-93 expert ids). The D3 dataset got version 2 with the Qwen Community License 1.0 text and a NOTICE (ursuciprian attribution) but was still private then (reported public on 2026-11-03, not re-requested); the submitted kernel was private then and is not re-checked. *Source:* `release/` kit and `docs/PROGRESS.md` 2026-11-02; GitHub API and anonymous curl on 2026-11-02; the licence audit in `docs/research/2026-11-01-harness/LICENCES_AUDIT_2026-11-02.md`, read. *Tier:* measured; strong. *Hedge:* Dataset and notebook publicity not done; the audit is not legal advice and could not read any Kaggle notebook licence field.
V3-W43	Final public board, read 2026-11-03 01:06Z by the daily job, after the 2026-11-02 23:59Z deadline; later snapshots to 16:06Z are identical. Our best public score is 40.98 (submission (withheld)), our rank withheld teams, 81 scored submissions. Rank 1 75.56, rank 2 70.24 (Tufa Labs), rank 3 60.95 (NVARC3), rank 4 58.59 (xz), rank 5 56.02 (Team Turducken), rank 10 51.04, rank 20 42.34; field median 2.58. This supersedes the live read of V3-W30 (our rank withheld at about 12:15Z on 11-02, before the deadline). *Source:* `daily/board-2026-11-03.csv` (5,073 rows), `figs/data/leaderboard_final_2026-11-03.txt`, recomputed. *Tier:* measured; strong. *Hedge:* Public scores on half of the hidden games; private scores were not posted when this was written. The refresh request gave the same figures for a 00:23Z read; this pass made no network call.
V3-W44	Same-card no-adapter control for the fine-tuning result, run 2026-11-03. Two runs of the adapter's own base arm `l1a1y8sv2r3hm32` with no adapter, public 25 games, on the same RTX PRO 6000 (G4 Spot) as Vertex AI jobs in us-central1: adctl-1 cleared 132 levels (mean game score 62.80, 12 games won; preempted once and ran as attempt 2) and adctl-2 130 (59.33, 10 won), launched 21:06Z and 21:07Z. Both are above all three adapter runs (109, 115, 117). Among these five same-card runs the three adapter runs are the three lowest: exact one-sided rank p = 1/C(5,3) = 0.10. The arm's earlier Kaggle run (126) is also above all three. Pooling the two controls with the 18 runs of V3-W9 (20 runs, mean 125.05, SD 4.89), 5 of 1,771 splits are as extreme (p 0.0028, tied sums counted as extreme), but that pool mixes platforms and is not same-card. In both runs READY came before play (454 s after the restart in adctl-1, 598 s in adctl-2) and no deadline release is logged. *Source:* `solution/kaggle/generated/kernel-outputs/vertex-adctl-1/benchmark.json` and `-2/benchmark.json` (sum of `levels_completed` over 25 games, re-summed 2026-11-03: 132 and 130); `paper/v3/evidence/data/lora_controls_vertex.csv`; `progress.txt` and `kernel.log` in each directory (READY lines, SIGTERM at 21:44:20Z in adctl-1); `lora_replicas.csv`; `bench_pool18_levels.json`. Rank tests recomputed by enumeration 2026-11-03, check marked inline. *Tier:* measured; moderate (counts), weak (any effect of the adapter). *Hedge:* Two controls only, so the smallest possible p is 0.10 at five runs. The adapter ran on a rented GCP VM on 2026-10-19 and the controls on Vertex AI 15 days later: different services and days, not interleaved, so service and date are not separated from the adapter. One control restarted after a preemption. The later harness control found Vertex AI runs 9.7 levels above two Kaggle runs (V3-W41), so an offset toward higher Vertex scores is not excluded. No causal claim: the result is that the adapter stayed below a same-card control, not that it caused the drop.

Not run in v2, and named as such there: the interleaved yield test; K0 headroom (the acceptance ceiling c) and per-position acceptance at K=4; the token-only cut to 70% of the budget; any second benchmark. K0 headroom has since been measured (V3-T7).

C All draws

Table 5 lists every scored submission since 2026-09-30: two Kaggle system errors, three draws of the retired September stack, and the 31 completed draws since the fork to 2026-11-02 (28 live, 3 dead). An error did not spend the daily slot: both errors were followed by a completed run the same UTC day. Submission ids are in `paper/v3/evidence/data/board_draws.csv`; the best draw is (withheld).

Groups. The seven live draws before the yield change average 29.25 (SD 1.58). The eight live draws of the yield family to 2026-10-20 average 34.28 (SD 4.25); the exact one-sided permutation p against the earlier seven is 18/6,435 = 0.0028 (Mann-Whitney U 50.5 of 56). That is a rank test across different, time-ordered configurations: it ranks the stack and attributes nothing. The shipped configuration's three draws average 38.43 (SD 3.67) and the variant's four valid draws 31.15 (SD 1.70); the three are the top three of the seven (exact one-sided $p = 1/35$; Welch t 3.19). We formed the hypothesis that the shipped configuration draws higher on 2026-10-19 from two draws against four ($p = 1/15$); the one draw since, 40.98, sits above all four of the variant's, which has probability 1/5 under no difference. The five keepalive draws after it, play-identical to the shipped configuration, average 35.04, between the two, so the gap has shrunk as chance and calendar predict; the adapter's three average 37.22. The variant changes only the draft model's vocabulary, which should leave the output distribution unchanged (speculative sampling preserves it when the verify path is correct), so we do not attribute the gap to it; calendar or node variation plus chance is the economical reading. The same calendar term applies to every time-ordered comparison in the paper.

Selection. The expected maximum of eight normal draws is 1.42 SD above their mean, about 40.3 for the yield family's eight to 2026-10-20; of fifteen, 1.74 SD (V3-B6). Figure 12 draws the same point for the agent's twelve valid draws to 2026-10-28, with Dodge's expected maximum over the empirical distribution [Dodge et al., 2019] beside a normal model.

Table 5: All scored submissions since 2026-09-30 (dates 2026, times UTC; public score on half of the 110 hidden games, higher is better). Bytes: size of the submission file Kaggle reports (not read for the last five draws). †: dead run, excluded from means. Shipped: the configuration behind the 40.09 and 40.98 draws; variant: the same agent with the own-traffic draft vocabulary, which should leave the output distribution unchanged; keepalive: the shipped agent with a warmup keepalive that acts only while the server boots; adapter: the round B adapter of Sec. 7; 15 streams: the keepalive agent with 15 games in flight instead of 14; harness: the 15-stream agent with the harness changes of Sec. 6 (two on 11-01, all four shipped on 11-02).

Date	UTC	Group	Change from the previous submission	Score	Bytes
09-30	00:01, 21:17	September stack	Kaggle system error, twice; the slot was not spent	error	0
09-30	22:04	September stack	retired stack, before the fork	5.90	3,846
10-01	00:02	September stack	none	5.76	3,907
10-02	00:54	September stack	a different build of the retired stack	4.03	3,815
10-03	00:06	base notebook	Franzen's notebook as published	0.00†	3,411
10-04	00:06	before the yield change	startup trims; expert pruning (448 of 512); 14 streams	28.97	3,865
10-05	00:06	before the yield change	slot priority across games (shiiin9's formula)	31.90	3,811
10-06	00:06	before the yield change	none	29.44	3,810
10-07	00:06	before the yield change	none	30.56	3,893
10-08	00:06	before the yield change	none	28.19	3,851
10-09	00:06	before the yield change	transcript-waste fixes; reply cache; agent fixes	27.15	3,841
10-10	00:06	before the yield change	none (same kernel version as 10-09)	0.00†	3,411
10-11	02:06	before the yield change	two fail-closed checks (they act only on a dead run)	28.51	3,832
10-12	02:06	yield family	yield after 8,192 tokens (was 2,048); index-key fix; asynchronous M-RoPE upload	34.34	3,835
10-13	02:06	yield family, shipped	watchdog; memory fraction 0.955 and sandbox cap; image hashing off the scheduler; incremental tool state; drafter refit; prompt fields; +11% tokens	40.09	3,878
10-14	02:06	yield family, shipped	none	34.22	3,882
10-15	02:06	yield family, variant	own-traffic draft vocabulary	31.63	3,881
10-16	02:06	yield family, variant	none	30.97	3,838
10-17	02:06	yield family, variant	none	33.05	3,886
10-18	06:56	yield family, variant	none; submitted late after two missed fires	0.00†	3,411
10-19	02:06	yield family, variant	none	28.97	3,868
10-20	02:06	yield family, shipped	back to the shipped configuration	40.98	3,847
10-21	15:06	yield family, keepalive	warmup keepalive: the warmup game re-sends RESET while the server boots (acts only before it is ready)	35.05	3,862
10-22	14:10	yield family, keepalive	none	36.03	3,850
10-23	02:25	yield family, keepalive	none	37.56	3,820
10-24	03:40	yield family, adapter	round B adapter, submitted before its gate read; it failed the gate (Sec. 7)	35.90	3,898
10-25	03:08	yield family, adapter	none	39.79	3,853
10-26	03:07	yield family, adapter	none	35.98	3,894
10-27	16:19	yield family, keepalive	back to the keepalive agent, with a longer keepalive cap (about 2 h)	33.09	3,855
10-28	04:34	yield family, keepalive	none	33.45	3,859
10-29	02:41	15 streams	one more game in flight (15 streams, not 14); submitted by hand after a missed fire	38.17	n/a
10-30	03:06	15 streams	none	29.63	n/a
10-31	03:06	15 streams	none (before the harness changes of Sec. 6)	35.25	n/a
11-01	02:06	harness	the stall check and corner-joined regions (Sec. 6); one draw says nothing about this build's mean	33.56	n/a
11-02	03:06	harness	adds the tool fixes and the replay check (Sec. 6); the final draw	36.38	n/a

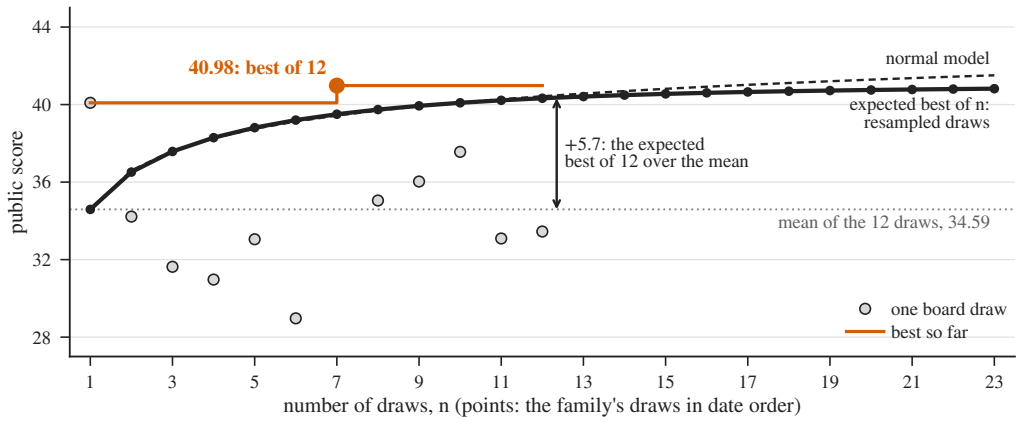


Figure 12: How much of 40.98 is selection. Gray points: the agent's twelve valid board draws in date order (shipped configuration, its draft-vocabulary variant and its keepalive variants). Solid line: expected best of n draws resampled from those twelve (Dodge's closed form on the empirical distribution, so it cannot exceed 40.98), 40.33 at twelve and 40.82 at 23, the live draws since the fork to 2026-10-28; dashed: a normal model with the same mean and SD; dotted: the mean, 34.59. Orange: best so far. Higher is better.

Dead draws. All three zeros are status complete with the same 3,411-byte file, against 3,810 to 3,907 bytes for live files. The 2026-10-03 and 2026-10-10 zeros fell in field-wide failures of the 00:00 to 00:20Z wave; the 2026-10-18 zero came from a 06:56Z submission after the two morning fires never ran, inside a field cluster of early endings (Appendix H). Its cause is unknown and cannot be known, because a scored run returns no log.

Final Submissions. Finals rule v4 (adopted 2026-10-20 at 07:30Z, after the 40.98 draw was submitted at 02:06Z, under the author's delegation and open to the author's veto until 2026-10-28) defines the agent's family as the shipped configuration and its play-identical variants; valid draws exclude errors, zeros and field-wide failure days. With no eligible adapter, it marks the family's two best valid draws, 40.98 and 40.09. Both are marked by hand: on 2026-10-31 two marks were set and removed at the author's request (counter 0 of 2) while draws continued, and on 2026-11-02, after the last draw, they were set again (2 of 2 after a reload). They are also the two highest public draws. The draws of 2026-10-29 to 10-31 are of a 15-stream build outside the family, and those of 2026-11-01 (33.56) and 2026-11-02 (36.38, the final draw) are of harness builds (Sec. 6); neither beat 40.09, so the two highest public draws are unchanged.

D Tried and not shipped, with costs

Table 6 lists the levers we built, ran and did not ship, each with the number that decided it and what it cost. Level counts are single development runs on the demonstration set unless a pair is stated, so they screen and do not settle (Sec. 3). A row reports this stack at this size and says nothing about the idea elsewhere; "held" means not shipped and not ruled out. Costs are in runs: one development run is about 2.2 free Kaggle GPU-hours, or $3.88 to $5.45 on a rented RTX PRO 6000 at list prices (about $4 on the earlier rented VM). Figure 13 places every measured arm, shipped or not, against the noise of single runs.

Table 6: Levers built and not shipped, with the deciding number and the cost. Levels are single development runs unless stated; counters come from server or behaviour logs.

Lever	Result	Decision	Cost
Fourth draft step	Accept length $+9.8\%$, step $+13.5\%$, decode -4.2%, harness tokens per second -2.8%	Not shipped: 0.275 against 0.379 needed (Appendix G)	1 run against 2 controls
Relaxed acceptance (0.5)	Decode $+9\%$; levels 113 against 114, fewer at 150K tokens per game	Held: lossy, levels tie	1 run

Lever	Result	Decision	Cost
16 streams, host-RAM KV tier	Tokens per second $+7\%$; levels 115 against 114; later code line 104 against 112 (the pooled pair of 123 and 101)	Held: tie twice (-8 against the pooled control)	2 runs
12 streams without pruning	Cache hit 72.2%, 90 levels	Failed: cache evictions	1 run
Strict thinking budget (10,240 tokens)	109 levels; mechanism works (cut replies removed); an earlier run 115 against 116 and 113	Held: tie	2 runs
Output cap raised to 16K	120 levels; cut replies 26 against a limit of 20	Rejected	1 run
BF16 draft KV	Full-size run: CUDA out of memory seven seconds into play, 0 levels; probe pair accept length $+0.010$ [-0.040, 0.062]; costs 8% of the KV pool	Rejected	1 run, 2 probes
keepdata overlay	2 of its 6 counters failed in the one run	Dropped by its predeclared rule	1 run
Solved-level memory, strategy audit, early handover, opener port	111, 110, 108 and 105 levels against the pooled control of 114 (3 to 9 below; single runs, inside the noise)	Dropped for lack of a mechanism, not proven harm (Table 7)	4 runs
71K retained history	113 levels against 114	Dropped: tie	1 run
Leaner history (older reasoning and images as markers)	Paired decision test: first-action agreement -0.071 against the noise floor (90% interval -0.132 to -0.004, which contains the -0.05 tolerance; two-sided paired p 0.09); acts sooner; a parser defect found later leaves the verdict unchanged (-0.081)	Killed before any development run (Sec. 3)	0.72 GPU-hours
Rejection-sampling LoRA, rank 16	Levels 109 and 115 at full strength, 117 at half, against 119 to 129 on Kaggle and 132 and 130 in two later same-card controls on Vertex AI (not interleaved)	Stopped by the project's rule: 130 per run to proceed, 120 or less at half strength stops (Sec. 7.1)	Play data about 13.5 h of rented GPU (about $17, estimated), training 4.78 h ($7.89), 3 runs at most 9.6 h (at most $16)
Own-traffic draft vocabulary	Gate passed: decode $+1.2\%$ (predicted $+1.4\%$ to $+1.7\%$), 126 levels against 129	Shipped 2026-10-14 to 2026-10-19, then pinned back on a board hypothesis that we do not attribute to it (Sec. 8)	1 run, 1 probe
Group-relative RL, round A	Two adapters gated; the first-wave lead did not replicate, and the full-strength merge read -0.6 levels	Not shipped (Sec. 7.2)	Program ledger $87 to $110 at list prices by 2026-10-21 01:00Z (Appendix L)

D.1 First-run comparisons replayed

Table 7 replays our first-run comparisons under one rule: beating the control run by 10 levels or more is a win, trailing by 10 or more a loss, anything else a tie. The replay is post hoc. At the time each comparison used the control then at hand, predeclared counters and often more than one run, so the table shows what a one-run rule would have called. Our decisions used more evidence.

The rule calls 2 wins, 6 losses and 9 ties. Both wins shipped on a verified mechanism and held up on replication (pairs 122 and 106, 128 and 124). Every control in a loss call was the higher run of its byte-identical pair: 122 of 122 and 106, 123 of 123 and 101, 128 of 128 and 124. The six loss calls rest on three control runs (122 three times, 123 twice, 128 once), so this is three facts. Against the pooled control five of the six losses shrink to 8 levels or fewer. Row 16 stays at -11, but that run failed its own mechanism check and lost its server to an out-of-memory error in the last minutes of the window. Row 16 is also the one run that differs from a treated run in the yield line alone, with all other changes in place: 115 levels against 119 for row 17. Row 12 was a loss call (-10) that we shipped on its mechanism, and a tie against the pooled control.

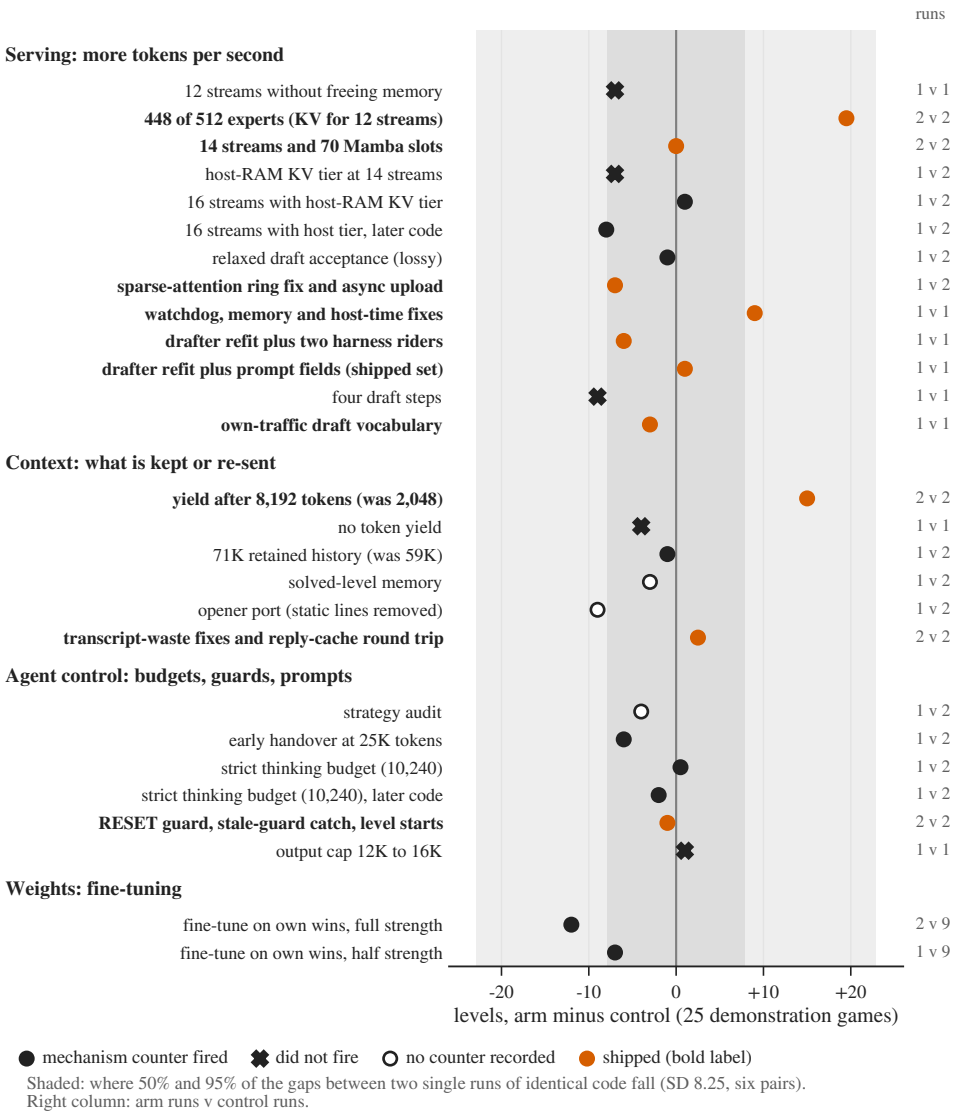


Figure 13: What we tried, against the noise of single runs. One row per measured arm, grouped by lever class: levels of the arm minus its control on the 25 demonstration games (the pooled byte-identical pair where one exists, so a few gaps differ from the control used at the time). Shaded: where 50% and 95% of the gaps between two single runs of identical code fall (SD 8.25, six pairs). Filled circle: the mechanism counter fired; cross: it did not; open circle: no counter recorded; orange with a bold label: shipped. Right column: arm runs against control runs. Not drawn: arms with no level reading (BF16 draft KV, leaner history, round A).

On the older stack five arms were promoted on a first or lucky run. Their advantage over the control family was $+7.0$, $+9.0$, $+11.0$, $+4.0$ and $+9.0$ levels on that run and $+2.5$, $+1.0$, $+0.7$, -0.5 and $+7.0$ after the byte-identical replicate: means $+8.0$ and $+2.1$ (73% shrinkage). Two of the five were withdrawn the same or the next day, and one survived its replicate.

Table 7: The 17 first-run comparisons on the current stack. Rows refer only to earlier rows. Levels are single replica runs. "Pooled" is the difference against the pooled control of its byte-identical pair, shown for the six loss calls.

No.	Arm	Levels	Control	Diff.	Call	Pooled
1	Expert pruning to 448, startup trims	122	92, startup-trims run	+30	Win	
2	Row 1 plus solved-level memory	111	122, row 1	−11	Loss	−3
3	Row 1 at 14 streams	117	122, row 1	−5	Tie	
4	Row 1 plus strategy audit	110	122, row 1	−12	Loss	−4
5	Row 1 plus early handover	108	122, row 1	−14	Loss	−6
6	Row 1 plus opener port	105	114.0, row 1 pooled	−9	Tie	
7	Row 3 plus host-RAM tier	107	114.0	−7	Tie	
8	Row 1 at 16 streams plus host-RAM tier	115	114.0	+1	Tie	
9	Row 3 plus 71K retained history	113	114.0	−1	Tie	
10	Row 3 plus relaxed acceptance	113	114.0	−1	Tie	
11	D-prime stack plus waste fixes and reply cache	116	123, D-prime stack run 1	−7	Tie	
12	Row 11 plus agent fixes	113	123, D-prime stack run 1	−10	Loss	+1
13	Row 11 plus strict thinking budget	115	123	−8	Tie	
14	D-prime stack at 16 streams, host tier	104	123	−19	Loss	−8
15	Row 12 plus yield 8,192	128	113, row 12	+15	Win	
16	Row 15 plus ring fix, async upload, no token yield	115	128, row 15	−13	Loss	−11
17	Row 15 plus ring fix, async upload	119	128, row 15	−9	Tie	

E Yield: the comparison and secondary analyses

The nine yield runs are every full-length run at 8,192 on the current code line. The first two carry no change besides the yield and two checks that act only on the scored path; the later seven also carry serving changes. The 21 controls are every full-length run at 2,048 on that line.

Table 8 is the comparison behind Sec. 5. Since 2026-10-14 every run in it is adjudicated: the own-traffic draft-vocabulary run, open at the time of v2, passed its gate, so the eight-run row is now a plain sensitivity row (V3-H5).

Table 8: Yield 8,192 against 2,048: difference in mean levels per run, run bootstrap 95% interval (n/a for two runs or days), and exact two-sided permutation p over run labels, assumed exchangeable. Controls are the 21 runs at 2,048.

Comparison	Runs	Diff.	95% CI	Exact p
All nine	9 vs 21	+12.5	+9.2 to +15.8	2.4×10^{-6}
Eight	8 vs 21	+12.3	+8.8 to +15.8	7.9×10^{-6}
Six	6 vs 21	+13.5	+9.7 to +17.1	2.4×10^{-5}
First two, their two controls	2 vs 2	+15.0	n/a	0.33
By UTC day	3 vs 6 days	+10.9	n/a	0.012

The day row's p is its least possible, $1/84$. The 30 runs are every full-length run at the two settings on the current stack. On a development run the first two at 8,192 differ from their controls in the yield setting alone; the seven later runs carry other changes. Eight leaves out the own-traffic draft-vocabulary run, whose gate was still open in v2 and has since passed; six also leaves out the 16K-output-cap run and the four-draft-step run. Left out: three older-stack runs (they would widen the gap), the run without a token yield (115 levels against 119 for the same code at 8,192; server out of memory at 6,893 s) and an 8,192 run that died at start (0 levels; counted, the mean of ten is 111.6).

The interleaved wave. Sixteen Vertex AI Spot jobs (one RTX PRO 6000 each, the replica image, the 25 demonstration games, 14 streams, one 7,200 s window) were launched on 2026-10-21 between 02:50 and 02:54 UTC: eight per arm, four per region per arm in us-central1 and us-east4, in A B B A order inside each region. The two builds differ in one line, the yield token count (8,192 against 2,048); both log requests. The design and decision rules are in a file dated 2026-10-21, written before the first job was submitted; its rules were amended at 02:55 while all 16 jobs were still pending. Four jobs never ran: three sat pending in us-east4 and were cancelled at 06:40 to free capacity for the adapter wave, and one failed on a regional stockout. No replacement ran, so the read uses six runs per arm (Table 9). The 8,192 arm scored 2.5 levels below the 2,048 arm (permutation

p 0.42; by region -3.8 in us-central1 and $+4.0$ in us-east4), 3.2 below at matched tokens, on 1.5% more generated tokens and a third of the resume openers. At the six pairs' SD (8.25, 6 df) the 95% interval on the difference is -14.2 to $+9.2$; at the SD of the eight same-day runs (5.95, 7 df) it is -10.6 to $+5.6$. The test therefore excludes the across-days $+12.5$ and any gain above about 9 levels, and does not exclude smaller gains. Two candidate causes of the non-reproduction are drift between days and the extra tokens that seven of the nine across-days runs generated, worth about 5 of the 12.5 levels at the curve's elasticity; this test holds tokens equal and cannot apportion the two.

Table 9: The twelve valid runs of the interleaved wave: levels cleared on the 25 demonstration games and generated tokens, by arm (yield threshold) and the region where the run completed. Means 123.17 (2,048) and 120.67 (8,192).

Yield	Run	Region	Levels	Tokens (M)
2,048	`y2-c-1`	us-east4	123	6.30
2,048	`y2-c-3`	us-central1	125	6.30
2,048	`y2-c-4`	us-central1	119	6.30
2,048	`y2-e-2`	us-central1	124	6.31
2,048	`y2-e-3`	us-central1	123	6.39
2,048	`y2-e-4`	us-central1	125	6.34
8,192	`y8-c-1`	us-central1	114	6.43
8,192	`y8-c-3`	us-central1	129	6.37
8,192	`y8-e-1`	us-central1	113	6.47
8,192	`y8-e-2`	us-east4	127	6.37
8,192	`y8-e-3`	us-central1	120	6.39
8,192	`y8-e-4`	us-central1	121	6.49

Run flow. Table 10 lists every full-length replica run on this stack. Of the 34, 30 enter the main comparison. No 2,048 run lost its server. The 300 s probes, one 1,800 s smoke run and two watchdog rehearsals are not full-length runs.

Table 10: The 34 full-length replica runs on this stack, with levels per run, and one run outside them. Used: in the main comparison of Table 8.

Group	Runs	Levels	Use
Yield 2,048, same-stack controls	9	123, 101, 104, 116, 113, 113, 109, 115, 109	Control
Yield 2,048, earlier-stack controls	12	122, 106, 117, 111, 111, 110, 108, 105, 107, 115, 113, 113	Control
Yield 8,192, first two	2	128, 124	Treated
Yield 8,192, later seven	7	119, 120, 128, 122, 129, 120, 126	Treated
No token yield	1	115	Left out: server out of memory at 6,893 s, 5.7 minutes before the window ended
Fork before expert pruning	3	92, 97, 90	Left out: earlier code line; counting them would widen the gap
Yield 8,192, died at start (not among the 34)	1	0	Left out: counting it makes a ten-run mean of 111.6

Other comparisons. Table 11 gives the comparisons that Table 8 omits. The unit is the run. Intervals that resample two runs are narrow by construction, so we print none for the first two against any control group. Same-stack controls are the nine 2,048 runs of the code line that the 8,192 runs share, and earlier-stack controls the 12 before it. Pinned-code controls are 12 whose code was, or replicated, a pinned or near-pinned configuration. Kaggle controls are the 13 run on Kaggle; eight ran on a cloud VM.

With replicates averaged, 8 yield configurations against 16 control configurations differ by $+12.8$ (Monte Carlo $p < 10^{-5}$, no hit in 100,000 shuffles). Per game, the nine are ahead on 22 of 25 games, behind on 2 and tied on 1 (run-label permutation $p = 3 \times 10^{-4}$). The first two against all controls are 21, 3 and 1, and against their own two controls 12, 2 and 11 ($p = 0.17$). The 22 games that gain carry $+13.0$ levels and the two that lose -0.5, so the gain is spread.

Table 11: Further comparisons of mean levels, yield 8,192 against 2,048, on the replica. Exact two-sided permutation p over run labels; bootstrap over runs; the last interval is at the six-pair SD of 8.25 (6 df). Control groups are defined in the text.

Comparison	Runs	Diff.	Bootstrap CI	CI at SD 8.25	Exact p
First two, same-stack controls	2 vs 9	$+14.6$	n/a	-1.2 to $+30.3$	0.018
First two, all controls	2 vs 21	$+14.5$	n/a	-0.4 to $+29.5$	0.0040
First two, pinned-code controls	2 vs 12	$+13.1$	n/a	-2.3 to $+28.5$	0.022
First two, Kaggle controls	2 vs 13	$+14.7$	n/a	-0.6 to $+30.0$	0.0095
Later seven, all controls	7 vs 21	$+12.0$	$+8.3$ to $+15.7$	$+3.1$ to $+20.8$	2.7×10^{-5}
Later seven, same-stack controls	7 vs 9	$+12.0$	$+7.0$ to $+17.0$	$+1.8$ to $+22.2$	0.0012
Later six, all controls	6 vs 21	$+11.5$	$+7.8$ to $+15.6$	$+2.2$ to $+20.9$	1.05×10^{-4}
Nine, same-stack controls	9 vs 9	$+12.6$	$+7.8$ to $+17.2$	$+3.0$ to $+22.1$	2.9×10^{-4}
Nine, earlier-stack controls	9 vs 12	$+12.5$	$+8.8$ to $+16.0$	$+3.6$ to $+21.4$	3.4×10^{-5}
Nine, pinned-code controls	9 vs 12	$+11.1$	$+6.9$ to $+15.3$	$+2.2$ to $+20.0$	3.1×10^{-4}
Nine, Kaggle controls	9 vs 13	$+12.7$	$+9.2$ to $+16.0$	$+3.9$ to $+21.4$	1.8×10^{-5}

Contrasts that hold tokens equal. The matched contrast of Sec. 5 sets the first three 8,192 runs (128, 124 and 119 levels; 5.67M tokens on average) against the Kaggle thinking-budget control and the two cloud controls (109, 113 and 109 levels; 5.62M). With 20 possible splits, the exact p of 0.05 is the least possible. The same-minute pair is the first 8,192 run and the thinking-budget control, which started at 04:08 and 04:11 UTC on 2026-10-10.

Where on the token axis the gap opens. Table 12 reads levels at equal generated tokens per game; games that stopped earlier keep their final levels.

Table 12: Levels at equal generated tokens per game: yield 8,192 minus the 21 controls (higher is better). The later seven include the own-traffic draft-vocabulary run. Nine horizons were computed. The first two lead at 100K ($p = 0.016$) and 150K ($p = 0.028$, exact one-sided), the only horizons where we computed it.

Tokens per game	First two	Later seven	Later-seven p	Games running (first two / later seven / controls)
25K	$+3.2$	-0.7	0.78	25.0 / 25.0 / 25.0
50K	$+3.1$	0.0	0.53	25.0 / 24.4 / 24.8
75K	$+3.3$	$+1.6$	0.23	24.5 / 23.9 / 23.9
100K	$+9.1$	$+0.3$	0.46	22.5 / 23.1 / 23.2
150K	$+8.8$	$+0.3$	0.45	19.5 / 20.9 / 19.6
200K	$+9.1$	$+2.0$	0.18	15.0 / 17.9 / 14.6
250K	$+12.8$	$+5.6$	0.015	10.0 / 13.9 / 8.0
300K	$+13.4$	$+8.9$	4.5×10^{-4}	4.0 / 8.3 / 2.8
400K	$+14.5$	$+11.8$	2.7×10^{-5}	0.5 / 0.6 / 0.0

The first two are ahead from 100K with the same total tokens as their control. The later seven match the controls through 150K and open a gap after 200K, where more of their games are still running (Sec. 5).

Mechanism and replay. Resume openers fell from 565, 634 and 572 per run at 2,048 to 185, 151 and 196 in the first three runs at 8,192, and stayed at 151 to 215 in all eight runs with transcripts, about 70% fewer. Turn-start invocations, requests per turn and tokens per turn did not move. A resume costs 6.2K to 7.0K new prefill tokens on its first request, against 1.3K to 1.8K for a request inside an invocation. Re-cutting each recorded turn at the run's own threshold reproduces its resume count exactly: 546, 587 and 523 in the three 2,048 runs and 185 in the one 8,192 run with usage logs. These are counts over clean turns, 99% of all turns. The replay is a deterministic rule checked against itself. Across thresholds it misses: the 2,048 traces re-cut at 8,192 give 135, 144 and 154 resumes, and the 8,192 run has 185, which is 20% to 37% more. The 8,192 trace re-cut at 2,048 gives 625, which is 6% to 20% more than the 523 to 587 observed.

For each 2,048 run the replay also predicts the new prefill that the larger threshold would avoid: 1.82M, 2.20M and 1.90M tokens, against observed gaps to the first 8,192 run of 3.69M, 3.85M and 1.51M. The first two controls lack the reply-cache fix, so their gaps include another difference; the third is on the same code line. The freed prefill is at most 2.3% of the window by arithmetic and 0.6% to 0.8% by the server's prefill timers (Appendix G.1).

Discontinuity at the threshold. A request that lands just under or just over the threshold differs mainly in whether it was interrupted, because the reply length near the threshold is chance (Table 13). The next request acts more often after a resume at both thresholds. Removing resumes removes that nudge, so this does not explain the gain. Power is low, and the 8,192 rows rest on one run.

Table 13: Jump in the probability that the next request acts at the yield threshold, local-linear, 95% bootstrap over turns. The 2,048 rows pool three runs, the 8,192 rows use one.

Threshold	Bandwidth	Yield / continue (turns)	Acts, yield and continue	Jump
2,048	400	348 / 423	0.385, 0.345	$+0.113$ (-0.027 to $+0.224$)
2,048	700	569 / 745	0.376, 0.349	$+0.081$ (-0.021 to $+0.188$)
2,048	1,200	814 / 1,326	0.380, 0.360	$+0.046$ (-0.032 to $+0.121$)
8,192	400	34 / 35	0.412, 0.143	$+0.396$ (-0.026 to $+0.863$)
8,192	700	48 / 60	0.333, 0.183	$+0.392$ ($+0.140$ to $+0.668$)
8,192	1,200	71 / 103	0.366, 0.204	$+0.196$ (-0.023 to $+0.424$)

Per-token clearance. Within game and level, the Mantel-Haenszel ratio of clearance per generated token is 1.073 for the nine against the 21 controls (run-cluster bootstrap CI 1.029 to 1.138, one-sided permutation $p = 0.032$), 1.192 for the first two ($p = 0.052$) and 1.071 for the later seven (CI 1.030 to 1.147, $p = 0.082$). Scoring each run against the controls' rate per token instead gives 1.070 for the first two and 1.001 for the later seven ($p = 0.17$). Censoring at the window end is informative, since the priority rule gives tokens to games that progress, so we read these ratios as indicative. They are strongest in the first two runs.

What else differs between the arms. A cell-by-cell diff of the notebooks (26 cells each) shows that the first two's control and the first 8,192 run differ in the yield line and two fail-closed checks that raise only on the scored path. The first 8,192 run and its replicate differ in no cell, and the run with no token yield differs from its parent in the yield line only. The Kaggle same-code control also carries a strict thinking budget, which tied on its own promotion test. No same-platform, same-code 2,048 control at full window exists: the control of the first two ran on a cloud VM and the first 8,192 run on Kaggle, though 13 Kaggle controls average 111.3 and eight cloud controls 111.8.

Choices made after seeing the data. We chose the control and treated sets (with eight and six as sensitivity rows), four further control subsets (Table 11), nine token horizons (Table 12), the token values of Sec. 5 and levels rather than score as the outcome. The score gap is $+6.9$ points of mean game score (56.65 against 49.72).

F Checks on the curve and the noise

Four checks bear on the curve of Sec. 4 and the noise of Sec. 3. They use development runs on the demonstration set only, and the curve was built after most of what they test. The tables call the ceiling of Sec. 4 the plateau, and a development run a replica.

Runs needed to see a serving gain. At the curve's elasticity of 0.31, +11% tokens is worth 4.2 of 124 levels. Seeing it takes about 32 runs per arm at the SD of eight identical runs made on one day (5.95) and 61 at the pairs' SD (8.25), both two-sided, as every detection number in the paper is (Table 14). Our promotion tests use two runs per arm.

Past serving steps. Table 15 sets twelve earlier steps that moved tokens per second against the curve. Fitted together they give an elasticity of 0.45 (95% interval 0.13 to 0.77 at the pooled SD of 8.25; 0.29 to 0.61 at 4.1), next to the curve's 0.31. The pruning step carries the fit: $+19.5$ levels at $+11.2\%$ tokens against 3.3 predicted. Without it the estimate is 0.02 (-0.48 to 0.53), and the ten steps after pruning lean below the curve at the shipped family's SD. One step's noise (8 to 12 levels) swamps the curve's prediction (4.5 levels at most), so the steps neither confirm nor refute the curve. We built it after all of them, and none of the 29 promotion-test records of this stack holds a level prediction from tokens.

Table 14: Runs per arm needed to see a token gain on the replica (80% power, two-sided 5% level, normal approximation, shown before rounding; round up for the stated power). The gain is the curve's prediction (elasticity 0.31 on levels) out of 124 levels. SD 5.95: eight byte-identical runs made on one day (six started together); SD 8.25: the pooled SD of the six identical pairs. Every detection number in the paper is two-sided at an identical-run SD.

Extra tokens	Gain (levels)	SD 5.95	SD 8.25
+5%	1.9	153.1	294.2
+11%	4.2	31.6	60.8
+26%	9.9	5.7	10.9

Table 15: Steps that moved tokens per second, each an arm against its control on the development replica (25 public games, not held out). Tokens: generated tokens, arm over control. Levels: control to arm, sums over 25 games (means where several runs exist). Curve: $0.31\times$ relative token change $\times$ control levels (Sec. 4). Runs: arm/control replicas; the control is every archived run of the preceding configuration. Output kept: yes, outputs given the prompt are unchanged; no, the model or its prompt changes; fix, a correctness repair; part, the refit is lossless but the pair also changes the prompt format. Steps share runs, and the curve was built after all of them. Fitted through the origin, the twelve serving steps give an elasticity of 0.45 (95% interval 0.13 to 0.77 at the pooled run SD of 8.25 levels, 0.29 to 0.61 at 4.1); without the pruning step, 0.02 (-0.48 to 0.53; -0.23 to 0.28). Twelve more pairs that change the agent or the scheduler are in the evidence file.

Step	Output kept	Runs	Tokens (%)	Levels	Δ levels	Curve
12 streams, no memory freed	yes	1/2	-15.6	94.5 to 90	-4.5	-4.5
REAP keep-448 experts, 12 streams	no	2/2	$+11.2$	94.5 to 114	$+19.5$	$+3.3$
14 streams, 70 Mamba slots	yes	2/2	$+11.6$	114 to 114	0.0	$+4.1$
Host-RAM KV tier at 14 streams	yes	1/2	-0.8	114 to 107	-7.0	-0.3
16 streams, host KV tier	yes	1/2	$+6.8$	114 to 115	$+1.0$	$+2.4$
Relaxed MTP acceptance (0.5)	no	1/2	$+6.4$	114 to 113	-1.0	$+2.2$
16 streams, host KV tier (D-prime stack)	yes	1/2	$+4.6$	112 to 104	-8.0	$+1.6$
QSA index fix, async M-RoPE upload	fix	1/2	$+3.3$	126 to 119	-7.0	$+1.3$
Image hashes off scheduler, toolfast, watchdog	yes	1/1	$+5.7$	119 to 128	$+9.0$	$+2.1$
Drafter refit (D3), prompt format	part	2/1	$+5.0$	128 to 125.5	-2.5	$+2.0$
Fourth draft step	yes	1/2	-2.5	125.5 to 120	-5.5	-1.0
32K draft vocabulary map	yes	1/2	$+1.0$	125.5 to 126	$+0.5$	$+0.4$
Yield at 8,192 tokens (the yield pair)	no	2/2	-0.2	111 to 126	$+15.0$	-0.1

Plateau against pool size. The plateau estimate rises as more runs are pooled, because a level that no pooled attempt cleared stays stuck and pooling unsticks some (Table 16). The share of it that a replica run earns is 94% at eight runs, 92% at ten and 82% to 85% when 21 to 34 runs are pooled, and extrapolates to 85% to 91%: about 90%. The 94% is the eight-run value, not an extrapolation. A bootstrap over the eight runs gives a simulated share of 94.7% (93.4% to 96.0%), so pool size moves it more than run sampling does. The elasticity stays at 0.31 to 0.33 from two to ten pooled runs, so the 4-level price does not move. The plateau used here is the expected score at unlimited budget with each level redrawn from its recorded attempts; counting every level that any attempt cleared would give 70.9 on the eight-run pool. On the older September stack the run share is 59% at 31 runs, so the near-plateau result is specific to our stack.

A noise model and the stuck mass. A model that clears every level independently at its recorded rate, under the replica's token budget, predicts run SDs of 5.2, 6.4 and 5.7 levels, against 3.9, 5.5 and 8.25 observed (Table 17); the standard error of each prediction is 0.3 to 0.8. It matches the stable groups and underpredicts the identical pairs by a factor 1.44 in SD (the chance of an SD this high under the model is 0.05). The gap is the pairs' common first-minus-second shift of 9.2 levels, which holds 62% of their sum of squares; around it the pairs' SD is 5.6 against the model's 5.7. Tokens per second and slot contention do not explain it. Completion explains 84% to 90% of within-game score variance on our replica, as the forum post reports (84.7%, by our reading of its definition [borro1980, 2026]), but the two largest games hold 27% to 32% of it, against 65% there. Of the points the nine

Table 16: Plateau P against the number k of pooled replica runs, and the share of it that a replica run earns. P is the expected mean game score if every level the pooled attempts can clear is cleared (Sec. 4), averaged over all subsets of k runs (or 200 random ones). $S(1)/P$: simulated score at the replica budget over P, in percent, on the same 110 games per seed. Run share: the pooled runs' own mean score over P, in percent, without the simulator. $k = 10$ adds two later runs that were not in the paper's pool. Limits: fits of a hyperbolic and an exponential form to the mean curves and transfers from the control and all-replica curves; the stress case does not scale the transfer by the pool's slower growth; the cap treats every level that some eight-run replica cleared as always clearable. Development replica (25 public games), not held out.

Pool	k	P	$S(1)/P$ (%)	Run share (%)
Eight-run pool, plus two later runs	1	56.8	96	100
	2	57.9	95	98
	4	59.1	95	96
	8	60.2	94	94
	10	61.4	94	92
Yield-2,048 controls (21 runs)	1	49.6	97	100
	2	51.0	95	98
	4	52.7	94	94
	8	54.8	92	90
	21	58.8	88	85
All 34 replicas	1	50.9	n/a	100
	4	54.6	n/a	93
	8	56.9	n/a	90
	21	60.4	n/a	84
	34	61.8	n/a	82
September stack (27B, 31 runs)	1	5.2	n/a	100
	4	5.9	n/a	87
	8	6.6	n/a	78
	31	8.8	n/a	59
Eight-run limit: five estimates	∞	62.1 to 66.4	92 to 94	85 to 91
		median 64.6	n/a	88
Eight-run limit: stress case	∞	68.6 to 69.4	n/a	82 to 83
Eight-run cap: no new level	∞	70.6	n/a	80

yield runs lose to unreached levels, 57% sit on levels that none of 30 runs cleared; the table splits the rest by tokens spent.

Table 17: A noise model and the stuck mass, on the development replica (25 public games, not held out). *Top:* SD across runs of total levels and of mean game score, observed and predicted from recorded level attempts. *Binary* clears every level independently at its recorded rate; *Budget* also plays the recorded costs on 14 shared streams in 7,200 s (SE: bootstrap, then jackknife, over runs). The yield pool is eight runs: seven of the nine yield runs compared in Sec. 5 and the run without a token yield. *Middle:* a level is never cleared when no attempt in the pool cleared it; points are the share of a game's 100 held by such levels, averaged over 25 games, as a share of the points the pool's own runs lose to unreached levels. *Bottom:* points per game lost to unreached levels, by where the run stalled, against all 30 runs.

		SD of total levels			SD of mean score	
Compared runs	Attempts	Observed (95% CI)	Binary	Budget (SE)	Obs.	Budget
Nine yield-8,192 runs	1,114	3.9 (2.6 to 7.5)	4.6	5.2 (0.4, 0.6)	2.7	3.1
21 yield-2,048 controls	1,862	5.5 (4.2 to 8.0)	6.4	6.4 (0.3, 0.8)	3.6	4.0
Six identical pairs, within pair	1,588	8.25 (5.3 to 18.2)	5.8	5.7 (0.3, 0.3)	5.0	3.8

Pool	Attempts	Levels never cleared (of 183)	Games	Points held	Share
Eight runs (seven at 8,192, one with no token yield)	1,114	35	13	26.1	64%
Twelve plain runs	1,862	40	13	29.0	57%
All 30 runs	3,969	31	12	23.2	50%

	Nine yield runs		21 controls	
Unreached points by where the run stalled	Points	Share	Points	Share
No run cleared the level	23.2	57%	23.2	48%
Stalled with at least the tokens of the longest clear of that level	9.9	24%	9.8	20%
Stalled with fewer tokens than that	7.4	18%	15.4	32%
All unreached levels	40.5	100%	48.3	100%

G Serving detail: draft steps, vocabulary, the index-key fix and the time ledger

Everything here is serving: it changes tokens per window, not which tokens the model would choose, so it is priced by the curve of Sec. 4 and gated on counters (Sec. 3). Our serving levers rest on speculative sampling [Leviathan et al., 2022, Chen et al., 2023], whose acceptance under sampling is capped by the target's top probability [Li et al., 2026]. Table 18 lists the serving levers with their counters. Development runs, temperature 0.7, 14 running requests unless stated; step times are decode-line arithmetic, not kernel timers.

Table 18: Serving levers on development runs at temperature 0.7 and 14 running requests (accept length at 8 or more for the refit), with 95% intervals. The refit pair also changes the prompt format; a probe pair without it agrees. The fourth step is one run against two controls. Host-path changes are bundled, one run each. No levels are reported (Sec. 3).

Lever	Counter	Result	Verdict
Drafter refit [ursuciprian, 2026, albucino, 2026]	accept length; decode rate; step	2.699 to 2.830 (+4.8%, +0.130 [0.108, 0.151]); +50.5 tok/s [39.7, 61.2] (+5.5%); step +0.18 ms [−0.04, 0.36]	Shipped
Fourth draft step	accept length; step; decode rate	+9.8%; +4.93 ms [4.76, 5.12] (+13.5%); −40.9 tok/s [−49.1, −32.3] (−4.2%)	Not shipped
Image hashing, toolfast	prefill-side GPU time (log lines); host gaps	1,217 s before, 1,005 to 1,058 s in three later runs (direct timers 1,188 before, 1,082 to 1,119); 3,410 before, 344 to 563 of 100,800 stream-seconds	Shipped
Own-traffic draft vocabulary	decode rate; step; accept length	served map misses 1.416% of generated tokens; own 32K map: decode +1.2% at 14 running (predicted +1.4% to +1.7%), step 36.6 to 36.0 ms, accept +0.3%; one run against one control	Gate passed; later pinned back
BF16 draft KV	accept length	no change at probe scale; 8% of the KV pool	Not shipped

Verify rule. Under sampling the server accepts a drafted token with the probability the target gives it, which caps acceptance at the target's maximum probability: Bebop's Proposition 1 [Li et al., 2026]. The paragraph on the sampling cap below measures the cap on our traffic. We read the rule in the shipped wheel and the kernel on SGLang main; the shipped binary was not inspected. We draft a chain, and trees lift the cap [Chen et al., 2024].

Break-even. Let A_k be the probability that the first k drafts are accepted, $L = 1 + \sum_{k \le K} A_k$ the tokens per request per verify step, τ the step time and $\Delta\tau$ what one more draft adds. That draft pays exactly when $A_{K+1} > L\,\Delta\tau/\tau$, the marginal form of the walltime result of Leviathan et al. (Theorem 3.8) with a step that grows with K [Leviathan et al., 2022]. Cascade, Amankulov and Mamatin, TurboSpec and MagicDec give variants for expert cost and load [Saxena et al., 2025, Amankulov and Mamatin, 2026, Liu et al., 2024, Sadhukhan et al., 2024].

The fourth draft step. At 14 running requests three drafts give $L = 2.814$ and $\tau = 36.6$ ms. A fourth adds $A_4 = 0.275$ [0.246, 0.303] to L (the probability that all four drafts are accepted) and 4.93 ms [4.76, 5.12] to τ. Break-even is $2.814 \times 4.93/36.6 = 0.379$; we had expected A_4 of 0.35 to 0.40 and a step about 10% longer. Given three accepted drafts the fourth is accepted with probability 0.61, against 0.77 at positions 1 to 3. It would need 0.838, above the first position's 0.773, so no chain whose conditional acceptance does not rise with position can pay here. The 0.275 is a difference of two runs that assumes positions 1 to 3 are unchanged, and A_1 to A_3 are counts from 5 of 25 games. Five verify rows are also the limit of the sparse-attention ring as patched [lovedheart, 2026], so a build effect at position 4 is not excluded. A per-position count above 0.379 would reverse the verdict.

The sampling cap, measured. On the kernel of the paired decision test we logged the top-20 log-probabilities of 30 full-history replies (33,364 generated positions). The mean post-temperature, top-p-cut maximum token probability is $c = 0.912$ (per reply 0.857 to 0.974). A one-hot chain drafter of three steps is then capped near $1 + c + c^2 + c^3 = 3.50$ accepted tokens per step (4.19 at four steps), against 2.830 live, and our first-position acceptance of 0.773 is 85% of c. The fourth position's measured 0.275 sits far below $c^4 = 0.692$, so the four-step loss is the drafter's, not the sampler's. The cap assumes independent positions and rests on 30 replies capped at 2,048 tokens; the idea of the cap is Bebop's [Li et al., 2026].

The own-traffic draft vocabulary. The served 64K draft vocabulary misses 1.416% of the tokens our model generates (Table 19). We built a 32,768-id map from our own transcripts and gated it on one 25-game run against the shipped configuration, under bars declared before the push: decode at least $\times 1.008$ at 14 running requests, step at most 36.45 ms, accept length at least 2.827, levels at least 105. It passed: decode 987.1 against 975.3 tokens per second ($+1.2\%$; an identical-stack reference read 973.3), clean step 36.0 against 36.6 ms, accept length 2.8385 against 2.8300, 126 levels against 129, every number recomputed from the server log. The measured gain is below the predicted $+1.4\%$ to $+1.7\%$. The map should leave the output distribution unchanged (speculative sampling preserves it when the verify path is correct), and its probe installed and served cleanly (READY after 550 s, 0 tracebacks).

Everything is at temperature 0.7 on development replica runs unless stated. Step times are decode-line arithmetic on intervals without prefill, not kernel timers.

Step time. Over 8,132 clean intervals from six runs with three drafts, at 8 to 14 running requests R, the step is $\tau = 20.92$ [20.29, 21.45] $+ 1.120$ [1.082, 1.168] R ms (300 s block bootstrap). With context it is $18.21 + 1.163R + 0.280\,[0.233, 0.307]$ ms per 10K tokens of mean context per request, so the step is not flat: at $R = 14$ the median rises from 34.4 ms under 20K to 36.8 ms at 80K to 100K (+7%). A fourth step adds 3.58, 3.59, 4.61 and 4.93 ms at $R = 11$ to 14 (11 to 677 four-step intervals).

Where it would pay. The four-draft run covers $R = 11$ to 14, so any crossing below 11 depends on how the added verify row is costed. We tried three cost models, each fitted to the four measured gaps: proportional to R (0.353 ms per request), affine ($-2.16 + 0.507R$ ms) and expert coverage plus 0.12 ms per request. With $A_4 = 0.275$ they cross the needed gain $L\,\Delta\tau/\tau$ of the break-even rule above at R of about 3, 8 and 11, so we do not say where the fourth step pays. Under the

proportional model the needed gain is 0.266, 0.309, 0.347 and 0.380 at $R = 8$, 10, 12 and 14, and the geometric 0.350 crosses it at $R \approx 12$. The coverage model, fitted on three drafts, predicts $+2.6$ ms for the fourth step at $R = 14$ against 4.9 measured. Expert bytes alone give 2.2 to 2.3 ms (0.076 more of the 52.8 GB of expert weights, read at 1.79 TB/s). We did not attribute the rest, which is row-proportional work.

Scope of the fourth-step result. It is one run against two controls at one operating point: 14 streams, temperature 0.7, top-k 20, top-p 0.95, a chain of drafts. We ran no two-draft arm and no policy that sets the draft length per request. A draft tree lifts the sampling cap [Chen et al., 2024]; we drafted a chain. The target is the pruned (448 of 512 experts), 4-bit-weight model, and the refit drafter was trained against a different 4-bit target [ursuciprian, 2026]. The vendor reports an accept length of 4.06 to 4.07 at four draft steps on benchmark text [Qiu et al., 2026], and ours is 2.83 at three on agent traffic under sampling. We cannot say whether temperature, text, pruning or quantisation accounts for the gap.

Drafter refit. ursuciprian's KL refit of the drafter's dense tensors (29 in albucino's file, 24 in the card), which we transplant at boot [ursuciprian, 2026, albucino, 2026], raises accept length from 2.699 to 2.830 ($+0.130$ [0.108, 0.151], +4.8%) and decode by 50.5 tokens/s [39.7, 61.2] (+5.5%) at an unchanged step ($+0.18$ ms [-0.04, 0.36]). The replica pair also carries a prompt-format change; a probe pair that isolates the refit agrees ($+0.133$ [0.076, 0.181]). Fine-tuning an MTP head on self-distilled data is the recipe of FastMTP [Cai et al., 2025].

Draft vocabulary. The served 64K draft vocabulary misses 1.416% of 41,560,633 generated tokens. Table 19 compares it with maps rebuilt from our transcripts. Runs outside the build set replay the same 25 games, so only the leave-one-game-out rebuild leaves the scored game out of its map. Tokens are re-tokenized transcript text, a proxy for the sampled ids.

Table 19: Generated tokens missing from the draft vocabulary (lower is better). The build set is seven development runs; the three further runs play the same 25 games. Own maps are built from our traffic at 32K and 64K entries.

Set	Tokens	Served 64K	Own 32K	Own 64K
Build set, pooled (7 runs)	41,560,633	1.416%	in-sample	in-sample
per run	5.3M to 6.4M	1.32% to 1.51%	in-sample	in-sample
Outside the build set, run 1	6,261,780	1.407%	0.047%	0.014%
Outside the build set, run 2	5,618,162	1.468%	0.031%	0.010%
Outside the build set, run 3	5,295,450	1.305%	0.073%	0.014%
Leave-one-game-out, 25 games (better than served on 25 of 25)	pooled	n/a	0.106%	0.038%

Three spellings of "Hmm" (capitalised, capitalised after a space, lower case after a space) are 0.304%, 0.218% and 0.123% of tokens: 0.646% in all and 46% of the served map's misses. FR-Spec truncates the draft vocabulary by token frequency [Zhao et al., 2025]; FastMTP applies the idea to an MTP head [Cai et al., 2025], and a vocabulary-trimming study fits coverage to workload responses [Ben Shoham, 2026]. The nearest work to our rebuild from the target's own samples is VocabTrim (Goel et al., arXiv 2506.22694). We estimate the acceptance gain at 0.4% to 1.0% of accept length (hit rates of 0.18 to 0.50 on the recovered tokens) and claim none. A 1.4% miss caps first-position acceptance at 98.6%, far above the 77% measured.

BF16 draft KV. A probe pair with the target pool capped at 1,375,552 tokens gives accept length $+0.010$ [-0.040, 0.062] at 8 or more running requests and $+0.027$ [-0.010, 0.069] at 14; a null pair reads -0.012 [-0.053, 0.040]. Both probes of the pair carry the 32K vocabulary map, and probe contexts stop near 40K. The cache costs 8% of the KV pool.

The QSA ring replay and the invalid probe. The fix is lovedheart's [lovedheart, 2026]; we replayed it on our wheel in an offline test (CPU torch, the wheel's index builders, no CUDA kernels). For 4-token verify windows after prefixes of 1 to 256 tokens, the unpatched stride corrupts 192 of 256 completed groups and the patched one 0; one-token decode is clean. That is 3/4, the share of window alignments that start mid-group. We measured no corruption rate on traffic. lovedheart's

commit reports the same kind of sweep: 90 of 118 groups corrupted, 0 after. After the patch, windows up to 5 rows are clean and 6 wrap (64 of 384), so four draft steps sit at the limit, a build effect at position 4 cannot be excluded, and five would be unsafe. Their guard rejects drafts longer than twice the ratio; ours is tighter, at the ratio plus one, because our replay wraps at six rows. The GPU probe was invalid by its own predeclared rule: 1 of 16 prompts (unpatched) and 2 of 16 (patched) reused the decode-time keys, and the rule required 12. Median divergence between two fresh runs of the same prompt is 16 tokens on the unpatched boot and 4 on the patched one, so the noise floors differ by boot. Output that varies with batch composition through floating-point reductions is a known property of LLM serving [He and Thinking Machines Lab, 2025, Yuan et al., 2025]. The effect on the GPU is unmeasured. Accept length is unchanged by the fix together with the upload fix (2.697 and 2.698 before, 2.704 after).

Greedy against sampled. On a greedy boot of the original drafter (temperature 0, one request, 16 recorded prompts of about 4.9K tokens) accept length is 3.221 (SE 0.05), 0.857 per position if geometric. At temperature 0.7 it is 2.59 to 2.61 across runs and probes, 0.713 to 0.720 per position, with 28% fewer accepted drafts per step. The two sides ran different text, so this indicates the size of the tax without matching it. The headroom is now measured on the same kernel as the paired decision test (above).

Per-position counts. Table 20 comes from the own-traffic draft-vocabulary run (5 of 25 games, 479 replies, 374,855 verify steps), whose gate was open in v2 and has since passed; we use it for counts only.

Table 20: Probability A_k that at least k drafts are accepted, and the conditional $a_k = A_k/A_{k-1}$. Rows 1 to 3 are counts from the own-traffic draft-vocabulary run (95% request bootstrap). Row 4 is the difference of two runs at 14 running requests, assuming positions 1 to 3 unchanged. Break-even for row 4 is 0.379, that is $a_4^* = 0.838$.

k	A_k [95% CI]	a_k	Geometric A_k ($a = 0.769$)
1	0.7734 [0.7687, 0.7780]	0.773	0.769
2	0.5917 [0.5844, 0.5993]	0.765	0.591
3	0.4521 [0.4432, 0.4608]	0.764	0.455
4	0.275 [0.246, 0.303]	0.608	0.350

The fourth-step gain is stable across context (+0.244 to +0.298 over five bins of 0 to 100K), so it is not a context mix.

G.1 Time ledger

Read the totals. Table 21 splits the 7,200 s window of six single development runs by log-line arithmetic. Prefill-side GPU time (new-token prefill, batch setup and scheduler stall together) is 1,287 s in a 2,048-yield control and 1,217 s in an 8,192 run before the later changes. It is 1,005, 1,026 and 1,058 s in three runs that carry them. The server's own prefill timers agree in direction: 1,223 and 1,188 s against 1,082, 1,099 and 1,119 s. The fall is 159 to 212 s against the 8,192 run and 229 to 282 s against the control by the log lines, and 69 to 106 s and 104 to 141 s by the timers. The later changes include the host-path changes: the image-hash flag and incremental tool state. They are bundled, so the ledger cannot say which one moved prefill-side time. Only run 3 is the shipped set: run 1 lacks the drafter refit and run 2 adds one overlay we did not ship.

Method. The server [Zheng et al., 2023] logs a decode line every 40 forwards with 1 s stamps, so head, intervals and tail sum to 7,200 s exactly. Decode time in an interval is 40 times the step, fitted on clean intervals (no prefill line, equal running count at both ends; $17.9 + 1.19R + 0.27$ ms per 10K tokens of context for run 3; 27% of the window is clean). The rest is extra time. Where a prefill line is present, least squares splits it into a cost per prefill batch (setup), per new token and per cached token (stall), with a 300 s block bootstrap; elsewhere it is summed as measured. Stream time (Table 22) uses the scheduler's per-request timers over $14 \times 7{,}200 = 100{,}800$ stream-seconds.

New-token cost is stable (70 to 84 ms per 1K tokens). Setup and stall trade off in the fit, so their sum is steadier than either: 273, 259 and 336 s in runs 1 to 3 and 549 s in the control. The stall term is a coefficient per cached token; that image hashing causes it rests on the flag that removes it.

Table 21: GPU wall time in seconds over the 7,200 s window (decode share in percent), and the fitted costs. Stall intervals are 95% block-bootstrap. Ctrl: yield 2,048, before the later changes. A: yield 8,192, before them. B: Ctrl's code apart from the yield. Buckets sum to 7,200 by construction. Single runs; no kernel timers.

	Run 1	Run 2	Run 3	Ctrl	A	B
Decode steps	6,161	6,170	6,146	5,867	5,951	5,933
Decode share (%)	85.6	85.7	85.4	81.5	82.6	82.4
New-token prefill	732	766	722	739	825	686
Prefill batch setup	171	120	298	242	9	129
Scheduler stall	102	139	38	306	382	420
95% interval	23 to 192	43 to 267	−56 to 107	255 to 367	300 to 455	332 to 526
Prefill-side total (three rows above)	1,005	1,026	1,058	1,287	1,217	1,234
Direct prefill timers	1,082	1,099	1,119	1,223	1,188	1,179
Extra, no-prefill intervals	21	0	−22	42	19	31
Fit residual	−4	−8	6	−11	0	−11
Head and tail	17	12	12	15	13	13
Per 1K new tokens (ms)	74.6	76.4	69.9	71.9	84.0	76.5
Per prefill batch (ms)	47.7	33.2	81.9	77.7	2.8	39.9
Per 10K cached tokens (ms)	5.0	6.9	1.9	18.6	22.0	22.9

Table 22: Stream time as a share of 100,800 stream-seconds (14 streams). The "waiting" row assumes prefill and decode batches do not overlap on the GPU. The two last rows split the "no request" row.

Share of stream time (%)	Run 1	Run 2	Run 3	Ctrl	A	B
GPU decoding	79.9	80.2	79.9	73.9	73.3	73.2
Waiting for non-decode GPU work	13.0	13.0	13.2	16.4	14.9	15.2
Own prefill	1.1	1.1	1.1	1.2	1.2	1.2
Queued in the server	0.4	0.4	0.4	0.5	0.6	0.4
No request on the server	5.7	5.3	5.4	7.9	10.0	10.0
in-batch host gaps (measured)	0.6	0.5	0.3	2.1	3.4	3.0
not logged	5.2	4.9	5.1	5.9	6.6	7.0

Checks that do not share the construction. Direct prefill timers sit within 8% of the log-line prefill seconds, tokens within 0.3%, mean running requests within 1.1% and the step model on intervals it was not fitted on within 1.4%. The decode bucket carries about 50 s (1 SD) of calibration noise. The server held a request in all but about 1 s of every window, and intervals without a prefill line hold -22 to $+42$ s of extra time against a timestamp noise of about 20 s, so idle GPU time cannot be told from model error below about 40 s.

Freed prefill. Direct prefill seconds are 43 to 58 s lower (0.6% to 0.8% of the window) in the 8,192 pair than in the 2,048 controls, against 163 s (2.3%) by arithmetic at the all-in 130 ms per 1K tokens (Sec. 5). The two cloud controls differ from each other by 109 s, so the pair difference sits inside the spread between controls. At 11.2 levels per million tokens the timer value is worth at most 0.4 to 0.5 levels (inference).

Where the +11% shows up. From the 8,192 run before the later changes (A) to run 3, tokens per second on the window clock rise by a factor 1.108. Decode share contributes 1.033, accept length 1.046, running requests 1.055 and the step cost per running request 0.972. The factors multiply by construction, so they locate the gain without attributing it.

Unobserved. The ledger gives no kernel-level split of a decode step, which is one number, and none exists for the four-draft run. The 5.1% of stream time that no log accounts for (about 5,100 stream-seconds in run 3, with no request on the server) has no timer. Tool execution, prompt assembly, image encoding and hand-over between games are candidates that no log separates. It fell by about 1,600 stream-seconds from A to run 3 together with the measured gaps, so the later changes did not act only on the gaps we can see. At 14 streams the decode ceiling is 1,083 tokens per second (14 running requests times accept length 2.830 over the 36.6 ms step), and run 3 reaches 892 on the window clock. With this step and accept length, zero non-decode time and 14 running requests at every step would add at most 21% tokens, about 8 levels at the curve's elasticity (inference).

H Keepalive and silent failures

A scored run returns a score and no log, so a run that dies can bank a completed 0.00 that looks like a result. This appendix lists the paths we found, the guards we built and the zeros we could not explain.

Two paths that bank a dead run as 0.00. The framework's `Benchmark.run` catches any solver exception, marks the games still playing as crashed and returns normally (`benchmark.py`, lines 90 and 178 to 190, in the vendored bundle under `solution/sim/fixtures/franzen-bundle`). The notebook's wait for the server to finish loading (READY), on a dead server, prints the log tail, breaks out and plays the whole run against nothing (`build_franzen_arms.py`, line 518, docstring 521 to 529). `failclosed_server()` raises when the server exits before READY; `failclosed()` raises at the end of a run that cleared fewer than 20 levels. Both act only on the scored path. An errored submission does not spend the daily slot: two errors on 2026-09-30 were followed by a completed run the same UTC day. The retry applies to errored submissions, which Kaggle does not count against the slot. We read the offline test (`test_failclosed.py`) and did not rerun it. A run that dies after 20 levels, or that Kaggle kills from outside, is not caught, and neither check has fired on a scored run.

The midnight zeros. Both were submitted in the 00:00 to 00:20Z window: the unchanged base notebook on 2026-10-03 and, on 2026-10-10, the configuration that scored 27.15 the day before. A dead draw shows as complete, 0.00 and a 3,411-byte submission file; live draws are 3,810 to 3,907 bytes. Table 23 counts early endings (finished inside 60 minutes) among all teams in that window.

Table 23: Early endings among runs submitted 00:00 to 00:20Z, all teams. The base rate covers the 27 days other than 2026-10-03 and 2026-10-10. A row appears only when a run completes, so each denominator counts completed runs.

Day	Early / completed	Note
Saturday 2026-09-12	6 / 55	
Saturday 2026-09-19	3 / 57	
Saturday 2026-09-26	5 / 56	
Friday 2026-10-09	13 / 100	$p = 0.035$ against the base rate
Saturday 2026-10-03	48 / 61	our 0.00
Saturday 2026-10-10	63 / 64	our 0.00
27 other days	140 / 1,880	7.4% (daily 3% to 16%)

Two of five Saturdays were events, so the weekday alone is not the trigger, and the cause is unknown. Of our 11 fires in the window since 2026-09-24 two banked 0.00, and none of the 9 later fires did; we moved submissions to 02:06Z. The waves continued: on Saturday 2026-10-17, 73 of 118 runs submitted 00:00 to 00:59Z ended early (68 of them 23 to 34 minutes in), against 8 to 18 of 140 to 148 on 2026-10-13 to 2026-10-16, the third Saturday in a row; our 02:06Z fire drew 33.05 that day. These field counts come from a research pass over a third-party monitor and were not recomputed here.

CUDA out-of-memory after READY. Of 34 full-length replica runs (68.6 play-hours), one lost its server to `torch.OutOfMemoryError` with 91.06 MiB free, 1 h 54 min into play and 5.7 minutes before the window ended; its 115 levels include a dead server for that tail. A second error came seven seconds after play began, in a run that deliberately added memory (BF16 draft KV); it is outside the rate. One event is 2.9% per two-hour run, with an exact 97.5% upper bound of 15.3%. A constant hazard, which is an assumption, extrapolates to 12.3% for the nine-hour run. Failures cannot show that the memory fraction of 0.955 works: with none in 10.1 post-change hours, the chance of seeing none at the earlier hazard is 0.86. The warning can (Table 24).

Watchdog. A daemon thread relaunches a dead server (at most three times), reuses the model precache and retries requests in place for up to 30 minutes. In the rehearsal we killed the server 60 s after READY and gave each game 2,700 s, so games outlived the relaunch. The log shows "relaunch 1 of 3" and "READY again after 346s". No game gave up, and the run finished 47 levels, 1,810 of its 1,862 actions coming after the relaunch. The run of the shipped set logged 0 relaunches, so behaviour in a scored run is unknown.

Table 24: Low-free-memory warnings after READY, before and after the memory fraction was set to 0.955. Lowest free memory is the smallest value logged in any run of the group.

Group	Runs	With warning	Lowest free (GiB)	OOM
Before 0.955	29	23	0.15	1
0.955, no added memory	4	0	n/a	0
0.955 with a fourth draft step	1	1 (8 warnings)	0.66	0

The 2026-10-18 zero. The third fork-era zero is also complete, 0.00 and 3,411 bytes. The machine that runs our scheduler missed the 02:06Z and 04:06Z fires (its log jumps from the 00:06Z run to 06:53Z), so the draw was submitted at 06:56Z, inside a field cluster of early endings (05:00 to 07:59Z: 18 of 101 runs short on 2026-10-18 and 15 of 93 on 2026-10-17, against 2 to 3 of 69 to 88 on 2026-10-13 to 2026-10-16). Its cause is unknown. The idle rule below is a candidate, and an early outside kill fits the bytes as well.

The 15-minute idle rule. On 2026-10-17 Jeroen Cottaar wrote in the competition forum that a run with no move for 15 minutes ends with its score so far, and that a thread that resets one game every 10 minutes until startup avoids it [Cottaar, 2026a]. It is a competitor's statement; the hosts have not confirmed it. Our configuration waited up to 12 minutes for the server, then sent one warmup RESET, so a server not serving by about 27 minutes would bank 0.00.

The keepalive. After the warmup RESET a daemon thread re-sends RESET on the warmup game only, every 9 minutes while the server is not ready, at most 5 times; it stops at READY, the first reply or the game's first action, logs and never raises. An offline test on the organisers' localhost competition gateway, with a scaled clock and the real agent, reproduces the failure and its fix: without the thread the scorecard closed 15.05 minutes after the warmup RESET; with it the RESETs came at 9.05, 18.05, 27.05, 36.06 and 45.06 minutes and the card stayed open to 60.12 minutes; a late READY played every game; an injected RESET failure was logged and play continued. On a Kaggle probe the thread stopped after 0 RESETs because the server was ready within 9 minutes, so the live RESET path has not fired on Kaggle. We pinned it on 2026-10-20, after the 40.98 draw. It is a guard, not a lever: its expected value is about 0 to the final score, and it converts a slow-boot day from a banked 0.00 into lost play time.

I Fine-tuning protocols: the rejection-sampling run and GRPO round A

I.1 The rejection-sampling run

Data. The shipped agent, with a request logger and a RE-ARC-3 loader added for development only, played 127 RE-ARC-3 games with no game family from the demonstration set, at the development budget per game and the shipped sampling (temperature 0.7, top-p 0.95, top-k 20), over three Spot attempts on one rented RTX PRO 6000. It cleared 548 of 601 levels (91.2%) and won 106 games, against 67.8% of levels on the demonstration set. Each request chain was cut to the system prompt plus the latest suffix that fits 15,000 estimated tokens, giving 13,652 windows split by game family (10,810 train, 2,206 validation, 636 test); 99% are cut suffixes, while logged prompts at play time had medians of 21K to 53K tokens. A reply is a target when its level is one the run cleared, so every turn of a cleared level counts, deaths and RESETs included; 6,806 train windows (63.0%) carry a target. By efficiency, 17.0% of the 7,426 supervised train turns come from levels cleared at or under the human baseline's actions, 50.9% from levels at over twice the baseline and 20.4% at over four times.

What the adapter changed. Table 25 gives the counters that Sec. 7 reads.

Training. 1,400 random train windows from 100 games (1,507 supervised replies, 16.0M tokens) trained a LoRA of rank 16 and alpha 32 (merge scale 2.0) on the Gated DeltaNet projections, attention and the shared expert, with routed experts frozen; weighted token negative log-likelihood at temperature 1 over the whole reply; AdamW at 10^{-4}, cosine to 10^{-5}, 350 steps, one pass. It ran at 977 tokens per second with a peak of 78.4 GiB: 4.55 h of training time, 4.78 h wall, $7.89 at the

Table 25: What the adapter changed on the demonstration set, by strength (25 games; one run is one development run). Family: the yield-8,192 family without an adapter (seven configurations, two of them later rejected), range across nine runs for levels, actions and RESETs (mean in brackets), eight with transcripts for the other counters, and two runs for tokens per second. Against all 18 family runs on Kaggle and Vertex, levels run from 115 to 132 (mean 124.4); two later same-card controls of the adapter's base arm on Vertex AI cleared 132 and 130. Style counters are on cleared levels. RE-ARC-3: the shipped agent's own training-data play, read from the project's records and not recomputed. Rates are comparable across columns; whole-run counts are not comparable with RE-ARC-3, which has more games.

Counter (per run unless a rate)	Family, no adapter	Half	Full (two runs)	RE-ARC-3 play
Levels cleared (of 183)	119 to 129 (124.0)	117	109, 115	91% of levels
Actions	6,108 to 7,082 (6,511)	7,375	7,859, 7,927	n/a
RESET actions	52 to 87 (65)	100	129, 138	n/a
Deaths (GAME OVER echoes)	8 to 13	10	26, 27	n/a
Reasoning characters per action	2,021 to 2,346	1,979	1,729, 1,772	966, 1,347
Actions per reply	2.35 to 2.67	2.67	2.69, 2.75	4.52, 4.28
NO-OP share of actions	0.042 to 0.058	0.058	0.073, 0.081	0.032 to 0.037
Tokens per reply	2,097 to 2,277	2,152	2,031, 1,962	1,783, 2,152
Harness tokens per second	885.5, 894.9	930.2	916.8, 917.2	n/a

Spot list price. Validation loss on 32 fixed windows from 15 held-out families fell from 0.7853 to 0.6759 (-13.9%), 81% of the drop by step 40; two step-0 readings of the same base differ by 0.29%. The merged adapter as served matched the trained adapter (mean absolute log-probability difference 0.029 on 30 requests, argmax agreement 97.8%) under a parity gate revised before training because two collects of one server differ by 0.040.

Runs and decision. Two development runs at merge scale 2.0 cleared 109 and 115 levels and one at 1.0 cleared 117, all on the rented VM with the shipped configuration plus the draft vocabulary of Appendix G and the adapter folded in at load. On 2026-11-03 two runs of the same base arm without the adapter, on the same RTX PRO 6000 as Vertex AI jobs, cleared 132 and 130 levels (one restarted after a preemption); among the five runs the adapter's three are the lowest, exact one-sided rank p 0.10. They were not interleaved with the adapter runs, which ran on the VM 15 days earlier, so service and date are not separated from the adapter. The project's stated rule, written in its progress notes and not in a timestamped predeclaration file, was 130 levels in each full-strength run to proceed, and at half strength 120 or fewer to stop; all three fall in the stop region. The loss is concentrated: per game the full-strength mean is up on 6 games, down on 12 and tied on 7 (two-sided sign test $p = 0.24$). On 20 RE-ARC-3 prompts the mean completion length rose but the median fell, and one 5,663-token reply carries the rise, so "longer outputs" is not a finding; greedy-token confidence on 30 prompts is flat. The rented-GPU time for the line was about 13.5 h of play data (about $17, estimated), the 4.78 h of training and at most 9.6 h of development runs; runs took 2.13 to 2.14 h each.

I.2 GRPO round A: protocol and amendments

The recipe and gates were written into `solution/lora/grpo/GATES.md` at 07:50Z on 2026-10-20, before any round-A window, adapter or gate output existed; the data record followed at 17:40Z, two amendments at about 17:52Z and 17:56Z, and a family amendment at 19:49Z, before the second and third members trained.

Recipe. Reward per play and level is 1 if the level is cleared within a budget like the hidden run's, minus 0.1 per death up to five. Groups are the plays of one game that reach the level; the advantage is the reward minus the group mean, times $2i/(n+1)$ for level i of n, with no supervision where its size is under 0.05. The loss is the token mean over 16 windows of $-\mathrm{sg}(\rho)\, A\, m\, \log \pi + 0.01\, k_3$, where ρ is the ratio to the base, m masks tokens with ρ outside $[0.8, 1.28]$ or outside either top-k/top-p keep set, and $\log \pi$ uses the logits at temperature 0.7 renormalised over the server's keep set. AdamW at 3×10^{-5}, constant after 3 warmup steps; LoRA rank 16, alpha 16 (scale 1.0) on attention, Gated DeltaNet and the shared expert; one epoch is 57 steps. Stops at each validation: val k_3 over 0.02 or entropy more than 10% from step 0; null below 0.002.

Data. The reference member's 900 train windows come from 264 plays: 200 RE-ARC-3 rollouts and 64 plays by eight concurrent development runs of the shipped agent on the eight games of the demonstration set's training half; 27% are RE-ARC-3 windows, the family both rejection-sampling adapters drifted toward. The third member's 882 windows come only from those 64 plays, and its 25 validation windows are byte-identical to the reference member's.

The defect and the amendments. At step 0 the adapter is zero, so the reading is base against base; the reference member's first start read raw val k_3 12.909 on 66,949 validation tokens, which one token moving 13.7 nats explains. Raw step-0 readings in the other base-against-base comparisons were 0.1357, 0.1238, 0.1267 and 23.14. Across ten pairs of five independent base passes (three VM processes, two Vertex hosts), 12.1% to 12.4% of tokens match exactly, 16.9% to 17.1% leave the trust-region band, the RMS gap is 0.365 to 0.373 nats, and raw k_3 spans 0.099 to 1,525 while clipped k_3 is 0.0629 to 0.0658 (Figure 14). At step 1 a gradient pass and a no-gradient pass of the same zero adapter, a different comparison from the pairs above, mask 16.3% to 16.6% of tokens: 4.95% outside a keep set and 11.3% to 11.7% by the band, counted after the keep-set misses. Amendment 1 clamped each token's log ratio to $[-2, 2]$ in the val k_3; amendment 2 read both stops and the null rule net of the run's own step-0 clipped reading and dropped amendment 1's null rule. The loss keeps the raw k_3. The floor's SD is 40% of the 0.002 null margin, so the null rule sits near its noise.

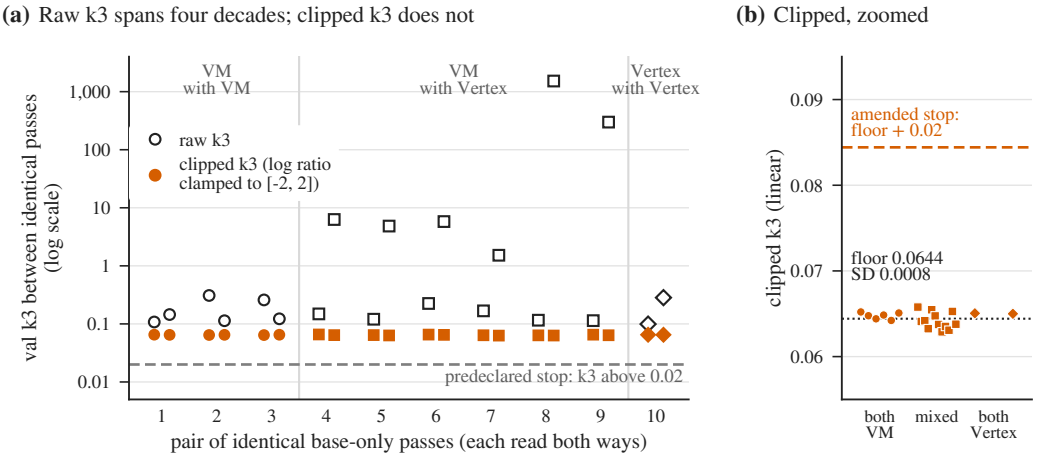


Figure 14: The training forward pass is not deterministic. (a) Validation k_3 between pairs of identical base-only passes over the same 66,949 tokens, each pair read both ways, on a log scale: raw (open) spans four decades, clipped to ± 2 nats per token (filled) does not; the predeclared stop at 0.02 is dashed. Shape: VM with VM, VM with Vertex, Vertex with Vertex. (b) The clipped values on a linear scale: floor 0.0644 (SD 0.0008) and the amended stop at the floor plus 0.02. Lower and flatter is better for a guard.

Readings. Table 26 gives the validation readings net of each run's floor. The reference member broke both stops at step 24 and kept step 16; the doubled-rate member broke both at step 16 and kept step 8; at equal learning rate times steps they read alike (V3-G11). Supervision mass was net negative on 17 of 24 steps of the reference member and 11 of 16 of the doubled-rate member, so failed plays carried more of each update; masked share rose as the adapter moved. Validation readings measure distance from the shipped agent's own play on held-out RE-ARC-3 windows, not quality, and the rise in val NLL equals k_1 on the same tokens.

Gates. Gate (a') compares the style of replies to 400 recorded states (reasoning length and action calls per reply) for the base, a known-bad control (the full-strength rejection-sampling adapter) and the member, against the base's own sample-to-sample band. Its first run failed to boot the member's server, an infrastructure fault; as interim information only, the known-bad control moves reasoning length to 0.831 of the base's, outside the base's band, so the gate can see that direction. Wave 1 runs every non-null member against shared controls in one concurrent batch, and wave 2 runs the best survivor fresh, promoting only if the pooled 12-against-12 difference on 17 bench games clears a bar (2.0 levels at three members), the wave-2 difference alone is not negative and no style veto fires.

Table 26: Round A validation readings, net of each run's step-0 floor (about 0.064). Entropy and val NLL are changes from step 0. Stops: net k_3 above 0.02 or entropy beyond 10%. One seed per member; 24 validation windows of held-out RE-ARC-3 play.

Member	Step	lr × steps	Net k_3	Entropy	Val NLL	Note
Reference (3×10^{-5})	8	2.4×10^{-4}	+0.0001	+1.6%	+0.75%	
	16	4.8×10^{-4}	+0.0116	+5.4%	+1.86%	kept
	24	7.2×10^{-4}	+0.0260	+11.1%	+3.92%	both stops
Doubled rate (6×10^{-5})	8	4.8×10^{-4}	+0.0136	+6.3%	+1.82%	kept
	16	9.6×10^{-4}	+0.0658	+25.0%	+8.24%	both stops
Public half only (3×10^{-5})	8	2.4×10^{-4}	+0.0022	−0.08%	−0.37%	running at writing

On a pool of 16 recorded runs this design promotes a family of null members with probability 0.038 (0.111 under a stress model) and a lone true +6 member 0.82 (0.69). Before any result the project put the chance that an adapter with a true +6 levels ships at 0.05 to 0.12. The round's spend on trainer smoke runs and members, at list prices, is in Appendix L.

J The harness program: changes, gates and runs

This appendix holds what Sec. 6 leaves out: each change as built, each predeclared gate, and every run read. The changes are overlays that the notebook builder applies as asserted string edits (`solution/kaggle/franzen_overlays/`: `unstick2.py`, `seg8.py`, `scroll.py`, `tool2.py`, `verify.py`); the gates are the files `solution/kaggle/generated/gate-predeclared-2026-10-30-*`, `-2026-10-31-*` and `-2026-11-01-*`, each written before the first run of its build. The per-run tables are rebuilt by `paper/v3/analysis/harness_read.py`, except the six rows of the tool-fix and replay builds, which come from the gate readers; the gate readers and record writers are in `solution/analysis/harness_2026-10/gates/`.

J.1 The changes as built

Stall check, coverage and step checking. Four parts. (1) A stall check at the third, fifth, seventh and later idle turn on a level asks the model to call `action()` before any long search, and says that a filler key press or a RESET wastes actions. (2) After a stall check, a game over or a model RESET on the level, the opener lists the object types (colour and bounding-box size, joined by corner contacts, dot textures left out) that no click on the level has touched, when a click is a valid action, and the valid actions unused on the level; engine action names (`ACTION1` to `ACTION7`) are mapped to the history's names first, and UNDO is dropped. (3) On the first opener of each level after the first, the opener lists the object types new on that level, same colour and box merged. (4) Tufa's step-verification addendum, off in Franzen's settings, is on: mechanics first and goal last, replay the level's recorded transitions through a simulator before a long plan, and retry a no-op from another state before ruling it out. On frames replayed through the real patched helpers, the coverage list costs about 6 ms per call and the new-type list about 175 ms per level start. The offline test feeds engine action names and fails on the first build.

Corner-joined regions. Every node of the segmentation gains `group`, the lowest node id among nodes of the same colour joined to it through corner contacts, one 8-connected object; ids, hashes, boundaries and adjacency are unchanged, the node line of the system prompt names the field, and the module is spliced into the sandbox so both views carry it. The offline test shows that groups equal the 8-connected objects on a synthetic board and on all 184 human level starts. A start holds such an object when a group of two or more nodes has a bounding box of at least 3 by 3 cells: 76 of the 184 starts, in 13 of the 22 replayed games, do (73 with sparse dot lattices, fill under 0.1, dropped). The gate record's earlier count, 71 starts in 13 of 21 games under a dot-texture rule it does not state, is not reproduced and is not used.

Scroll. The diff tool detects a whole-interior shift along one axis: a 4-cell edge band is left out, candidates come from whole-line matches, and a shift is accepted at a cell mismatch rate of at most 0.1 and at most 0.3 of the unshifted rate over at least 60 changed cells. It shifts the earlier frame

and its objects by the shift, adds `scroll: [rows, cols]` to the output (unchanged when nothing scrolls), and the turn opener says when the last sequence of frames scrolled and by how much. The offline test covers a synthetic scroll, a plain move that stays unchanged, the sandbox copy, the opener line and every human-replay frame pair. In the 22 human-replay games it fires on 103 of 363 changed pairs in bp35, on 40 of 772 in lf52, and on none of the 5,178 in the other 20 games (6,170 silent pairs in all); a synthetic scroll leaves the player and the status bar as the only changes, where the unpatched diff reports 76 moved objects.

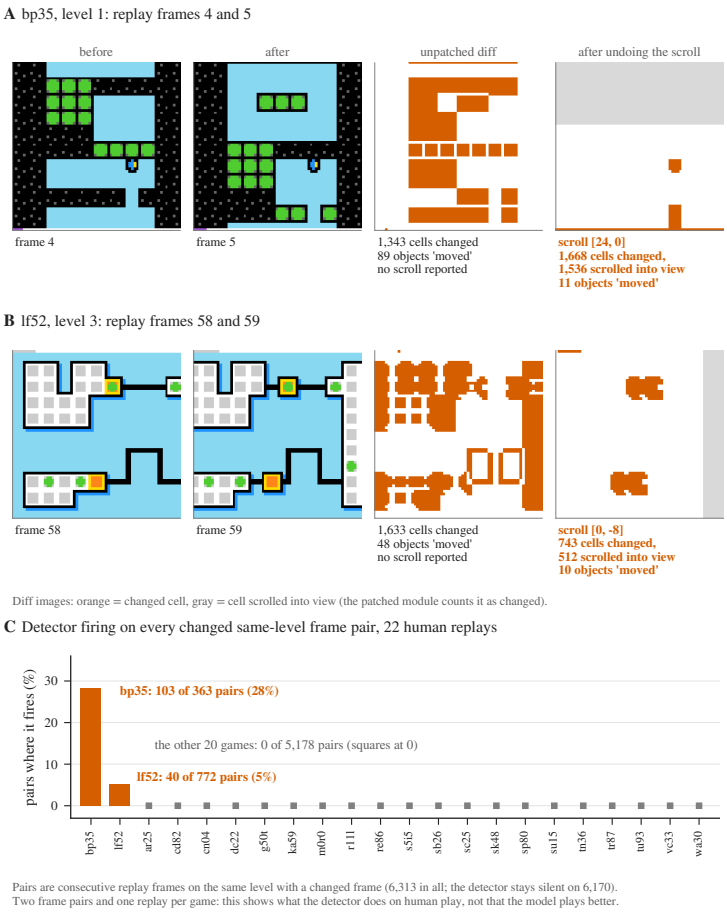


Figure 15: A camera scroll in the diff, before and after. (A, B) Two human-replay frame pairs, bp35 level 1 and lf52 level 3: before, after, the unpatched diff and the diff after undoing the detected shift. In bp35 the unpatched diff shows 1,343 changed cells and 89 moved objects; with the shift undone, 11 objects moved (the cell count rises because the rows that scrolled into view count as changed). (C) The detector fires on 103 of 363 changed pairs in bp35, 40 of 772 in lf52 and 0 of 5,178 in the other 20 games. The pairs are chosen examples, not a sample, and the figure does not show that the report helps the model play (Figure 8 says it did not).

Tool fixes. In the sandbox child a timer on the snippet's own compute (time spent waiting on the host for an action or animation reply is not counted, and a timer that fires during such a wait is deferred, so a reply is never cut) raises `SnippetTimeout` at the compute limit minus 3 seconds; it derives from `BaseException` so an `except Exception` clause in model code does not swallow it (a bare `except` that does is stopped again a second later), and the child returns the printed output, the traceback line and the retained functions instead of the host's empty "Tool timed out". `inspect.getsource` is patched to return the stored source of a function retained from an earlier snippet or defined in the current one. The system prompt says both. The offline test runs the real child: a loop stops at 3.1 seconds of a 6-second limit with its output kept (unpatched: timed out at 6.0 seconds, empty output), a swallowed first stop is stopped by the second, and a 3-second host reply against a 1-second compute limit is never cut. A loop that swallows `BaseException` every time is not stoppable by the child; the host's hard deadline still applies.

Replay check. Two helpers in the sandbox. `check_model(step, observe=None)` takes the model's own transition function and, when `observe` is given, a map from a recorded frame to the model's state (or `step` takes the frame itself and returns its board text or a dict of cells; `None` abstains). It replays the current level's recorded transitions newest first, skipping RESET, automatic RESET, death and level-change steps, and prints the first three mismatches with predicted and actual state (an exception raised by the model's code counts as a mismatch), plus a count of board and action pairs seen twice with different results (hidden state: a counter, timer or facing). `check_goal(is_goal)` runs a goal test on every recorded board of the level and returns the indices where it is true; the printed note says a hit means the test is wrong or incomplete, or that board needed one more action, since the winning board itself is never recorded. `action()` is disabled during both, model exceptions are caught and reported, output is capped at 1,400 characters, and each call checks its clock between transitions and stops after 8 seconds, or 2 seconds before the snippet's own tool-fix stop if that is sooner; a single slow step is not interrupted. The system prompt documents both with a one-line adapter, and the stall check and the turn opener name them. The overlay is applied after every other one so its prompt edits find their anchors. The offline test runs nine checks through the real child on both bases (with and without the tool fixes): a correct rule matches every level-2 transition with RESET and death skips counted, a wrong rule names the newest mismatch, a rule that calls `action()` is refused and no action runs, a slow rule stops at the limit before the snippet's own stop, and an ordinary snippet is unchanged.

J.2 Predeclared gates

Table 27 gives the items. Every build faced the same control and the same no-harm bars; every item must pass for a build to ship, which is necessary and not sufficient, and no run is added after the first read. The control is the 15-stream build, which was itself pinned on 2026-10-28 on its own gate (125.5 levels against 124.0, no gain claimed).

Table 27: Predeclared gate items by change. Control: seven runs, mean 123.4 levels (SD 7.8), mean score 54.85 (SD 3.17), 7,105 actions, median actions over human baseline 0.674 on cleared levels. All three gates read "kill on any miss, no further runs after reading". Written 2026-10-30 03:40Z, 2026-10-30 05:39Z and 2026-10-31 12:05Z.

Item	Stall check	Corner-joined regions	Scroll
Mechanism fires	per run, at least 5 stall lines, 5 untested lists and 20 new-type lists; no unused-action line naming an engine action	model code blocks reading `group`: mean at least 15, every run at least 5	scroll lines in bp35 and lf52: mean at least 3, every run at least 1; at most 2 per run elsewhere; the `group` item as before
Model responds	of the third, fifth and seventh idle turns in a row, at least 0.70 act (0.54 before)	the stall check's item, pooled	the same
Actions	actions per run at most 1.10 times the control (7,816); median ratio at most the control's plus 0.05 (0.724)	the same	the same
No harm	mean levels at least 120.9 (control minus 2.5); mean score at least 53.35 (minus 1.5)	the same	the same
Probe	installs and plays, no traceback	the same	at least 25 actions, no traceback

The tool-fix gate (written 2026-10-31 18:09Z) and the replay-check gate (2026-11-01 19:33Z) kept the stall check's response item and the actions, no-harm and probe items. The tool-fix gate added: `getsource` failures at most 2.0 per run on average (7.8 before) and at least 2 `SnippetTimeout` results per run. The replay-check gate added: at least 10 helper calls per run on average, at least 5 in every run, in at least 4 games per run on average; the corner-joined and tool-fix items; and mean levels at least the replaced build's mean minus 5 (122.3 minus 5).

J.3 Every run read

Tables 28 and 29 give each run. Levels are of 183; score is the mean game score; ratio is the median of actions over the human baseline on cleared levels. Stall, untested and new are the number of openers carrying each list; idle acts counts the third, fifth and seventh idle turns that were followed

by an action; streaks counts levels with five or more idle turns in a row. The first arm ran only two runs because its rule killed it.

Table 28: Play in every run of the program (25 demonstration games). The two Kaggle controls ran on 2026-10-29; the five Vertex AI controls are the 15-stream build's own waves. The two Kaggle controls alone cleared 123 and 110 levels (mean 116.5).

Build	Run	Levels	Score	Actions	Ratio
Control	Vertex, run 1 (kva)	129	57.66	7,050	0.694
Control	Vertex, run 2	120	52.42	7,008	0.645
Control	Vertex, run 3	125	55.98	6,766	0.696
Control	Vertex, run 4	122	52.89	7,519	0.705
Control	Vertex, run 5	135	59.71	8,639	0.639
Control	Kaggle, run 1	123	54.63	7,147	0.644
Control	Kaggle, run 2	110	50.66	5,608	0.698
Stall check, first build	Kaggle, run 1	131	58.55	7,797	0.636
Stall check, first build	Kaggle, run 2	127	58.58	6,643	0.615
Stall check and coverage	Kaggle, run 1	141	65.48	7,713	0.616
Stall check and coverage	Kaggle, run 2	126	55.11	6,753	0.660
Stall check and coverage	Kaggle, run 3	118	54.83	5,683	0.584
Plus corner-joined regions	Kaggle, run 1	132	63.16	6,338	0.618
Plus corner-joined regions	Kaggle, run 2	128	59.97	6,695	0.686
Plus corner-joined regions	Kaggle, run 3	127	58.86	6,260	0.667
Plus scroll detector	Kaggle, run 1	111	48.67	5,735	0.591
Plus scroll detector	Kaggle, run 2	122	53.31	7,335	0.725
Plus scroll detector	Kaggle, run 3	130	60.31	7,369	0.646
Groups plus tool fixes	Kaggle, run 1	123	55.69	6,585	0.659
Groups plus tool fixes	Kaggle, run 2	129	57.96	6,603	0.655
Groups plus tool fixes	Kaggle, run 3	115	50.25	8,108	0.667
Plus replay check	Kaggle, run 1	128	59.12	7,078	0.614
Plus replay check	Kaggle, run 2	124	55.67	6,863	0.729
Plus replay check	Kaggle, run 3	128	55.44	7,009	0.693

Table 29: Mechanism counters in every run of the changes (none exists for the control). Idle acts: third, fifth and seventh idle turns followed by an action, of all such turns. Scroll lines count the openers saying that the view scrolled, in bp35 and lf52 (none appeared in any other game). Reference for streaks: 2.75 per run in the four runs diagnosed in Sec. 6.

Build	Run	Stall	Untested	New	Idle acts	Streaks	`group` blocks	Scroll lines
Stall check, first build	1	14	0	0	13 of 14	1	0	0
Stall check, first build	2	23	0	0	14 of 23	2	0	0
Stall check and coverage	1	20	46	107	16 of 20	0	0	0
Stall check and coverage	2	17	45	96	15 of 17	1	0	0
Stall check and coverage	3	10	38	88	10 of 10	0	0	0
Plus corner-joined regions	1	13	39	97	9 of 13	0	39	0
Plus corner-joined regions	2	18	50	98	16 of 18	1	64	0
Plus corner-joined regions	3	8	33	93	8 of 8	0	33	0
Plus scroll detector	1	19	57	86	15 of 19	1	49	21
Plus scroll detector	2	24	66	90	18 of 24	1	41	29
Plus scroll detector	3	15	61	101	9 of 15	1	49	21
Groups plus tool fixes	1	22	78	91	18 of 22	2	46	0
Groups plus tool fixes	2	25	60	101	20 of 25	3	53	0
Groups plus tool fixes	3	21	62	90	16 of 21	4	47	0
Plus replay check	1	12	51	96	11 of 12	1	59	0
Plus replay check	2	26	63	94	17 of 26	2	39	0
Plus replay check	3	20	57	98	17 of 20	1	59	0

Pooled readings. The stall check and coverage build: 41 of 47 idle turns act (0.87), 0.33 streaks per run, 6,716 actions per run, ratio 0.620, mean levels 128.3 and mean score 58.48, every item passed. With corner-joined regions: 33 of 39 act (0.85), 0.33 streaks, 6,431 actions, ratio 0.657, 129.0 levels and 60.66 points, every item passed. With the scroll detector: 42 of 58 act (0.72), 1.00 streaks, 6,813 actions, ratio 0.654, 121.0 levels and 54.10 points; every item passed, the no-harm items by 0.1 level and 0.75 points (bars 120.9 and 53.35). It was not promoted: its mean sits

below the control (123.4) and below the build it would replace (129.0), with no gain in the games it targets (bp35 at one level in all three runs, lf52 at two, three and four). The gate text makes passing necessary and not sufficient; the record says promote false. Groups plus tool fixes: 54 of 68 act (0.79), 3.00 streaks, 7,099 actions, ratio 0.660, 122.3 levels and 54.63 points; `getsource` failures 0, 0 and 0 per run (7.8 before), `SnippetTimeout` results 6, 5 and 6, "Tool timed out" 0, 0 and 0 (6.0 before); every item passed; promoted 2026-11-01. Plus replay check: 45 of 58 act (0.78), 1.33 streaks, 6,983 actions, ratio 0.679, 126.7 levels and 56.74 points; helper calls 33, 50 and 18 per run in 6, 6 and 5 games; `check_model` printed 85 reports, 33 matching every checked transition and 52 naming a mismatch (42 a wrong prediction, 10 only exceptions from the model's own code, such as a `NameError` in its adapter; 8 stopped at the time limit), and `check_goal` printed 25, none with a counterexample; `getsource` failures 0 in every run, `SnippetTimeout` 6, 3 and 3; in its 300 seconds per game the probe called a helper in one game (tu93); every item passed, including the bar of 117.3 against the tool-fix build; promoted 2026-11-02 00:28Z for the final draw. The first stall build: 27 of 37 act (0.73), 1.50 streaks, 7,220 actions, 129.0 levels and 58.57 points; its untested-list counter read zero against a bar of 5, so its rule killed it.

Readings declared and not computed. The replay gate also listed readings to report but not gate on: levels on a window aligned to server readiness, and whether a mismatch was followed by a redefined rule within two turns. Neither was computed for this paper.

A start-time confound. Franzen's notebook starts play when the server reports ready, or at a deadline about 720 seconds after the notebook starts with the server still loading; games released at the deadline wait on a server that cannot answer while their clock runs. Of 23 Kaggle replica runs of the 15-stream family, 13 were released at the deadline and 6 started 12 to 21 minutes before the server was ready: one of the two Kaggle control runs, one of three stall check runs, two of three corner-joined runs, one of three scroll runs, one of three tool-fix runs and none of the three replay-check runs, whose latest start was one minute early; the five Vertex control runs were not read. The offset therefore sits unevenly in the arms; it could flatter the replay build against the others and understate the corner-joined build, and we did not estimate its size.

Leads over the control, with their uncertainty. Against the control's 123.4 levels the stall check build leads by 4.9, the corner-joined build by 5.6, the replay build by 3.2 and the first stall build by 5.6; the scroll build trails by 2.4, and by 8.0 the build it extends, and the tool-fix build trails by 1.1. At the identical-run SD of 8.25 and three runs against seven, a 95% interval is the lead plus or minus 11.2, so every interval contains zero: stall check -6.3 to $+16.1$, corner-joined -5.6 to $+16.8$, replay -8.0 to $+14.4$, scroll -13.6 to $+8.8$, tool fixes -12.3 to $+10.1$; the first stall build, two runs against seven, -7.4 to $+18.6$. Against the two Kaggle controls alone (116.5 levels; both ran on Kaggle, as the arms did) the leads read $+11.8$ (stall check), $+12.5$ (corner-joined), $+10.2$ (replay) and $+5.8$ (tool fixes), and the scroll build $+4.5$; the five Vertex AI controls average 126.2, so the control sets differ by 9.7 levels, and two runs cannot say whether that is platform or noise. The same three-against-seven design resolves a shift of about 16 levels at 80% power. On score the control's SD over seven runs is 3.17, and the stall check, corner-joined and replay builds lead by 3.6, 5.8 and 1.9 points; the scroll build trails by 0.8 and the tool-fix build by 0.2.

J.4 Earlier arms on the same agent

Two earlier arms ran on the 15-stream build in the same weeks and are not part of the program. A kept-variables overlay (plain data kept for the length of a level so the model does not rebuild it after each action) was built to halve NameErrors per python call; two versions read 0.51 and 0.52 against a bar of 0.50, were killed by their own rule and did not move levels (pooled 127.8 and 125.3 against 125.7 and 123.4). A shorter-thinking model (ThinkingCap) cleared 122 levels against 129 for the build beside it on Kaggle and took 70% more actions, so its game score fell 15%; it was killed, and its Vertex wave was cancelled.

K The research pass behind the replay check

This appendix records the research pass behind the replay check of Sec. 6; Appendix J has the build and its runs.

Method. On 2026-11-01 five agents each took one angle: public ARC-AGI-3 harnesses, the agent literature, a first-principles read of our harness code, a quantitative read of our transcripts, and the competitive meta. A synthesizer merged their 26 proposals into a model of where the clock goes, five ranked candidates (each with a spec, an offline test and a gate metric) and a list of 15 rejections, some of them merged or folded into candidates and three from public evidence. One skeptic per top-four candidate re-measured its evidence on our runs and returned an objection, an adjusted effect and build pitfalls: ten agents in all. Every skeptic put the level effect below what three runs can resolve, so we built one candidate and gated it on mechanism and no harm.

What sets the clock. We recomputed the transcript angle's headline numbers on the 11 Kaggle runs of the 15-stream family it read (275 game plays, 16,750 turns; Figure 7) with `paper/v3/analysis/discovery_read.py`. Execution is cheap. Of 7,494 responses that ran three or more actions, 7.2% were stopped by the no-op or stale-state guard and 0.9% ended in a game over. Cleared levels took a median 0.64 of the human baseline's actions, 8.2% of tool calls returned an error, and 73% of level openings (level 2 and later) call a function written on an earlier level. Discovery is not. Later levels that took over 15 minutes are 27% of later-level clears and hold 66% of their clock. On those levels, code that resembles the clearing turn's (identifier Jaccard index at least 0.5) first appears with a median 13% of the level's clock left; the proxy matched 94 of 272 levels. On cleared later levels, 17.3% of the clock passes before the level's first action, and turns that take no action are 14% of all turns and 30% of the clock. Caveat: the transcript rule needs a later turn at a higher level, so it misses a level cleared on a game's last turn (1,016 later-level clears against the benchmark's 1,113). The pass concluded that the clock per level is set by how fast a wrong rule or a wrong goal dies, and that the cheapest counterexample is one the recorded history already holds.

The candidates. All four skeptics refuted their candidate as a level lever three runs could see (Table 30). We built the replay check anyway: four of five angles proposed it, and it computes something for the model instead of adding advice.

Table 30: The five ranked candidates. Effects are the skeptic's adjusted levels per 25-game run. The key facts are the skeptics' own measurements on three or more of our runs and were not rerun; the replay check's predictions are checked against its runs below.

Candidate	What it changes	Skeptic, effect	Key fact behind the verdict	Outcome
Replay check	Two sandbox helpers: replay the level's recorded transitions through the model's simulator, and test a goal guess on every recorded board; plus a host-time prompt clause	Refuted; 0 to $+1$	In three runs, 17 of 75 stuck level plays had a simulator and 3 a check; 8 of 10 false-rule stuck levels fit every recorded transition; an earlier validation helper on this model was used in 1.5% of calls	Built without the clause; promoted 2026-11-02
Code first	After a reply cut at the 12K output cap, the next opener redirects to code or a one- or two-action probe	Refuted; about 0	The turn after a cut already acts: its first call defines a function or acts in 66 of 79; about 1.6% of the clock is addressable; a forced tool call at this moment tied	Not built
Coverage list	At stall and failure openers, list actions and click types tried from only one board state with no effect, plus an UNDO line	Refuted; about 0 (at most $+0.2$)	Its own back-test: 14 of 154 listed items (9%) were later used with effect in a clearing run, against a 20% bar; 1 of 49 on stuck levels	Not built (failed its back-test)
Cost model	Replace the prompt's only objective line (fewest actions while reliable) with the scoring and clock model	Refuted; about 0 (-6 to $+3$)	Reasoning from the objective takes 0.7% of the clock; a uniform 1.15 times rise in actions would cost about 4.5% of score yet pass the proposed guard; deleting the same line in September cleared 30 levels against 35	Not built
Group diff	`frame_diff` reports a corner-joined group whose parts all moved by one offset as one entry	No skeptic	Scroll, the last `frame_diff` change, fired in its target games while bp35 stayed at one level in all three runs	Not built

Rejected without a skeptic. The main rejections, with the synthesizer's reasons. A halt-on-mismatch executor: the model already writes checked loops by hand (445 blocks in three runs), and an earlier host-enforced prediction gate lost 10 of 35 levels. A search helper: search snippets fail more often (15.5% against 7.8%), but the saving is tokens, the agent already runs near its token ceiling (Sec. 4), and the model writes its own search on 170 of 272 long levels. Traceback hints in the style of SWE-agent [Yang et al., 2024a]: tool errors are 8.2% of calls, mostly the model's own bugs. Opener compaction: 71K retained history tied. Level-start object tables: 0.5K to 1.5K tokens per level start to save about 1% of the clock. Cross-level memory: solved-level memory and a strategy audit were already dropped (Appendix D). A stall cutoff and more streams: single public offline runs lost 9.1 points and scored 29.90 against 47.30 [Liang, 2026]. A shorter prompt lost about 12 points on Son Pham's team's own bench [Barney, 2026], and fine-tuning on our own cleared levels lost (Sec. 7).

The built candidate, predicted against measured. *Helper use.* The skeptic predicted low use, after an earlier validation helper on this model that was called in 1.5% of calls. The model wrote 33, 50 and 18 code blocks per run that call a helper, in 6, 6 and 5 games: 1.14% of all code blocks (101 of 8,842), so the prediction held by the precedent's measure, whose denominator may differ. The gate's bar of 10 calls per run passed only because a run has about 2,900 code blocks. *Level effect.* Predicted 0 to $+1$ level per run, with a standard error of about 4.5 levels for three runs; measured 126.67 against the control's 123.43, a lead of 3.2 inside that error, not claimed. *Mechanism.* The skeptic warned that abstract-state simulators would turn every transition into an error without an adapter, and that `check_goal` would have little to check. With a one-line adapter example in the prompt, `check_model` printed 85 reports: 33 matched every checked transition, 42 named a wrong prediction and 10 only exceptions from the model's own code; 8 stopped at the time limit. `check_goal` printed 25 and found no counterexample, as predicted. Not shown: a rule fixed after a mismatch, a false rule passed, or a level cleared because of a report. *Build pitfalls.* The build fixed 10 of the skeptic's 12 pitfalls, left one (the call bar) and did not check one; the call bar stayed at 10 per run, below the hand-validation rate of about 19 to 23 blocks, so it could pass by substitution alone. Two readings the gate made report-only, levels on a window aligned to server readiness and a rule redefined within two turns of a mismatch, were not computed.

The public harnesses. The case for the replay check rested on public harnesses that play ARC-AGI-3 with an executable world model; Table 31 gives what each source's own text supports.

Three caveats matter. Twin's 93.3 against 61.1 keeps the model and agent but removes five parts at once (world model, replay validation, exploration, planning and the halt-on-mismatch executor), with one scored run per game; its goal check ran by prompt in the reported runs (its Appendix D), and no experiment isolates enforced from prompted validation [Skoutnev et al., 2026]. Rodionov's verification variant ranks first in every setting, by 0.62 to 8.56 points over the variant it extends, but it is a prompted bundle (fixed interface, tools, simplification and replay), and the author says the experiments do not show how much comes from verification alone [Rodionov, 2026a]. Kepler instructs the backtest without forcing it and lets an action run when its prediction fails [Wu, 2026]. All six run hosted frontier models; ours is a tool an open-weight model may call, closest to the prompted rows.

What rivals published. As read on 2026-11-01, no team in the top 12 had published what it changed after Franzen's Milestone 2 notebook [Franzen, 2026a]. The Kaggle user xz, whose team was third on the board at 58.59 on 2026-11-01 (fourth at the deadline, behind NVARC3 at 60.95), wrote that it ported its solution to Kaggle on 2026-10-25 and validates on the 25 public games plus a 30-game copycat set made by Team Turducken [xz, 2026]. In another thread it wrote that it starts the games at the 10-minute mark and sends RESET every 7 minutes [Cottaar, 2026b]. In that thread Tufa Labs reports that its scored notebooks lose about 40 minutes, which count against the 9 hours [Cottaar, 2026b]. Team Turducken's expert pruning with a short healing fine-tune and its prompt coaching, and Son Pham's team's 12-point loss with a shorter prompt [Barney, 2026], rest on our repository's records of earlier reads, not on copies saved for this pass.

Corrections to the research itself. Checking the pass against its sources changed six statements.

Table 31: Mechanisms of public ARC-AGI-3 harnesses, from each source's own text (read 2026-11-02). (1) Every frame logged and queryable; (2) a hypothesis checked against all recorded history; (3) planning inside a simulator; (4) plans run step by step, stopping at the first prediction miss; (5) every recorded board treated as a non-goal. Enforced: the harness blocks the action. Prompted or instructed: the model is asked. EWM is read from its abstract and Twin's account. Retrodict [ryanbbrown, 2026], also named for this pattern, is left out: we hold no copy of its text.

System	(1) Log	(2) Check all history	(3) Plan in simulator	(4) Stop at first miss	(5) Non-goals
Twin [Skoutnev et al., 2026]	yes: transition log	enforced before every scored action	yes: BFS in the checked model	enforced executor	yes; prompted in the reported runs, enforced in the released code
Tycho [Lehmann et al., 2026]	yes: typed frames, query library	partial: latest attempt per level, on every save	optional checked route	no: the route is advice	implicit: decision states must stay ongoing
Kepler [Wu, 2026]	yes: daemon log	instructed, not forced	yes: certified per-level programs	yes, but a failed prediction does not block	not stated
arc-skill [Gowtham, 2026]	yes: append-only log	optional rules tier, used once	optional A*	enforced: a written prediction per press, halt at the first miss	not stated
Rodionov ablation [Rodionov, 2026a]	recorded attempts replayable	verification variant only; prompted	yes, in executable variants	no	implicit: status compared at each step
EWM [Rodionov, 2026b]	implied	yes: verifier programs	yes	plan executor	not stated

1. On 2026-11-01 rank 3 was xz at 58.59, not Team Turducken, which was fifth at 56.02 (at the deadline NVARC3 held rank 3 at 60.95, xz rank 4 and Team Turducken still rank 5); one angle read a stale board.
2. The digest's 81% of level openings reusing earlier functions did not reproduce; by our definition (the first turn of a level 2 or later calls a function defined earlier in the game) it is 73%.
3. The digest said public harnesses had converged on five mechanisms; only Twin has all five and enforces the full-history check (Table 31).
4. The digest credited Twin's 32-point gap to enforced replay validation; the ablation is not replay-only, and Twin does not credit enforcement with most of it.
5. The digest's specific EMPA rules [Tsividis et al., 2021] and AutumnBench's reset and no-op shares [Warrier et al., 2025] are in no text we hold; nothing here uses them.
6. The digest put 6 of 19 runs 12 to 22 minutes early, and an earlier draft of this paper said play starts about 728 seconds after the kernel whatever the server's state. Franzen's notebook releases the benchmark when the server is ready, or at a deadline about 720 seconds after start if it is not; 6 of 23 Kaggle runs of the 15-stream family started 12 to 21 minutes before the server was ready, and none of the three replay-check runs by more than a minute (Appendix J); the five Vertex control runs were not read.

L Regeneration and compute

L.1 Scripts from the v2 analysis

Most claims in Appendix B come from a script in `paper/v2/analysis/` that reads recorded runs and writes small text or JSON files; none starts a run or needs a GPU. Seven are read from records or source code instead: A3, D14, E1, E9, F1, F2 and F3 (the time ledger re-derives part of F1). Table 32 lists each script, its output, the claim ids it produces and its runtime. Scripts run on one CPU core with the Python standard library and no network unless marked. The three new analyses and the plateau figure also import the repository's simulator (`scheduler_sim.py`, through `e5_lib.py`)

and read the recorded runs. The only cost is the development run itself: about 2 h 10 min and $4 per 25-game run on the earlier rented VM with the same card (Sec. L.3 has current rates). The leaderboard score cannot be regenerated; it is verified by submission ID (Sec. 8).

Table 32: Scripts, outputs, the claims they produce (Appendix B) and runtimes. Paths under `out/` are in `paper/v2/analysis/`; `evidence/` paths are in `paper/v2/`. Runtimes come from the driver scripts' headers, or were timed on 2026-10-13 where marked.

Script	Output	Claims	Runtime
Replica runs (the only costly input; every script below reads recorded runs and starts none)			
25-game replica run on a GCP G4 spot instance	`benchmark.json`, `serve.log` per run	inputs of every row below	2 h 10 min; $4 per run
Instrument: `e4_run_all.sh`, about 2 min for all			
`e4_inventory.py`	`out/e4_inventory.json`	inputs of the rows below	in driver
`e4_identity.py`	`out/e4_identity.txt`	supports B1	in driver
`e4_noise.py`	`out/e4_noise.txt`, `.json`	B1 to B5	in driver
`e4_protocol.py`	`out/e4_protocol.txt`, `.json`	B5, B6, G3	in driver
`e4_gates.py`	`out/e4_gates.txt`, `.json`	B7	in driver
`e4_serving.py`	`out/e4_serving.txt`, `.json`	B8	in driver
`e4_incidents.py`	`out/e4_incidents.txt`, `.json`	B10, B12	in driver
`e4_wave_recount.py`	`out/e4_wave_recount.txt`	B11	in driver
`e4_board.py`	`out/e4_board.txt`, `.json`	B11, G6	in driver
`e4_varcomp.py`, `e4_stats.py`, `e4_figure.py`	`out/e4_varcomp.txt`, `e4_fig_data.csv`	run-level variance and figure data; no map claim	in driver (varcomp is the slowest)
Scoring theory: `e5_run_all.sh`, about 25 min on one core for all			
`e5_scorer.py`	`out/e5_scorer.json`	A1, A2	in driver; needs the competition wheel
`e5_conversion.py`	`out/e5_conversion.json`	C1, C3, C6	about 8 min
`e5_ceiling.py`, `e5_poolcheck.py`	`out/e5_ceiling.json`, `e5_poolcheck.json`	C2, C3	in driver
`e5_detect.py`	`out/e5_detect.json`	C4, C9	in driver
`e5_curvecheck.py`	`out/e5_curvecheck.json`	C5	in driver
`e5_yield_ceiling.py`, `f1_curves.py`	`out/e5_yield_ceiling.json`, `f1_curves.json`	C7	in driver; `f1_curves.py` about 2 min alone
`e5_ablation.py`, `e5_equiv.py`	`out/e5_ablation.json`, `e5_equiv.json`	C8	in driver
`e5_fluid.py`, `e5_scalelaw.py`, `e5_hazard.py`, `e5_patience.py`	`out/e5_*.json`	supporting checks; no map claim	in driver
Yield: `yield_all.sh`, about 3 min for all			
`yield_runs.py`	`out/yield_runs.md`, `.json`	run table behind D1 to D3	in driver
`yield_stats.py`	`out/yield_stats.md`, `.json`	D1, D3, D4, D7, D8, D10	in driver
`yield_mech.py`	`out/yield_mech.md`, `.json`	D5, D6	in driver
`yield_rd.py`	`out/yield_rd.md`, `.json`	D12	in driver
`yield_pertoken.py`	`out/yield_pertoken.md`, `.json`	D13	in driver
`yield_board.py`, `yield_arm_diff.py`	`out/yield_board.md`, `yield_arm_diff.md`	board context and cell-level diffs of arms; no map claim	in driver
`framing_stats_check.py`	prints only	D1, D2, G5	8 s (timed)
`framing_final_check.py`	prints only	B2, B4, D2, D9, E6	9 s (timed)
Speculative decoding: `run_spec.sh`, about 1.5 min without `spec_vocab.py` (timed)			
`spec_runs.py`	`out/spec_runs.txt`	per-run serving table (input)	31 s (timed)
`spec_step.py`	`out/spec_step.txt`	E4	3 s (timed)
`spec_contrasts.py`	`out/spec_contrasts.txt`	E5, E7	58 s (timed)
`spec_hist.py`	`out/spec_hist.txt`	E2	under 1 s (timed)
`spec_greedy.py`	`out/spec_greedy.txt`	E3	1 s (timed)

Table 32, continued

Script	Output	Claims	Runtime
`spec_model.py`	`out/spec_model.txt`	E6	under 1 s (timed)
`spec_vocab.py`	`out/spec_vocab.txt`	E8	needs the served tokenizer and vocabulary file; uv with torch, offline; not timed
`solution/sim/test_franzen_qsaring.py`	`out/spec_qsaring.txt`	E10	CPU torch from the uv cache; a first run downloads about 1.5 GiB
Leaderboard, components, time and figures			
`e7_board.py`	`evidence/board_draws.csv`, `board_stats.json`	G1 to G5, G7	1 s (timed); `--refresh` re-pulls the Kaggle listing
`e6_pin_components.py`	`evidence/e6_pin_components.json`	A4	under 1 s (timed)
`e6_stack_numbers.py`	`evidence/e6_stack_numbers.json`	replica and board numbers of the component table	not timed
`e6_license_probe.py`, `e6_fetch_license_threads.py`	`evidence/e6_license_probe.json`, `e6_arcagi3_rules.md`, `e6_license_threads.md`	G7 (rules); licences, Appendix M	network, read-only; not timed
`time_ledger.py`	`out/time_ledger.txt`, `.json`	F1, F4 to F11 (time ledger, Appendix G.1)	about 3 s
v2 figure scripts (`fig_*.py`)	`paper/v2/figs/`	v2 figures; v3 redraws its own (below)	matplotlib via uv
Checks on the curve and the noise (Appendix F)			
`new_token_steps.py`	`out/new_token_steps.json`, `tex/sections/tab_steps.tex`	N1 (N1.1 to N1.8)	about 20 s
`new_plateau_pool.py`	`out/new_plateau_pool*.json`, `tex/sections/tab_plateau.tex`	N2, C9	mode `raw` (eight-, ten- and 21-run pools) about 45 min on a busy laptop, the other raw modes not timed; mode `fit`, seconds
`fig_plateau.py`	`paper/v2/figs/fig_plateau.pdf`	N2, C9 (v2 only)	matplotlib via uv; not timed
`new_noise_model.py`	`out/new_noise_model.json`, `tex/sections/tab_noisemodel.tex`	N3-1 to N3-14	about 20 min; `--quick` about 2 min; `--partb` seconds

L.2 Scripts added for v3

Every V3 claim names its source in Appendix B; most were recomputed from the CSV and text files in `paper/v3/evidence/data/`, which were fetched read-only from the Kaggle API, the cloud job API and the run folders. Table 33 lists the scripts. They run with `python3 -I -B` from the repository root, on one CPU core, with the standard library unless marked.

L.3 Compute and cost

Table 34 sums every lane from 2026-10-13 00:00Z to 2026-10-21 01:00Z: about 130 GPU-hours on the RTX PRO 6000 and about $222 to $244 at list prices. Billing was not read, so these are list-price sums, not billed spend; disks, snapshots and longer bucket storage are not costed. Kaggle's scored runs are free and do not draw on the weekly development quota. v2 did not total compute before 2026-10-13; its unit costs were a development run of about 2.2 free Kaggle GPU-hours or about $4 and 2 h 10 min on the rented VM.

The round-A program has its own caps, set by the author: no new round at $500 or more of program spend and a stop at $700; its ledger read about $110 to $115 at 2026-10-21 01:00Z. On 2026-10-20 the Vertex quota was 14 Spot GPUs in each of the two regions that hold our inputs; the most we ran

Table 33: Scripts added for v3 (paths under `paper/v3/analysis/`).

Script	What it does	Claims
`v3_board_trim.py`	board statistics from `board_draws.csv` and the paired decision test's per-state table	V3-B4 to V3-B6, V3-T4, V3-T5
`v3_trim_parser.py`	re-parses first actions of the decision test (dict form)	V3-T8
`forum_topic.py`	fetches one forum topic anonymously (network, read-only)	V3-R4
`gate_sim/par/cmp2.py`	simulates the round-A wave gate on 16 recorded runs	V3-X14
`figs/*.py`	the fifteen v3 figures; each asserts the numbers it plots (matplotlib via uv); the five added on 2026-10-31 read run folders and Kaggle's submission list	Figures 1 to 10, 12, 13, 14, 6 to 15, 11
`harness_read.py`	per-run levels, score, actions and counters of the harness program, and each gate item against the pooled control (the tool-fix and replay arms are read by `solution/analysis/harness_2026-10/gates/`)	V3-W19 to V3-W29
`harness_gates/*.py`	the gate readers of each change (copies of `solution/analysis/harness_2026-10/gates`), run from the repository root	V3-W20 to V3-W27
`discovery_read.py`	where the clock goes per level from the transcripts (long-level share, when the winning code first appears, history checks, plan failures, tool errors), per group; writes `figs/data/discovery_levels.csv` and `discovery_summary.json` (stdlib, about 20 s)	V3-W31
`start_time_read.py`	when play started against when the server was ready in each 15-stream replica run; writes `figs/data/start_times.csv`	V3-W32
`build_claims_appendix.py`	builds Table 4; `--check` lists ids cited without a row	all
`v3_style_check.py`	counts numbers per prose sentence and paragraph, words per section, dashes and record codes in the body	none

Table 34: Compute from 2026-10-13 00:00Z to 2026-10-21 01:00Z, all on the RTX PRO 6000, at list prices. Vertex job-hours are priced at $1.74 (Spot machine and disk) to $2.42 (with the listed training fee, which may not apply to Spot).

Lane	GPU-h	Cost	What ran
Rented VM (Compute Engine Spot)	95.9	$159.54	from 2026-10-16: RE-ARC-3 play data, the rejection-sampling fine-tune and its runs, round-A rollouts and the reference member; 6 preemptions, 45 refused starts
Vertex AI jobs	33.0	$57.45 to $79.90	from 2026-10-20: the calibration run, eight concurrent identical runs, probes, trainer smoke runs, two round-A members
Kaggle development quota	1.0	free	the paired decision test and the keepalive probe
One-off items	n/a	about $5	bucket copy, image builds and pulls, one day of storage
Total	about 130	about $222 to $244	
Unit costs (list prices)			
Development run, rented	2.23 to 2.25	$3.88 to $5.45	2 h 17 min from submission to end
Development run, Kaggle	2.2 to 2.4	free	weekly quota of 30 GPU-hours
Probe, 300 s per game	0.25 to 0.38	$0.44 to $0.92	
Concurrent wave, 12 per arm	24 jobs	$93 to $131	one round
Round-A member, stopped at step 16	2.55	$4.45 to $6.18	a full 57-step member is about $23 to $27 expected
Spot waste	n/a	$15.45	two preempted development runs ($7.46) and checkpoint-free trainer attempts ($7.99)

at once was 8, and Spot stockouts, not quota, limited the day. Two container traps had to be fixed

first: the driver mount was missing from the job's path, and inputs left in the container's page cache slowed the weight load past the notebook's 12-minute deadline until we evicted them. The GRPO trainer also runs as a Vertex job and matched the VM step for step on a smoke run, about 14% slower. The Vertex calibration run and its Kaggle twin cleared 126 and 129 levels, inside one run's spread.

M Licences, release and AI use

We list the licence of every upstream piece, say how our files are licensed, and state the release status as last checked. We make no claim about prize eligibility. The licence survey is from 2026-10-13 and was not re-fetched for this version.

Upstream licences. Table 35 gives what we found on 2026-10-13 on the model cards, repositories and Kaggle pages. Readings of ours are marked. This is not legal advice.

Table 35: Licence of every upstream piece, as found on 2026-10-13. "Our reading" marks an inference, not a finding.

Piece	Licence found	Note
Qwen3.8-Flash-Next [Qwen Team, 2026]	Qwen Community License 1.0 (text of the LICENSE file in the model repository): an MIT-style grant (use, modify, distribute, sublicense, fine-tune, derivatives) with conditions. The copyright and permission notice must travel with copies. The model name must be shown on products above 100M monthly users or US$20M monthly revenue. A separate Qwen licence is needed to run a "Model as a Service" or "AI Work Assistant" business (internal use excepted).	Our reading: a custom licence with field-of-use conditions, not an OSI-approved text.
Intel W4A16 AutoRound [Intel, 2026]	Model card "other / qwen-community-1.0", "follow the license of the original model"; Kaggle copy "Other (specified in description)".	Derivative of the Qwen weights.
albucino drafter [albucino, 2026]	`qwen-community-1.0`; Kaggle copy "Other (specified in description)".	
ursuciprian D3 weights [ursuciprian, 2026]	`qwen-community-1.0`, "the license conditions apply to this derivative"; training tools Apache-2.0.	Our public dataset (version 2) holds the refit tensors unchanged and so inherits this (our reading); version 2 carries the Qwen licence text and a NOTICE with this attribution.
lee-chang-93 REAP-448 checkpoint; REAP method [Lasby et al., 2025]	Checkpoint: NVIDIA Open Model License and Qwen Community License 1.0. Method repository: Apache-2.0.	We use only which expert ids were kept, never their weights.
Pennyroyal SGLang, SGLang, Ratsimbazafy fork	Apache-2.0.	
Olympie fork (low-M GEMM patch)	No licence file.	An upstream problem in Franzen's wheelhouse, not ours.
lovedheart sglang fork (QSA fix)	Apache-2.0.	
Tufa Duck harness [Bessis et al., 2026]	No LICENSE file at the repository root, in `ARC3-Inference` or in `tufa-arc-agi-framework`. The `pyproject.toml` classifier reads "License :: OSI Approved :: MIT License".	A classifier is a claim, not a grant.
Franzen's notebook and patch [Franzen, 2026a]	His repository is Apache-2.0. The notebook is a public Kaggle notebook; our fork's first cell states "Apache 2.0 by default" and keeps the original's terms.	

Piece	Licence found	Note
Franzen's two datasets	Kaggle licence field "Unknown" for both; both public.	The forum asks whether rule 5.a.3 covers "unknown"; no answer.
shiiin9 D-prime file [shiiin9, 2026]	The source notebook is public. Its licence is not recorded in our repository and we did not read it.	Kaggle's default would be Apache-2.0 (our reading).
ARC-AGI repository (arcprize)	MIT.	The competition wheels are separate artifacts.
Our D3 dataset and kernel	Dataset: public, version 2, with the Qwen Community License 1.0 text and a NOTICE (ursuciprian attribution); our 2026-10-13 survey found no licence set. Kernel: private on 2026-11-02; not re-checked since.	See below.

Our files. We will release the files we wrote under CC-BY 4.0, the licence that Kaggle's competition rules apply to a winning submission and its source code; third-party files keep their own licences (Table 36). The 12 overlays behind the 40.98 configuration hold 1,316 lines, and ten of them (988 lines) are ours alone. The ARC Prize page separately asks that code authored by the submitter be released under a permissive public-domain licence such as CC0 or MIT-0 (below); the two texts differ, so our code is offered under either CC BY 4.0 or MIT-0 at the reader's choice, and our paper, figures and data under CC BY 4.0. The submitted kernel, version 1, cannot be edited without a new version, and its first cell says Apache 2.0 by default. Copyright in AI-generated code is unsettled in some jurisdictions, and we have not checked any assistant's terms on output ownership.

Table 36: How our files and the files inside them are licensed. Lines are file lines including docstrings, except the last row (added lines).

Files	Lines	Licence
wastefix, replycache, agentfix, memcap, toolfast, promptobs, watchdog, failclosed, asyncmrope_serve, d3draft_serve	988	CC-BY 4.0 for our lines; the anchor strings they quote keep their owners' licences.
p448_serve	214	CC-BY 4.0 for the code; the table of kept expert ids is derived from third-party models, so credit it.
qsaring_serve	114	Stays Apache-2.0 with its notice (derived from Apache-2.0 code).
Build script and tests	not counted	CC-BY 4.0.
shiiin9's D-prime file	252	Not ours; we cannot relicense it.
Franzen's patch, Duck harness, SGLang, weights	8,405	Third-party; cannot be dedicated by us.

Release status (as of 2026-11-03). The repository is public at github.com/vineethsaivs/arc3-decisions-or-tokens (tag v1, commit c77243c) with a LICENSE file and a THIRD_PARTY list. It holds the paper sources and figures, the analysis scripts and data, the overlays and build script (without the files whose licences we could not confirm), the gate records and the research notes; the figure of the noise ruler and the discovery figure rebuild from it with one command. It leaves out the Tufa Duck bundle, shiiin9's priority file and the expert-id table, so most overlay tests do not run from it alone. The D3 dataset is public as version 2, which carries Qwen's licence text and a NOTICE with ursuciprian's attribution (reported 2026-11-03). The submitted kernel was private on 2026-11-02 and we have not re-checked it: it embeds those files as one escaped string, so a reader cannot audit the code from the notebook.

Release record. Repository github.com/vineethsaivs/arc3-decisions-or-tokens, tag v1, commit c77243c, public from 2026-11-02; anonymous requests for the repository and its LICENSE returned 200 that day. The dataset (version 2) became public afterwards; the kernel was not yet public on 2026-11-02 and is to be re-checked from a logged-out browser. The dataset carries Qwen's licence text and ursuciprian's attribution (version 2). Franzen's four inputs are public today. If one goes private, every submission errors, so a fallback build mounts our copies from a private kernel that would also need to be public to be cited.

Rules. The ARC Prize 2026 page (arcprize.org/competitions/2026) says "all code and methods authored by the submitter must be made open source under a permissive public domain license (eg. CC0 or MIT-0)", and that third-party code must be available under "at least, an open source license which allows public sharing (eg. Apache-2.0, GPLv3)". The same page says "Participants must open source their solutions before receiving official private evaluation scores"; when this applies is not stated. Kaggle's rules for both competitions license a winning submission and its source code under CC-BY 4.0, a second licence text next to CC0 or MIT-0. The Paper Track asks for a Writeup of at most 1,500 words with a cover image and an attached public notebook, on its Kaggle page. A PDF linked from the Writeup has no word limit [Kamradt, 2026b]. A private Kaggle resource attached to a public Writeup "will automatically be made public after the deadline" (2026-11-09 23:59 UTC in the rules as fetched; arcprize.org says papers are due 2026-11-08). The Accuracy criterion matches the submission ID to the notebook.

The Qwen question. Rule 5.a.1.a of the ARC-AGI-3 competition requires an open source system, model and weights as defined by the Open Source AI Definition checklist. Rule 5.a.3 says that input data or pretrained models with an incompatible licence need no open-source grant. The first sentence is absent from the Paper Track rules we fetched (a string search finds none there and one in the ARC-AGI-3 rules); the second appears in both. Forum threads 745079 and 745837 ask whether Qwen Community License weights are eligible. The hosts had not replied in either on 2026-10-13. We draw no conclusion and make no claim about eligibility.

AI use. The author directed the work and selected what went into submissions. Claude Code agents (Anthropic) wrote much of the code, the analyses and the text.

The agent evaluated is the open-weight Qwen3.8-Flash-Next served locally; no external API is called at evaluation time. A scheduled job made the daily Kaggle submissions under the author's standing authorisation. Numbers in the paper come from recorded files and scripts in the repository (Appendix L). AI systems are not authors.